

目录

引言	1
1. RISC-V 服务器平台标准研究	1
1.1. 研究背景	1
1.2. 研究内容	2
1.3. 研究方法	3
1.4. 研究结果	4
2. 服务器基础运行组件适配与优化	5
2.1. 内核适配与优化	5
2.2. 基础服务组件适配与优化	17
2.2.1. 大数据处理软件移植与优化	17
2.2.2. 云计算适配	37
2.2.3. 数据库适配	58
2.3. 基础库适配与优化	71
2.3.1. OpenCV	71
2.3.2. OpenSSL	72
2.3.3. OpenBLAS	74
3. 服务器语言运行时环境支持	75
3.1. Java	75
3.2. Rust	78
3.3. Golang	81
3.4. JavaScript	84
4. PolyOS Server 操作系统发行版	86
4.1. PolyOS Server 操作系统构建	86
4.1.1. 研究内容	86
4.1.2. 研究方案	87
4.1.3. 研究结果	88
4.2. RISC-V 硬件适配	90

4.2.1. 香山处理器适配	90
4.2.2. 智能协处理器适配	93
参考文献	97

插图清单

图 1	RISC-V 服务器平台架构图	2
图 2	protoc 版本信息	21
图 3	protoc-jar 相关 issues	22
图 4	GitHub 信息	22
图 5	Milvus 高度解耦的系统架构	65
图 6	Knowhere 在 Milvus 架构上下游位置	66
图 7	RISC-V SIMD 优化加速对比结果	67
图 8	RISC-V SIMD 对比 ARM NEON 优化加速对比结果	68
图 9	RISC-V SIMD IPC 优化加速对比结果	68

引言

1. RISC-V 服务器平台标准研究

1.1. 研究背景

指令集是集成电路产业中处理器芯片设计的核心基础，其发展受到全球广泛关注。相比传统架构，RISC-V 指令集具备精简、可扩展、开源开放、模块化和通用兼容等优势，为芯片设计和系统开发提供了高度灵活性和可定制性。中国作为全球最大的 RISC-V 应用市场，产业正处于快速发展阶段。国内企业和科研机构，如海思、阿里达摩院等，积极布局高性能处理器与服务器芯片，面向物联网、边缘计算和数据中心等场景推出多款产品。同时，国家相关部门多次发布战略指示，批准专项项目并指导成立标准化组织，为产业发展与国际标准接轨提供有力支撑。

RISC-V 作为开源指令集，正在从学术研究向产业化快速推进，涵盖芯片设计、系统集成和软件开发的完整产业链。国内 RISC-V 产品不断迭代，并在云计算、大数据和人工智能等领域落地应用，教育和科研集群如“东山”、“北海”在技术研发和人才培养方面取得积极成果。然而，目前 RISC-V 服务器缺乏统一技术规范，不同厂商在硬件兼容性、软硬件协同和性能基准等方面存在差异，制约了生态的健康发展。

因此，制定 RISC-V 服务器平台标准势在必行。标准需统一硬件、固件和安全等核心技术要求，定义统一功能集，确保操作系统、虚拟机管理程序及系统软件在不同 SoC 平台间具备一致性和可移植性，降低上下游开发成本，推动产业协同发展。通过标准化设计，可减少生态碎片化，实现核心组件技术规格和平台管理接口的统一，使操作系统和虚拟化软件能够快速移植和优化，形成“一次开发、多平台部署”的良性循环，为企业和运营商提供可靠的技术选型依据，加速 RISC-V 服务器在数据中心和企业级市场的规模化应用。

同时，随着新兴技术快速发展，传统 x86 架构在技术垄断和安全可控方面的局限性日益凸显。RISC-V 服务器平台标准有助于构建自主可控的技术体系，支

撑国产芯片与系统软件发展，并通过建立端到端硬件信任链和安全机制，为关键基础设施提供可信计算保障，满足国家信息安全战略需求。该标准适用于基于 RISC-V 指令集的服务器系统及 SoC 开发与验证，覆盖硬件、固件和安全等关键能力，同时不涉及具体电路实现或非服务器级计算设备，为数据中心、云计算、高性能计算及边缘计算等场景提供统一技术参考^[4]。

1.2. 研究内容

本研究的核心目标是构建面向 RISC-V 服务器平台的统一标准体系，以支撑服务器 SoC 设计、操作系统和虚拟化软件的兼容性、可移植性及安全可信性。研究内容如图 1 虚框所示，主要包括以下四个方面：



图 1 RISC-V 服务器平台架构图

在服务器体系结构与系统芯片设计方面，本研究规定基于 RISC-V 指令集的服务器平台整体体系结构，包括应用处理器与服务处理器的功能划分、指令集扩展以及多核互联结构设计。通过明确服务器 SoC 需支持高性能计算、虚拟化能力及可靠性增强机制，构建具备自主可控、安全可信的服务器级芯片架构，为系统软件和虚拟化平台提供统一硬件基础。

在固件接口与系统描述规范方面，本研究基于 RISC-V SBI v2.0 与 UEFI 规

范，建立统一的固件接口体系，明确系统启动流程、内存映射、安全启动及运行时服务要求，同时采用 ACPI 和 SMBIOS 规范实现硬件资源描述、设备发现及系统信息管理标准化，从而为操作系统及虚拟机管理程序提供一致运行环境，确保不同厂商平台的软件可移植性和一致性。

在外设集成与互联接口标准化方面，本研究定义 RISC-V 服务器平台所需的核心外设及接口规范，包括 UART、XHCI/USB、AHCI/SATA 及 TPM 等标准外设的集成要求。研究强调外设接口的标准化和互操作性，以满足系统调试、高速数据传输及安全信任需求，确保不同厂商平台在外设层面的兼容性与可扩展性，同时降低平台开发成本，形成健康生态。

在安全体系与可信计算环境方面，本研究构建以硬件信任根为核心的安全体系，规范安全启动、固件签名验证、可信度量及安全生命周期管理机制，实现从硬件到固件的端到端信任链保护，保障 RISC-V 服务器平台的安全运行。安全体系标准化不仅提供硬件基础保障，也为软件开发、虚拟化及运维提供明确信任基础，使平台能够满足关键基础设施和高可靠计算任务的安全需求。

综上所述，本研究围绕服务器体系结构与 SoC 设计、固件接口与系统描述、外设集成与互联接口及安全体系四个方面展开，形成从硬件到系统软件的完整标准化框架，为 RISC-V 服务器平台建设提供全面技术参考和实施路径^[2]。

1.3. 研究方法

本研究的目标是构建适用于国产 RISC-V 服务器平台的标准化体系，以确保不同厂商的硬件和系统软件在兼容性、可移植性和安全可信性方面达到统一要求。在方法与过程上，本研究结合国际标准、社区规范与国内实际需求，采用系统化、分阶段的研究策略。

第一，国际标准和社区规范对标分析。本研究充分参考 RVI 社区现有的服务器相关规范，包括 ServerPlatform Spec、ServerSoC Spec 以及 Boot and Runtime Service Spec，梳理其中对服务器 SoC 体系结构、核心处理器配置、指令集扩展、固件接口、系统启动流程及运行时服务的关键要求。通过分析这些规范的技术细节与实现方案，明确国际 RISC-V 服务器标准的技术边界和设计原则，为后续标准制定提供参考框架。

第二，国内产业需求调研与定制化分析。在对 RVI 社区标准进行理解和分析的基础上，本研究联合国内主要 RISC-V 服务器芯片与系统厂商，对其实际应用场景、技术需求、兼容性要求及性能目标进行深入调研。重点关注国产芯片在多核处理器架构、互联网络拓扑、外设集成、固件接口、安全机制以及生态兼容性方面的差异和特定需求。调研结果用于指导标准在保持国际对标的同时，实现对国内市场和产业链的定制化适配。

第三，标准化设计与定制优化。结合国际规范和国内需求，本研究在 ServerPlatform、ServerSoC 及 Boot/Runtime Service 标准基础上，进行定制化增删改。包括对不适用或过于复杂的规范内容进行删减，对国内硬件平台特殊需求进行增加，并对接口规范、启动流程及安全机制进行优化，以满足国产 RISC-V 服务器在可移植性、兼容性和安全性上的具体要求^[3]。在设计过程中，确保定制化调整不破坏标准的统一性和可互操作性，同时兼顾未来技术演进的可扩展性。

第四，厂商验证与反馈。标准初稿完成后，本研究组织国内 RISC-V 服务器厂商进行验证试用，包括核心处理器功能、固件接口、外设集成及安全机制等关键模块的兼容性和适用性验证。厂商在验证过程中提供实际使用反馈，指出潜在问题和改进建议，研究团队根据反馈对标准内容进行修订和优化，确保规范在实际产品和系统软件中能够落地实施，兼顾技术可行性和产业应用需求。

最后，标准发布与生态协同。研究过程中持续收集操作系统、虚拟化平台及中间件开发者的意见和建议，保证标准在产业链各方的协同适配性^[10]。通过“国际标准参考 + 国内需求定制 + 厂商验证与反馈 + 迭代优化”的研究方法，本研究形成科学、系统、可落地的 RISC-V 服务器平台标准，为国内服务器产业的兼容性、可移植性及安全可信性建设提供坚实基础，并推动生态健康发展。

1.4. 研究结果

通过本研究，形成一套完整的 RISC-V 服务器平台标准体系，并推动其作为国内 RISC-V 服务器产业的团体标准（团标）发布，为国内外产业提供系统化、可落地的技术参考。该标准体系将覆盖处理器核心、指令集扩展、内存与 I/O 系统、虚拟化能力、安全体系、固件接口及平台管理等关键能力，为服务器 SoC 设计提供统一的技术基准。在此基础上，标准将明确服务器级处理器核心能力和

指令集扩展的最低要求，实现多厂商处理器之间的一致性与兼容性，确保产业链上下游能够在统一规范下进行开发和部署。

同时，本标准将提出统一的固件接口及系统启动规范，以提升操作系统、虚拟机管理程序及中间件在不同平台间的可移植性和部署效率^[11]。在安全方面，将建立面向关键基础设施的参考架构，包括硬件根信任、安全启动链路、隔离执行环境、加密能力及生命周期管理机制，为云计算、金融、政务等关键应用场景提供可信支撑。此外，标准还将定义平台管理和运维监控规范，包括性能计数器、RAS 能力、热管理及错误处理接口，为数据中心运维提供统一接口，提升整体管理效率和可靠性^[4]。

通过本研究形成的团体标准，不仅能够显著提升 RISC-V 服务器生态的可移植性、兼容性和安全性，降低上下游企业开发成本，还将推动产业协同发展，形成“一次开发、多平台部署”的良性循环。标准的制定将为构建自主可控的服务器体系、推动 RISC-V 服务器产业化落地提供坚实基础，同时为国内 RISC-V 产业在全球标准化进程中占据战略主动提供技术支撑，为国内服务器市场的技术发展和产业生态完善提供系统化参考。

2. 服务器基础运行组件适配与优化

2.1. 内核适配与优化

内核加密算法

1) Linux kernel RAID6 PQ 校验基于 RISC-V 向量扩展优化

RAID 为廉价磁盘冗余阵列（Redundant Array of Inexpensive Disks），RAID 技术将一个个单独的磁盘以不同的组合方式形成一个逻辑硬盘。RAID 技术有很多等级，不同等级有不同的实现和优缺点，但核心都是为了提高磁盘 I/O 性能和保证数据安全性。为保证数据安全可恢复，RAID6 每个条带使用两个奇偶校验块即 PQ 校验。此优化利用 RISC-V 向量扩展计算 PQ 校验，提高计算性能^[5]。

在 RISC-V 64-bit QEMU，香蕉派 BPI-F3，SG2044 服务器上分别测试此优化，可以看到均较标量指令算法有明显性能提升。具体测试数据如下（raid6: rv vx 是 RISC-V 向量扩展指令实现的算法；raid6: int64x 是通用指令实现的算法）：

```

raid6: rvvx1  gen() 1008 MB/s
raid6: rvvx2  gen() 1395 MB/s
raid6: rvvx4  gen() 1584 MB/s
raid6: rvvx8  gen() 1694 MB/s
raid6: int64x8 gen() 113 MB/s
raid6: int64x4 gen() 116 MB/s
raid6: int64x2 gen() 272 MB/s
raid6: int64x1 gen() 229 MB/s
raid6: using algorithm rvvx8 gen() 1694 MB/s
raid6: .... xor() 1000 MB/s, rmw enabled
raid6: using rvv recovery algorithm

```

基于 RISC-V 64-bit QEMU 测试数据

```

raid6: rvvx1  gen() 2916 MB/s
raid6: rvvx2  gen() 2986 MB/s
raid6: rvvx4  gen() 2975 MB/s
raid6: rvvx8  gen() 2763 MB/s
raid6: int64x8 gen() 1571 MB/s
raid6: int64x4 gen() 1741 MB/s
raid6: int64x2 gen() 1639 MB/s
raid6: int64x1 gen() 1394 MB/s
raid6: using algorithm rvvx2 gen() 2986 MB/s
raid6: .... xor() 2 MB/s, rmw enabled
raid6: using rvv recovery algorithm

```

基于香蕉派 BPI-F3 测试数据

```

raid6: rvvx1  gen() 3736 MB/s
raid6: rvvx2  gen() 7168 MB/s
raid6: rvvx4  gen() 10268 MB/s
raid6: rvvx8  gen() 9376 MB/s
raid6: int64x8 gen() 2856 MB/s
raid6: int64x4 gen() 3734 MB/s
raid6: int64x2 gen() 3657 MB/s
raid6: int64x1 gen() 2905 MB/s
raid6: using algorithm rvvx4 gen() 10268 MB/s

```

```
raid6: .... xor() 5806 MB/s, rmw enabled
raid6: using rvv recovery algorithm
```

基于 SG2044 服务器测试数据

内存管理

1) Linux kernel 新增对 RISC-V Svrs60t59b 扩展支持

Svrs60t59b 扩展增加了 PTE 软件保留位 60-59, 于 2025 年 8 月获得批准。

Linux kernel 利用 Svrs60t59b 扩展提供的两个 PTE bit(60-59)可实现 soft-dirty 和 uffd-wp 功能。

2) Linux kernel soft-dirty 新增对 RISC-V 支持

soft-dirty 是 PTE 上的一个 bit 位, 用于跟踪任务写入哪些页面。典型的用到 soft-dirty 功能的项目包括 CRIU(checkpoint-restore functionality for Linux) https://criu.org/Main_Page

3) Linux kernel Userfaultfd write protect(uffd-wp)新增对 RISC-V 支持

Linux kernel Userfaultfd 有两个功能:

其一, read/POLLIN 协议用于通知用户线程发生的故障;

其二, 各种 UFFDIO_* ioctl 可以管理在 userfaultfd 中注册的虚拟内存区域, 从而允许用户空间有效地解决收到的用户错误, 或者在后台管理虚拟内存。

Userfaultfd 支持写保护模式, 可以更精细地控制内存访问和故障处理。使用户能够注册具有写保护的内存范围, 从而在写入操作时触发页面错误^[6]。

虚拟化

1) Linux kvm perf 在宿主机测采集虚拟机信息

为 RISC-V 的 perf 添加基本的 guest 支持, 使其能够区分 PMU 中断是发生在主机还是 guest 中, 然后从主机侧收集一些基本的 guest 信息 (目前不支持 guest OS 调用链)。

。基于 x86/arm 的实现, 并使用 kvm-riscv 进行测试。

测试环境:

- 主机: qemu-9.0.0

- 虚拟机: qemu-9.0.0 --enable-kvm (仅启动一个虚拟机并运行 top)

```

-----
1) perf kvm top
```bash
./perf kvm --host --guest \
 --guestkallsyms=/<path-to-kallsyms> \
 --guestmodules=/<path-to-modules> top
...

...

PerfTop: 41 irqs/sec kernel:97.6% us: 0.0% guest kernel:
0.0% guest us: 0.0% exact: 0.0% [250Hz cycles:P], (all, 4 CPUs)

64.57% [kernel] [k] default_idle_call
 3.12% [kernel] [k] _raw_spin_unlock_irqrestore
 3.03% [guest.kernel] [g] mem_serial_out
 2.61% [kernel] [k] handle_softirqs
 2.32% [kernel] [k] do_trap_ecall_u
 1.71% [kernel] [k] _raw_spin_unlock_irq
 1.26% [guest.kernel] [g] do_raw_spin_lock
 1.25% [kernel] [k] finish_task_switch.isra.0
 1.16% [kernel] [k] do_idle
 0.77% libc.so.6 [.] ioctl
 0.76% [kernel] [k] queue_work_on
 0.69% [kernel] [k] __local_bh_enable_ip
 0.67% [guest.kernel] [g] __noinstr_text_start
 0.64% [guest.kernel] [g] mem_serial_in
 0.41% libc.so.6 [.] pthread_sigmask
 0.39% [kernel] [k] mem_cgroup_uncharge_skmem
 0.39% [kernel] [k] __might_resched
 0.39% [guest.kernel] [g] _nohz_idle_balance.isra.0
 0.37% [kernel] [k] sched_balance_update_blocked_

```

```

averages
 0.34% [kernel] [k] sched_balance_rq
 ...

2) perf kvm record
``bash
./perf kvm --host --guest \
 --guestkallsyms=/<path-to-kallsyms> \
 --guestmodules=/<path-to-modules> record -a sleep 60
 ...

 ...

[perf record: Woken up 3 times to write data]
[perf record: Captured and wrote 1.292 MB perf.data.kvm (1799
0 samples)]
 ...

3) perf kvm report
``bash
./perf kvm --host --guest \
 --guestkallsyms=/<path-to-kallsyms> \
 --guestmodules=/<path-to-modules> report -i perf.data.kvm
 ...

 ...

Total Lost Samples: 0
#
Samples: 17K of event 'cycles:P'
Event count (approx.): 269968947184
#
Overhead Command Shared Object Sy
mbol
.....

```

#				
	61.86%	swapper	[kernel.kallsyms]	[k] def
aault_idle_call				
	2.93%	:6463	[guest.kernel.kallsyms]	[g] do_r
aw_spin_lock				
	2.82%	:6462	[guest.kernel.kallsyms]	[g] mem
_serial_out				
	2.11%	sshd	[kernel.kallsyms]	[k] _ra
w_spin_unlock_irqrestore				
	1.78%	:6462	[guest.kernel.kallsyms]	[g] do_r
aw_spin_lock				
	1.37%	swapper	[kernel.kallsyms]	[k] ha
ndle_softirqs				
	1.36%	swapper	[kernel.kallsyms]	[k] do
_idle				
	1.21%	sshd	[kernel.kallsyms]	[k] do
_trap_ecall_u				
	1.21%	sshd	[kernel.kallsyms]	[k] _ra
w_spin_unlock_irq				
	1.11%	qemu-system-ris	[kernel.kallsyms]	[k] do_t
rap_ecall_u				
	0.93%	qemu-system-ris	libc.so.6	[.] ioctl
	0.89%	sshd	[kernel.kallsyms]	[k] __l
ocal_bh_enable_ip				
	0.77%	qemu-system-ris	[kernel.kallsyms]	[k] _raw
_spin_unlock_irqrestore				
	0.68%	qemu-system-ris	[kernel.kallsyms]	[k] queu
e_work_on				
	0.65%	sshd	[kernel.kallsyms]	[k] ha
ndle_softirqs				
	0.44%	:6462	[guest.kernel.kallsyms]	[g] mem
_serial_in				
	0.42%	sshd	libc.so.6	[.] pt

hread_sigmask	0.34%	:6462	[guest.kernel.kallsyms]	[g] seria
18250_tx_chars	0.30%	swapper	[kernel.kallsyms]	[k] fin
ish_task_switch.isra.0	0.29%	swapper	[kernel.kallsyms]	[k] sch
ed_balance_rq	0.29%	sshd	[kernel.kallsyms]	[k] __
might_resched	0.26%	swapper	[kernel.kallsyms]	[k] tic
k_nohz_idle_exit	0.26%	swapper	[kernel.kallsyms]	[k] sch
ed_balance_update_blocked_averages	0.26%	swapper	[kernel.kallsyms]	[k] _n
ohz_idle_balance.isra.0	0.24%	qemu-system-ris	[kernel.kallsyms]	[k] finis
h_task_switch.isra.0	0.23%	:6462	[guest.kernel.kallsyms]	[g] __no
instr_text_start				
...				

Perf kvm 功能验证

## 2) RISC-V KVM Svvpctc/Zabha/Ziccrse 虚拟化支持

扩展 KVM ISA 的 ONE\_REG 接口，以允许 KVM 用户空间检测并为 Guest/VM 启用 Svvpctc 扩展。KVM RISC-V 允许 Guest/VM 使用 Svvpctc/Zabha/Ziccrse 扩展，因此将它们添加到 get-reg-list 测试中。

## 3) RISC-V KVM Zicbom/BF16 虚拟化支持

zicbom/zicboz 块大小寄存器应取决于主机的 ISA，否则会在 VMM 中引入 ioctl 调用顺序依赖。当底层主机支持 zicbop 和 bfloat16 扩展时，向 KVM 客户机通告这些扩展，并将其加入 get-reg-list 测试中。

## 4) RISC-V KVM 热迁移优化 1: 基于环形缓冲区的脏页跟踪

在 RISC-V 上使能基于环形缓冲区（ring-based）的脏页内存追踪：

- 启用 CONFIG\_HAVE\_KVM\_DIRTY\_RING\_ACQ\_REL，因 RISC-V

属于弱序（weakly ordered）架构；

- 为环形缓冲区的物理页设置 KVM\_DIRTY\_LOG\_PAGE\_OFFSET；
- 在 `kvm_vcpu_kvm_riscv_check_vcpu_requests` 中增加检查，用于判断脏页环是否处于“软满”（soft full）状态；
- 对于退出至用户空间的 vCPU 请求，修改 ``kvm_riscv_check_vcpu_requests`` 函数，使其返回一个值（目前仅返回 0 或 1）。

Event name	Result(without patch)	Result(with patch)
total time	121785 ms	116706 ms
downtime	310 ms	286 ms
setup	1344 ms	904 ms
transferred ram	201247 kbytes	212605 kbytes
throughput	13.69 mbps	15.04 mbps
remaining ram	0 kbytes	0 kbytes
total ram	2162884 kbytes	2162884 kbytes
duplicate	495832 pages	492895 pages
skipped	0 pages	0 pages
normal	49093 pages	51933 pages
normal bytes	196372 kbytes	207732 kbytes
dirty sync count	4	4
page size	4 kbytes	4 kbytes
multifd bytes	0 kbytes	0 kbytes
pages-per-second	17962	17203
precopy ram	192345 kbytes	204106 kbytes
downtime ram	8781 kbytes	8377 kbytes

qemu-kvm 热迁移验证数据

■ **停机时间：**平均停机时间减少了 30.6 毫秒，性能提升了 7.5%。这意味着虚拟机对外服务的实际中断时间变得更短，对业务的连续性保障更有利。

■ **数据传输量：**包括传输的内存、预拷贝内存、普通字节在内的多项数据传输指标平均减少了超过 22%。优化了数据同步机制，极大地降低了网络负载，在带宽受限的环境下效果会非常明显。

### 5) RISC-V KVM 热迁移优化 2: 以小块逐步启用脏页日志

当启用 `DIRTY_LOG_INITIALLY_ALL_SET` 时, x86 仍会对所有大页进行一次写保护; 而 RISC-V 架构则允许用户空间对大页和普通页均逐步实施写保护。

在嵌套虚拟化场景下 (RISC-V KVM 运行于 [Intel® Core™ i5-12500H] 主机上的 QEMU 虚拟机内), 对一个使用不同后备页大小的 2 GB Linux 虚拟机进行了测试, L2 层 QEMU 中 `memory_global_dirty_log_start` 所耗时间如下所示:

页面大小	优化前	优化后
4k	4490.23 ms	31.94 ms
2M	48.97 ms	45.46 ms
1G	28.40 ms	30.93 ms

memory\_global\_dirty\_log\_start 函数耗时

除耗时显著降低外, 最大的收益在于: 启用脏页日志后, 可最大限度地减少对客户机性能的副作用 (例如, 避免因拆解大页及标记 `memslot` 为脏而引起的开销)。

### 6) RISC-V KVM Perf 功能优化: 完善虚拟机侧中断事件的采集

对于 RISC-V 上的 `'perf kvm stat'`, 为避免出现 `'UNKNOWN'` 事件名称, 除异常 (exceptions) 外, 还应上报中断 (interrupts) 事件。

应用补丁前:

testing without patch:				
---				
Event name	Samples	Sample%	Time(ns)	
-----	-----	-----	-----	
STORE_GUEST_PAGE_FAULT	1496461	53.00%	889612	
544 UNKNOWN	887514	31.00%	272857968	
LOAD_GUEST_PAGE_FAULT	305164	10.00%	189186	
331 VIRTUAL_INST_FAULT	70625	2.00%	134114260	

0 45	<b>SUPERVISOR_SYSCALL</b>	<b>32014</b>	<b>1.00%</b>	<b>5857711</b>
	<b>INST_GUEST_PAGE_FAULT</b>	<b>1</b>	<b>0.00%</b>	<b>25</b>

优化前 perf kvm stat 监控结果

应用补丁后:

<b>testing with patch:</b>				
---				
<b>Event name</b>	<b>Samples</b>	<b>Sample%</b>	<b>Time(ns)</b>	
-----	-----	-----	-----	
<b>IRQ_S_TIMER</b>	<b>211271</b>	<b>58.00%</b>	<b>738298680600</b>	
<b>EXC_STORE_GUEST_PAGE_FAULT</b>	<b>111279</b>	<b>30.00%</b>	<b>1307259</b>	
<b>14800</b>				
<b>EXC_LOAD_GUEST_PAGE_FAULT</b>	<b>22039</b>	<b>6.00%</b>	<b>254414</b>	
<b>80600</b>				
<b>EXC_VIRTUAL_INST_FAULT</b>	<b>8913</b>	<b>2.00%</b>	<b>21015381</b>	
<b>600</b>				
<b>IRQ_VS_EXT</b>	<b>4748</b>	<b>1.00%</b>	<b>10155464300</b>	
<b>IRQ_S_EXT</b>	<b>2802</b>	<b>0.00%</b>	<b>13288775800</b>	
<b>IRQ_S_SOFT</b>	<b>1998</b>	<b>0.00%</b>	<b>4254129300</b>	

优化后 perf kvm stat 监控结果

### 7) RISC-V KVM Memslot 检索优化

在 `user\_mem\_abort()` 中无需再次检索内存 slot，因为对应的内存 slot 已由调用者传入，此举可节省若干 CPU 周期。例如，在对由 2MB huge tlb 页面支持且写保护的 1GB 内存执行写操作时，耗时从 928ms 降至 864ms，减少了 6.8%。

此外，改用 `\_\_gfn\_to\_pfn\_memslot()` 替代 `gfn\_to\_pfn\_prot()`，以避免因重复定位内存 slot 而引入的不必要开销：该 slot 始终固定，即便在大页情形下对 `@gfn` 进行调整亦不改变。

应用补丁前:

```
root@Ubuntu-riscv64:~/repo/kvm# ./memslot_perf_test
```

Random seed: 0x6b8b4567

Testing map performance with 1 runs, 5 seconds each

Memslot count too high for this test, decrease the cap (max is 8209)

Testing unmap performance with 1 runs, 5 seconds each

Test took 21.205076818s for slot setup + 15.641792656s all iterations

Done 1 iterations, avg 15.641792656s each

Best runtime result was 15.641792656s per iteration (with 1 iterations)

Testing unmap chunked performance with 1 runs, 5 seconds each

Test took 22.570215321s for slot setup + 16.032879401s all iterations

Done 1 iterations, avg 16.032879401s each

Best runtime result was 16.032879401s per iteration (with 1 iterations)

Testing move active area performance with 1 runs, 5 seconds each

Test took 21.487109664s for slot setup + 5.007168232s all iterations

Done 717 iterations, avg 0.006983498s each

Best runtime result was 0.006983498s per iteration (with 717 iterations)

Testing move inactive area performance with 1 runs, 5 seconds each

Test took 22.747851104s for slot setup + 5.024175100s all iterations

Done 146 iterations, avg 0.034412158s each

Best runtime result was 0.034412158s per iteration (with 146 iterations)

#### 优化前 memslot 开销情况

应用补丁后:

```
root@Ubuntu-riscv64:~/repo/kvm# ./memslot_perf_test
Random seed: 0x6b8b4567
Testing map performance with 1 runs, 5 seconds each
Memslot count too high for this test, decrease the cap (max
is 8209)

Testing unmap performance with 1 runs, 5 seconds each
Test took 22.066499841s for slot setup + 17.677237464s all
iterations
Done 1 iterations, avg 17.677237464s each
Best runtime result was 17.677237464s per iteration (with 1
iterations)

Testing unmap chunked performance with 1 runs, 5 seconds
each
Test took 24.212600900s for slot setup + 17.921813923s all
iterations
Done 1 iterations, avg 17.921813923s each
Best runtime result was 17.921813923s per iteration (with 1
iterations)

Testing move active area performance with 1 runs, 5 second
s each
Test took 23.191738657s for slot setup + 5.020011846s all it
erations
Done 706 iterations, avg 0.007110498s each
Best runtime result was 0.007110498s per iteration (with 706
iterations)
```

Testing move inactive area performance with 1 runs, 5 seconds each  
Test took 25.176475825s for slot setup + 5.003838244s all iterations  
Done 142 iterations, avg 0.035238297s each  
Best runtime result was 0.035238297s per iteration (with 142 iterations)

优化后 memslot 开销情况

## 2.2. 基础服务组件适配与优化

### 2.2.1. 大数据处理软件移植与优化

#### 2.2.1.1. Hadoop

##### 研究方法

本研究围绕 Apache Hadoop 3.4.0 在 RISC-V (riscv64) 架构上的原生编译与运行适配问题展开，研究目标是在不依赖 x86/ARM 预编译产物的前提下，构建一套可复现、可维护的完整构建流程，并验证 Hadoop 全量 Native 组件在 RISC-V 平台上的可编译性与可用性。

##### 研究方法

本研究采用工程实证与源码级适配相结合的方式。首先以 Hadoop 3.4.0 官方发布的构建依赖要求作为基线，对 Java 工具链、Maven 构建系统、C/C++ 编译环境及各类 Native 依赖库进行逐项分析。在此基础上，结合 RISC-V 平台在软件生态中普遍存在的预编译产物缺失问题，系统性地采用“源码本地编译 + 本地 Maven 仓库注入”的方法替代远程依赖拉取。同时，通过对 Hadoop 及其关键依赖源码进行补丁级修改，解决构建过程中暴露出的架构识别、编译器兼容性以及 Native 功能缺失等问题。整个过程以可重复构建为核心约束，确保在相同环境下能够稳定复现构建结果。

首先对 Hadoop 3.4.0 官方依赖体系进行了梳理。官方文档要求构建环境基于 Unix 系统，使用 JDK 1.8 和 Maven 3.3 及以上版本，并在启用 Native 编

译时依赖 Boost、Protocol Buffers、OpenSSL、zlib、CMake、FUSE、ISA-L、PMDK 等组件。同时，YARN UI v2 的构建还引入了 Node.js、Yarn 和 npm 等前端工具链。这一依赖体系在 x86 和 ARM 平台上通常可以通过远程仓库直接获取，而在 RISC-V 平台上则需要进行显式适配。

在 Java 工具链选择上，本研究将官方推荐的 JDK 1.8 替换为 JDK 17。选择 JDK 17 的主要原因在于其在 RISC-V 平台上引入了更完善的架构优化支持，在实际构建 Hadoop 的过程中能够显著缩短编译时间。然而，JDK 17 已移除对 HTML4 格式 Javadoc 的支持，而 Hadoop 3.4.0 源码中仍包含基于 HTML4 的文档注释，导致编译阶段直接失败。为解决该问题，本研究通过补丁方式修复相关 Javadoc 源码，使其符合 JDK 17 的生成要求，从而在不回退 Java 版本的情况下完成构建。

在 Protocol Buffers 依赖方面，Hadoop 的构建流程同时涉及多个主版本的 Protobuf，包括 2.5.0、3.7.1 以及 3.21.x 系列。由于 RISC-V 平台无法像 ARM 平台一样从远程仓库获取已编译好的 protoc 工具和相关库文件，所有所需版本均采用源码方式进行编译安装。针对 Protobuf 3.7.1 在 RISC-V 平台上编译速度慢且缺失 abseil 支持的问题，本研究引入并编译了较新的 Abseil C++ 库，并将其集成到 Protobuf 3.21.12 的构建过程中。最终选择 Protobuf 3.21.12 作为 Hadoop 编译时的默认版本，同时保留 Protobuf 2.5.0 和 3.7.1 以满足 Hadoop 构建过程中对历史版本的依赖需求。所有 protoc 可执行文件均通过 Maven install 方式注入本地 Maven 仓库，并使用 classifier 标明 riscv64 架构。

围绕 Hadoop Native 模块的构建需求，本研究系统性地编译并安装了 Boost、zstd、ISA-L 以及 PMDK 等关键组件。其中，PMDK 的构建进一步依赖 kmod、ndctl、daxctl 以及 pandoc 等工具链组件，这些依赖同样在 RISC-V 平台上通过源码或已验证的本地二进制方式完成安装。通过该方式，HDFS、MapReduce 及其他 Native 模块在编译阶段能够正确链接所需的本地库。

在构建工具链和依赖解析层面，由于 leveldb 相关组件、grpc-java 以及 Node.js 和 Yarn 等工具在远程仓库中缺乏 riscv64 架构的发布版本，本研究采用手动注入本地 Maven 仓库的方式解决依赖解析问题。leveldbjni、leveldb-java 以及 hawtjni-runtime 等组件通过系统包管理器安装后，明确指定版本并安装至本地 Maven 仓库。grpc-java 1.53.0 则通过补丁方式增加对 riscv64 架构的支

持，并在本地完成构建。针对 YARN UI 构建对 Node.js 和 Yarn 版本的强约束，本研究通过本地仓库伪装版本号的方式，使构建脚本在不修改核心逻辑的情况下顺利完成依赖解析，从而保证前端组件能够成功构建。

在 Hadoop 源码层面，为实现对 RISC-V 平台的完整支持，本研究引入了一系列补丁。这些补丁用于锁定关键依赖版本、防止 Maven 插件无法识别 riscv64 架构、为 MapReduce NativeTask 模块补充必要的位操作支持、修复 level db 相关依赖的本地路径问题、适配 JDK 17 的文档生成机制，以及解决 OpenSSL Native 模块在构建阶段的链接问题。在完成补丁应用后，使用启用 dist 和 native Profile 的 Maven 构建命令，对 Hadoop 3.4.0 源码进行了完整编译。

### 研究结果

在完成上述环境构建、依赖重构和源码级适配后，Hadoop 3.4.0 可以在 RISC-V 架构上成功完成包含全部 Native 组件的原生编译。构建过程不依赖远程二进制仓库，对网络环境要求较低，且所有关键步骤均可通过明确的源码、补丁和构建参数进行复现。该结果验证了 Hadoop 在 RISC-V 平台上的工程可行性，为后续在国产架构环境中部署大数据基础软件栈提供了可参考的技术路径。

## 2.2.1.2.Spark

### 研究内容

本研究采用工程实证驱动的分层推进方式。首先建立可控的构建环境，在系统层面补齐编译工具链与运行时依赖，并以“先满足生成工具链，再满足上层依赖解析”的顺序推进。其次针对 Spark 构建链中与平台相关的关键节点进行定位，包括 Protobuf 编译器 protoc 的可用性、protoc-jar 体系对 riscv64 的平台探测缺陷、grpc-java 的多架构构建产物生成逻辑缺失，以及 Spark 自身 pom.xml 与 SparkBuild.scala 中版本约束的不一致风险。随后将“远程下载不可用/过慢”视为一等约束，通过引入镜像源与本地仓库优先策略降低网络波动对构建确定性的影响，并在必要处通过脚本级路径与缓存目录调整避免对系统临时目录的污染。最后在 Spark 侧完成构建插件版本调整、Java/Scala 目标字节码版本同步、Protobuf 与 GRPC 版本联动更新等修改，确保 Spark 的构建描述与实际可获得的 riscv64 产物严格一致，从而得到稳定可复现的构建结果。

## 研究方案

从构建环境开始。Spark 的构建包含 SparkR 模块，因此需要安装 R 语言环境；该部分采用源码编译方式完成，并要求在安装前确保依赖项可被正确获取与解析，以避免后续 SparkR 构建阶段出现链式缺失。随后对 Java 版本进行统一配置。由于高版本 JDK 在 RISC-V 架构上通常具备更好的优化支持，实际测试表明 JDK 17 相较 JDK 8 在编译速度上可产生数量级差异，编译耗时可缩短至原先的约五分之一到十分之一。因此本移植工作将 Spark 的构建执行统一在 JDK 17 环境中完成，并将此作为构建环境的基础设定，从而在保证功能等价的同时显著降低整体移植迭代成本。

在代码生成工具链方面，Protobuf 编译器 protoc 是 Spark Connect 等组件构建的关键前置条件。由于 protoc 的编译依赖中包含 abseil-cpp，且不同 Protobuf 版本对 abseil-cpp 的兼容窗口存在限制，本移植工作首先在系统内通过源码方式编译安装 abseil-cpp，并在 CMake 配置中明确指定 C++ 标准以匹配服务器端 GCC 能力。在实施层面，为避免构建产物路径与系统环境不可见而引发的后续链接或查找失败，本工作选择在源码目录内直接执行 CMake 生成与安装流程，并完成系统级安装，使 abseil-cpp 的头文件与库能够被后续 Protobuf 与 grpc-java 编译流程直接检索到。随后拉取 Protobuf v3.25.4 源码并采用 CMake 方式进行构建安装，在构建前将已安装的 abseil-cpp 复制至 Protobuf 的 third\_party 目录，以满足其构建系统对第三方依赖的发现逻辑。完成安装后，将 riscv64 平台编译得到的 protoc 可执行文件作为 Maven 依赖注入本地仓库，并以 linux-riscv64 classifier 标注平台，使后续 Maven 插件能够在不依赖远程仓库预编译产物的情况下解析到正确的工具链。

Protobuf 版本选择是该移植中的关键决策点之一。由于 protoc 的版本命名与 protoc --version 输出之间存在表征差异，构建系统中常见的依赖脚本以 b.c 形式引用工具版本，而外部发布标签通常以 a.b.c 形式标识版本线。实践中，Spark 既受到 Protobuf 主版本线升级的影响，也受到依赖插件生态（尤其是 protoc-jar）对新主版本支持滞后的约束。初始尝试使用更高版本的 protoc 虽可完成编译，但会在后续依赖解析中因版本线与插件兼容性产生失败；而使用较旧版本又会因 abseil-cpp 依赖版本过高导致无法直接编译。综合上述约束后，本移植选择 Protobuf 3.25.4 作为折中点：其一方面处于主版本线内较新的稳定点，另

一方面可避免落入 Protobuf 4 相关插件尚未支持的兼容性陷阱，从而使 Spark 构建链路中与 Protobuf 相关的组件能够形成闭环。

VERSION NUMBER	DATE PUBLISHED	DEPENDS ON	DEPENDED ON	VULNERABILITY COUNT	BROWSE
4.26.0-RC2	2024-02-07	0	4	0	<a href="#">Browse</a>
4.26.0-RC1	2024-01-26	0	8	0	<a href="#">Browse</a>
4.0.0-rc-2	2020-07-21	5	330	3	<a href="#">Browse</a>
4.0.0-rc-1	2020-07-17	5	1	3	<a href="#">Browse</a>
3.25.4	2024-07-25	0	705	0	<a href="#">Browse</a>
3.25.3	2024-02-16	0	15467	0	<a href="#">Browse</a>
3.25.2	2024-01-10	0	9908	0	<a href="#">Browse</a>
3.25.1	2023-11-16	0	17045	0	<a href="#">Browse</a>
3.25.0	2023-11-02	0	1435	0	<a href="#">Browse</a>
3.25.0-RC2	2023-10-19	0	2	0	<a href="#">Browse</a>
3.25.0-RC1	2023-10-17	0	2	0	<a href="#">Browse</a>
3.24.4	2023-10-05	0	3832	0	<a href="#">Browse</a>

图 2 protoc 版本信息

围绕 protoc-jar 体系的适配是第二个关键环节。Spark 编译过程中会触发 protoc-jar-maven-plugin，其核心依赖 protoc-jar 内部存在平台探测类 PlatformDetector.java，该探测逻辑缺失 riscv64 支持，导致在 RISC-V 环境中直接崩溃。针对该问题，移植工作采用“修订后的 jar 包本地替换”策略，即将已添加 riscv64 支持的 protoc-jar 版本以 Maven install 方式注入本地仓库，强制上层插件依赖解析落到本地版本而非远端预编译版本。随后对 protoc-jar-maven-plugin 进行源码级重新编译安装，使其依赖解析链显式绑定到本地安装的 protoc-jar，从而消除平台探测缺陷导致的构建中断风险。下图是该插件(protoc-jar)的官方 issues:

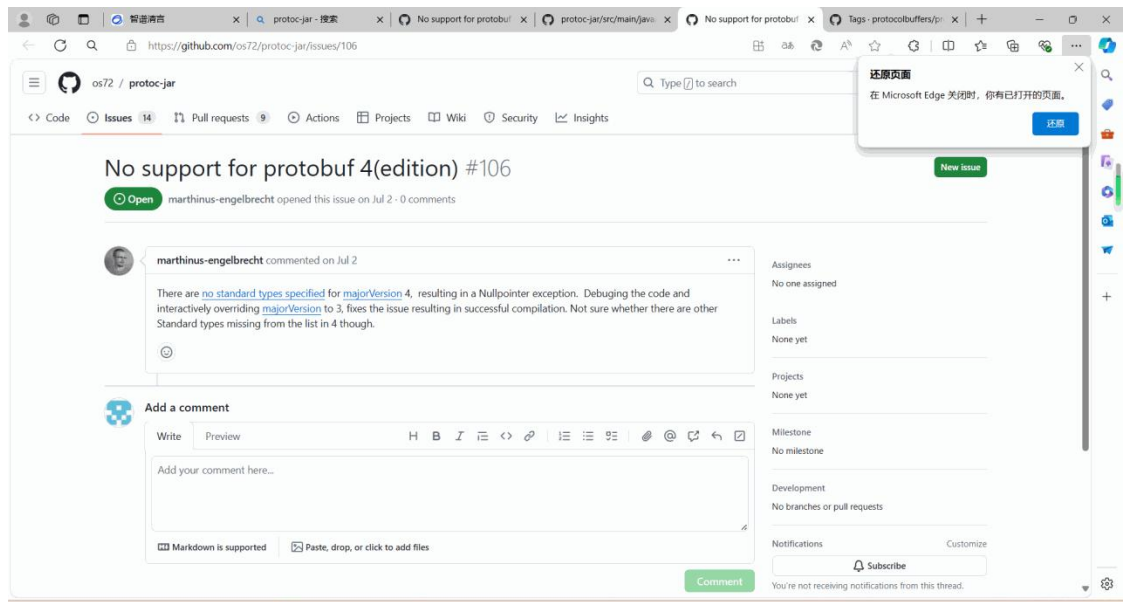


图 3 protoc-jar 相关 issues

原先的 3.23.4 版本，目前来看无法直接编译，会遇到两个问题，其中一个，是因为其的依赖：abseil-cpp 的版本过高。目前知道，3.23.4 是 23 年 7 月发布的，abseil-cpp23 年 1 月 lts 的版本就已经过高了。同时，官方也有人提出此问题，但得到的答复是，用最新的 protobuf，最终版本选择为 3.25.4，Google 公司表明它们的程序具有兼容性。

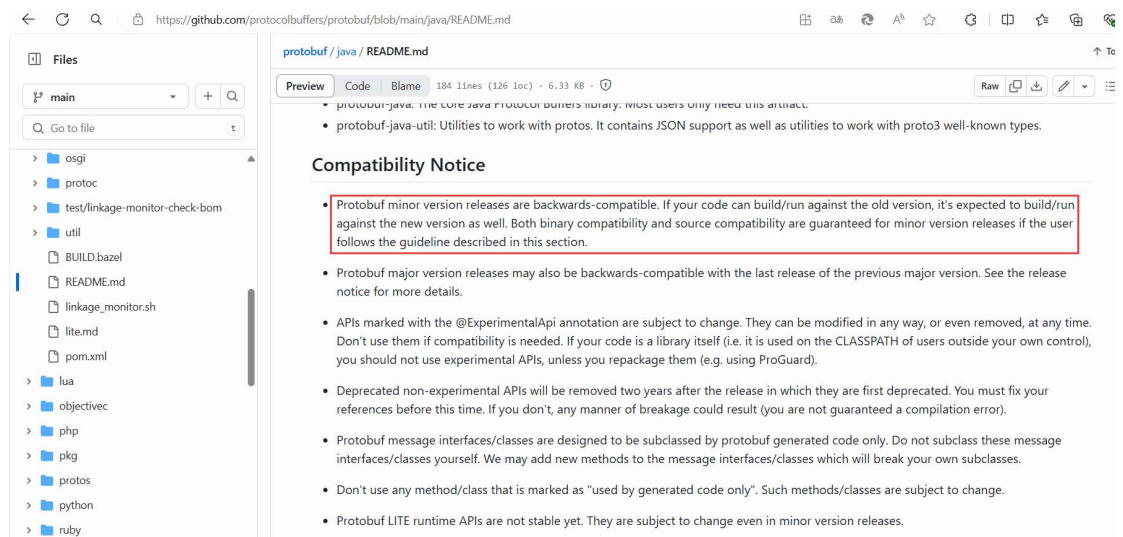


图 4 GitHub 信息

在 grpc-java 依赖方面，Spark 3.5.1 的 Spark Connect 模块通过 project/SparkBuild.scala 固定声明了 protoc-gen-grpc-java 版本为 1.56.0。由于在目标发行

版环境中无法直接获取该版本的 riscv64 预编译产物，移植工作必须通过源码构建得到 riscv64 平台的可执行插件并注入本地 Maven 仓库。为提升构建确定性并降低网络波动影响，首先预下载 Gradle 发行包并将 Gradle Wrapper 的下载地址改为本地 file 协议路径，同时在 grpc-java 的构建脚本中加入本地 Maven 仓库与镜像 Maven 仓库作为优先源。其后对 grpc-java 的 Kokoro 构建脚本、依赖生成脚本与 Gradle 构建配置进行系统性修改，以补齐 riscv64 平台的识别与产物校验逻辑，包括在多架构产物生成流程中加入 riscv64 分支，在依赖构建阶段为 riscv\* 架构引入正确的 configure 参数，并在编译器平台声明中加入 riscv64 target 与架构描述。与此同时，为确保 grpc-java 构建时所使用的 Protobuf 工具版本与前述 Protobuf 3.25.4 保持一致，移植工作显式将 PROTOBUF\_VERSION 对齐到 25.4 并在指定的 protobuf-cache 路径中准备对应版本的 protoc，使构建脚本能够在固定路径中找到预期工具，从而避免脚本自行下载或生成不一致版本导致的不可复现问题。完成构建后，将生成的 protoc-gen-grpc-java riscv64 可执行文件以 linux-riscv64 classifier 注入本地 Maven 仓库，确保 Spark 在构建时能够解析到该平台专用插件。

在 Spark 侧的适配与构建阶段，本研究对构建描述与依赖约束进行了必要修订。首先针对 os-maven-plugin 的兼容问题，将相关模块 pom.xml 中插件版本更新至 1.7.1，以修复在 RISC-V 环境中可能出现的架构识别或插件行为异常。其次由于目标发行版环境中 Maven 版本上限受限，且后续构建链存在 Maven 版本检测，移植工作将 Spark 根 pom.xml 中的 Maven 版本声明调整为 3.6.3，并同步降低 scala-parallel-collections 版本以维持依赖解析与编译一致性。随后将 Spark 的 Java 版本属性由 1.8 更新为 17，并在 scala-maven-plugin 中将目标字节码参数由 jvm-1.8 调整为 jvm-17，避免出现“声明升级但产物仍按旧目标生成”的不一致情况，从而使 JDK 17 的编译性能优势能够在 Scala 编译阶段真实生效。接着将 protobuf.version 从 3.23.4 更新为 3.25.4，并严格遵循 Spark 的版本一致性约束，同步修改 project/SparkBuild.scala 中的 protoVersion 以保持与 pom.xml 中 protobuf.version 一致，同时确认 grpcVersion 与前述本地构建并注入的 grpc-java 版本 1.56.0 一致，避免因版本漂移导致插件下载或代码生成不匹配。最后，为解决国外仓库拉取过慢导致的构建失败风险，移植工

作在 Spark pom.xml 中优先加入华为云 Maven 镜像作为 <repositories> 与 <pluginRepository> 的首位源，从而显著降低依赖下载时间的不确定性并提高构建成功率。

## 研究结果

在完成 R 语言环境准备、JDK 17 统一配置、Protobuf 3.25.4 与 abseil-cpp 的源码构建安装、protoc-jar 生态对 riscv64 的平台探测修复、grpc-java 1.56.0 的 riscv64 产物生成与本地仓库注入，以及 Spark 侧对构建插件版本、Maven 版本、Java/Scala 目标版本与 Protobuf/GRPC 版本联动一致性修订后，Spark 3.5.1 可在 RISC-V 平台通过 dev/make-distribution.sh 成功完成发行包构建流程，并生成面向 riscv64 的可分发 tgz 产物。该移植流程的核心价值在于将 Spark 构建链中对“远程预编译产物”的隐式依赖显式化并替换为“可控的本地构建产物”，从而在 riscv64 生态尚不完备的条件下实现可重复的工程构建闭环，同时保持 Spark Connect 等现代组件对 GRPC/Protobuf 工具链的版本一致性要求，确保最终产物在功能与依赖关系上与原始项目约束保持一致。

### 2.2.1.3.Storm

## 研究内容

本研究围绕 Apache Storm 2.6.3 在 RISC-V (riscv64) 架构平台上的可编译性与可部署性开展，研究内容聚焦于整体性目标与工程边界的明确界定。具体而言，本研究旨在验证在不依赖 x86 或 ARM 架构预编译二进制产物的前提下，Storm 是否能够通过源码方式在 RISC-V 平台完成完整构建，并以官方发行版本等价的形式运行其核心组件。研究内容不以性能优化或功能扩展为目标，而是以“能否构建、能否启动、能否稳定运行”为核心问题，覆盖 Storm 构建所需的基础运行环境、关键外部依赖 Zookeeper 的可用性，以及 Storm 自身源码在 RISC-V 架构下的编译与部署可行性。通过该研究内容的实施，本研究希望为 RISC-V 架构上实时流处理框架的落地提供明确的工程事实依据和可复现的实践参考。

## 研究方案

本研究采用以工程可复现性为导向的分阶段实施方案。方案首先从基础环境

入手，明确 Storm 2.6.3 对 Java 与构建工具的版本要求，并在 RISC-V 平台上构建满足要求的 JDK 与 Maven 环境，以消除工具链不一致带来的潜在风险。在此基础上，方案将 Storm 的外部依赖 Zookeeper 作为独立问题域进行处理，对比系统包管理器直接部署与源码编译部署两种路径在 RISC-V 平台上的可控性与稳定性，并优先选择更有利于路径控制与问题定位的源码部署方式，确保 Zookeeper 能够以可预测的方式启动并为 Storm 提供服务。

在完成外部依赖准备后，研究方案进入 Storm 源码构建阶段。本研究通过对 Storm 2.6.3 源码进行本地编译，统一跳过测试用例以降低构建过程对平台差异的敏感性，同时在构建参数中显式规避可能与硬件架构相关的即时编译或优化行为，以保证生成字节码的通用性。构建完成后，方案进一步对 Storm 的二进制发行包进行打包与解压，形成可直接部署的运行目录。在部署阶段，本研究采用最小化配置策略，通过明确指定 Zookeeper 地址、Nimbus 节点、本地数据目录以及 Supervisor 端口范围，使 Storm 能够在单节点环境中完成组件协同运行。整个研究方案以“Storm 核心组件能够成功启动并对外提供服务”为最终判定标准。

## 研究结果

在严格遵循上述研究方案的前提下，Apache Storm 2.6.3 能够在 RISC-V 平台上完成从源码编译到运行部署的完整流程。本研究在完成环境准备与 Zookeeper 服务启动后，成功对 Storm 源码进行了编译与打包，并在配置完成后依次启动了 Storm 的 Nimbus、Supervisor 以及相关运行组件。通过执行 Storm 提供的版本查询命令，系统能够正确输出 Storm 2.6.3 的版本信息、源码校验值及编译时间，表明当前运行实例由本地源码成功构建并处于可运行状态。

该研究结果证明，在不依赖非 RISC-V 架构预编译产物的条件下，Apache Storm 2.6.3 在 RISC-V 平台上具备工程上的可部署性。本研究形成的编译与部署流程具有明确的步骤边界和良好的可复现性，可作为后续在 RISC-V 架构环境中开展实时流处理系统部署、验证及进一步研究工作的基础。

### 2.2.1.4.Kafka

## 研究内容

本研究围绕 Apache Kafka 3.8.0 在 RISC-V (riscv64) 架构平台上的源码编译与构建可行性展开，研究内容聚焦于整体性目标与工程适配边界的明确界定。本研究旨在验证在 RISC-V 平台上，是否能够在不依赖其他主流架构（如 x86、ARM）预编译二进制产物的前提下，通过源码方式完成 Kafka 的完整构建流程，并生成与官方发行格式一致的可分发构件。研究内容不涉及 Kafka 的功能扩展、性能调优或运行时行为分析，而是以“能否成功构建、构建过程是否可控、构建结果是否可复现”为核心目标，重点关注 Java 运行环境、Gradle 构建工具链以及网络与依赖解析条件在 RISC-V 平台上的适配问题。通过该研究内容的实施，本研究希望为 RISC-V 架构环境中消息中间件的工程化落地提供基础事实与构建参考。

### 研究方案

在研究方案设计上，本研究以工程可复现性和构建稳定性为核心原则，对 Kafka 3.8.0 的构建流程进行系统性拆解与重组。方案首先从基础环境入手，在 RISC-V 平台上通过系统包管理器安装符合 Kafka 3.8.0 构建要求的 Java 运行环境。本研究统一采用 openjdk-17 作为构建与运行时 JDK，以确保与 Kafka 当前版本对 Java 特性的依赖保持一致，同时避免因 Java 版本不匹配引发 Gradle 构建阶段的潜在失败。在完成 JDK 安装后，同步准备 Git、patch 等基础工具，为后续源码获取与补丁应用提供必要条件。

在源码获取阶段，本研究通过官方 Apache Kafka 源码仓库获取 3.8.0 版本源码，确保研究对象的版本来源明确且可追溯。考虑到实际构建过程中可能受到网络环境限制，研究方案将“依赖仓库访问不稳定”作为显式约束进行处理。在 Gradle 构建配置中，本研究通过引入国内可访问的 Maven 与 Gradle 镜像源，替换或补充默认的远程仓库地址，以降低依赖下载失败的概率。同时，在构建属性配置中，明确指定 Gradle 使用的 Java Home 路径，使构建过程始终绑定到预期的 openjdk-17 环境，避免因系统中存在多个 JDK 版本而产生不确定行为。

在 Gradle Wrapper 适配方面，本研究针对分发校验与下载路径可能导致构建失败的问题，对 Gradle Wrapper 的配置进行了必要调整。通过注释校验相关字段并将 Gradle 发行包下载地址切换至国内镜像源，本研究避免了因校验失败或访问受限导致的构建中断。同时，为解决 Gradle 在部分网络环境中对源码托管站点访问受限的问题，本研究在系统层面对关键域名配置了 IP 映射，以保证

构建脚本在解析依赖或拉取资源时能够顺利完成网络访问。

在构建行为控制方面，本研究通过调整 Gradle 的 JVM 参数，在默认参数基础上追加解释执行相关选项，以降低即时编译或平台相关优化在 RISC-V 架构下引发的不确定性。此外，本研究通过补丁方式对 Kafka 的 build.gradle 以及 Gradle Wrapper 配置文件进行修订，使其在 RISC-V 平台上能够顺利完成构建流程。补丁的应用过程严格限定在源码目录内执行，确保修改范围清晰、可审计，并可在需要时回滚或复现。

在完成上述配置与适配后，本研究按照 Kafka 官方推荐的构建流程，依次执行清理与构建任务，通过 Gradle 完成 Kafka 核心模块的编译与打包，最终生成标准格式的发行构件。

### 研究结果

研究结果表明，在完成基础环境准备、构建工具链配置、网络与依赖解析适配以及必要的构建脚本修订后，Apache Kafka 3.8.0 能够在 RISC-V 平台上成功完成源码构建。本研究通过执行完整的 Gradle 构建流程，成功生成名为 kafka\_2.13-3.8.0.tgz 的发行包，该产物位于 Kafka 源码树中预期的分发目录下，结构与官方发布版本保持一致。

该结果证明，在不依赖非 RISC-V 架构预编译二进制的前提下，Kafka 3.8.0 在 RISC-V 平台上具备工程层面的可构建性。本研究形成的构建流程对环境变化具备一定鲁棒性，且关键配置与补丁均具有明确边界与可复现性，可为后续在 RISC-V 架构环境中部署消息中间件、开展功能验证或进一步性能研究提供可靠的构建基础。

## 2.2.1.5. Logstash

### 研究内容

本研究围绕 Elastic 生态核心组件 Logstash 在 RISC-V (riscv64) 架构平台上的源码编译与部署可行性展开。研究内容聚焦于整体性工程目标的界定，即验证 Logstash v8.17.4 是否能够在 RISC-V 平台上，在不依赖其他主流架构预编译二进制产物的前提下，通过源码方式完成完整构建，并以功能等价于官方发行版本的形式正确运行其核心能力。本研究不以插件扩展、性能调优或生产级集群

部署为目标，而是关注 Logstash 在 RISC-V 架构下的基础可用性问题，涵盖其对 Java、Ruby、JRuby 运行环境的依赖完整性，以及原生依赖在非主流架构环境中可能暴露的工程适配问题。通过该研究内容的实施，本研究旨在为 RISC-V 生态补齐日志采集与数据处理基础组件的工程事实依据，为后续 ELK 技术栈在 RISC-V 平台上的整体落地提供前提条件。

## 研究方案

在研究方案设计上，本研究采用以依赖闭环和构建可控性为核心的工程化实施方案。方案首先从系统环境入手，在 RISC-V 平台上通过包管理器安装 Logstash 构建所需的基础工具链，包括 Java 17 运行环境、Maven、Ant 以及 Ruby 相关开发工具，并通过显式配置 JAVA\_HOME 与 PATH，确保所有 Java 与构建过程统一运行在 openjdk-17 环境之下，以避免多版本 Java 并存导致的不确定行为。与此同时，本研究对 Ruby 生态进行了独立配置，通过替换 Gem 与 Bundler 的官方源为国内可访问镜像，降低网络不稳定对依赖解析的影响。

在依赖编译与适配阶段，本研究将重点放在 Logstash 所依赖的原生接口组件与 JRuby 运行时上。首先针对 JFFI 这一 JNI/FFI 关键组件，本研究通过源码方式完成其编译，并将生成的构件显式安装至本地 Maven 仓库，以保证后续 JRuby 与 Logstash 构建过程中能够正确解析该依赖。随后，本研究对 JRuby 9.4.9.0 进行源码级适配。由于 JRuby 在 RISC-V 架构下尚未形成完整稳定支持，本研究通过补丁方式引入对 riscv64 架构的识别与处理逻辑，并同步修订其构建脚本以适配当前平台环境。在该过程中，本研究识别并分析了 JRuby 在初始化 Ruby 运行环境时的异常处理缺陷，即 FFIService 在工厂初始化阶段对异常进行泛化捕获，导致真实的依赖加载失败被误判为系统调用未实现。针对该问题，本研究采用依赖预置方案，在构建前离线下载并本地缓存关键 Gem 组件，并通过修改构建脚本强制其从本地路径加载，从而规避网络依赖并消除错误定位歧义。

在完成 JRuby 构建并验证其可用性后，研究方案进入 Logstash 源码编译阶段。本研究获取 Logstash v8.17.4 官方源码，并在构建前对运行环境进行显式约束，包括统一临时目录路径、设置合理的 JVM 堆内存上限以及指定自定义 JRuby 路径，以确保 Logstash 构建过程完全基于前述已适配完成的 JRuby 运行时。在此基础上，本研究通过补丁方式修订 Logstash 构建脚本中与路径和平

台相关的问题，使其能够正确识别并使用自定义 JRuby，而非尝试下载或使用官方预构建版本。随后，研究方案采用分阶段构建方式，先完成默认 Gem 的安装，再执行核心组件的并行编译，以降低单阶段构建失败的风险。在构建完成后，本研究进一步安装基础输入输出插件，以验证 Logstash 插件系统在 RISC-V 平台上的基本可用性。

## 研究结果

研究结果表明，在完成系统环境准备、原生依赖 JFFI 的本地构建、JRuby 9.4.9.0 的 RISC-V 架构适配以及 Logstash 8.17.4 源码级构建后，Logstash 能够在 RISC-V 平台上成功完成启动并执行基础数据处理任务。本研究通过构造最小化的标准输入与标准输出管道，对 Logstash 的运行行为进行了验证，系统能够正确解析输入事件并以结构化形式输出结果，表明其核心执行引擎、Ruby 运行时以及插件加载机制均处于可用状态。

同时，本研究在构建与运行过程中对若干典型问题进行了定位与解决，包括因 Gem 依赖未正确加载导致的 waitpid 等“未实现”异常、因 JVM 堆内存不足引发的构建失败，以及因临时目录跨文件系统导致的文件操作错误。这些问题在通过依赖预置、内存参数调整和临时目录统一后均得到有效解决。综合来看，本研究验证了 Logstash 8.17.4 在 RISC-V 架构环境中具备工程层面的可编译性与可运行性，其构建与部署流程具有明确的技术边界和良好的可复现性，可作为后续在 RISC-V 平台上构建日志处理与数据管道系统的重要基础。

### 2.2.1.6.Elasticsearch

#### 研究内容

本研究围绕 Elasticsearch 在 RISC-V (riscv64) 架构平台上的源码编译与部署可行性展开，研究内容侧重于整体性工程目标的界定与技术边界的澄清。Elasticsearch 作为 ELK 技术栈的核心组件<sup>[7]</sup>，其在 RISC-V 架构上的可用性直接关系到日志分析、全文检索与实时数据分析能力能否在新兴计算架构上落地。

本研究的核心目标在于验证 Elasticsearch 是否能够在 RISC-V 平台上，在不依赖 x86 或 ARM 架构预编译二进制产物的前提下，通过源码方式完成完整构建，并以功能等价于官方发行版本的形式稳定运行。研究内容不涉及性能调优、

索引结构优化或分布式集群扩展，而是聚焦于基础工程问题，包括 Java 运行环境的适配、原生依赖与构建工具链的可用性、架构检测逻辑的修订以及运行时系统约束在 RISC-V 环境中的适配情况。通过该研究内容的实施，本研究旨在为 Elasticsearch 在 RISC-V 架构上的工程化落地提供明确的事实依据，为后续在边缘计算、物联网等场景中引入实时搜索与分析能力奠定基础。

## 研究方案

在研究方案设计上，本研究以“源码可控、构建可复现、运行可验证”为基本原则，对 Elasticsearch 在 RISC-V 平台上的移植过程进行系统化规划。方案首先从系统环境入手，在 RISC-V 平台上构建满足 Elasticsearch 编译要求的基础工具链，包括编译工具、构建工具以及 Java 运行环境。本研究统一采用 OpenJDK 17 作为构建与运行时 Java 环境，并通过显式设置 JAVA\_HOME 与 PATH，确保 Gradle 构建过程始终绑定至预期的 JDK 实现，避免因多版本 Java 并存导致的构建不确定性。

在源码与依赖处理阶段，本研究选取 Elasticsearch 7.17.21 作为研究对象，并通过官方源码仓库获取对应版本源码，以保证研究对象的版本来源清晰可追溯。考虑到 Elasticsearch 构建系统中包含大量与平台相关的检测逻辑，本研究通过补丁方式对源码进行最小侵入式修改，引入对 riscv64 架构的识别与支持，并暂时屏蔽 Docker 镜像构建流程，以避免在构建阶段因架构检测失败而中断整体流程。同时，鉴于 OpenJDK 官方尚未提供面向 RISC-V 架构的 Elasticsearch 内置 JDK 发行包，本研究在构建阶段解除对特定 JDK 发行包的强制绑定策略，允许构建结果在部署时使用系统环境中已安装的 OpenJDK 17，从而保证构建流程的可行性与灵活性。

在编译执行阶段，本研究通过显式限制 Gradle 的内存使用并关闭守护进程模式，降低构建过程中资源争用与状态残留对结果的影响。构建目标选择生成本地发行版（local distribution），以获得标准 tar 格式的 RISC-V 64 位可分发构件。该阶段以构建流程能够顺利完成且生成预期发行目录为判定标准<sup>[8]</sup>。

在部署与运行方案上，本研究采用最小化但符合生产约束的配置策略。通过创建专用系统用户与运行目录，明确 Elasticsearch 的权限边界，避免以高权限账户运行带来的安全与稳定性隐患。在系统层面，本研究针对 Elasticsearch 对虚拟内存映射数量的要求，提前调整内核参数，以满足其运行时约束。在 Elast

icsearch 配置层面，本研究针对 RISC-V 平台当前尚不支持的 SecComp 机制，显式关闭系统调用过滤功能，并根据实际需求关闭机器学习模块，以减少不必要的组件加载。服务启动阶段通过绑定本地 OpenJDK 17 并以后台方式启动 Elasticsearch 实例，最终通过 HTTP 接口验证节点状态与版本信息，以确认运行结果的正确性。

## 研究结果

研究表明，在完成系统环境准备、源码级架构适配、构建流程修订以及运行时配置调整后，Elasticsearch 7.17.21 能够在 RISC-V 平台上成功完成源码编译与部署。本研究通过 Gradle 构建流程成功生成面向 riscv64 架构的本地发行版，并在部署后通过 HTTP 接口验证 Elasticsearch 节点能够正常启动并对外提供服务。返回的节点信息中包含完整的版本号、构建类型、Lucene 版本以及集群标识，表明当前运行实例由本地源码构建而成，且核心搜索与分析服务处于可用状态。

在运行验证过程中，本研究识别并解决了若干与 RISC-V 平台相关的关键问题，包括因 SecComp 支持缺失导致的启动失败、虚拟内存映射数量不足引发的运行时错误，以及构建阶段对 JDK 发行包的强制绑定问题。这些问题在通过配置调整、补丁修订和系统参数修改后均得到有效解决。综合来看，本研究验证了 Elasticsearch 在 RISC-V 架构环境中的工程可行性，其构建与部署流程具备明确的技术边界和良好的可复现性，可作为在 RISC-V 平台上引入搜索与实时分析能力的重要基础，为后续在边缘计算和自主可控计算生态中的应用提供实践依据。

### 2.2.1.7.Flink

## 研究内容

本研究围绕 Apache Flink 1.19.0 在 RISC-V (riscv64) 架构平台上的源码编译与部署可行性展开，研究内容聚焦于整体性工程目标与技术适配边界的界定。Flink 作为新一代分布式流处理框架，其在 RISC-V 平台上的可用性不仅关系到实时计算能力能否在新兴指令集架构上落地，也直接影响 RISC-V 在数据中心与边缘计算场景中承载复杂分布式负载的工程可行性。本研究的核心目标在

于验证 Flink 是否能够在 RISC-V 平台上，在不依赖其他主流架构预编译二进制产物的前提下，通过源码方式完成完整构建，并生成与官方发行形式等价的可运行产物。研究内容不涉及算子优化、状态后端性能评估或集群级调度能力分析，而是集中于 Flink 构建过程中对 Java 工具链、代码生成插件、原生依赖以及构建脚本平台适配能力的验证，为后续在 RISC-V 架构上开展实时流处理应用提供工程基础。

## 研究方案

在研究方案设计上，本研究以“分平台对照、关键依赖先行、构建流程可复现”为基本原则，对 Flink 1.19.0 的移植过程进行系统规划。本研究在硬件与软件环境上同时覆盖 Kunpeng-920 (aarch64) 与 SG2044 (RISC-V64) 两类平台，通过对比构建行为，区分通用问题与 RISC-V 平台特有問題。在基础环境层面，本研究统一采用 JDK 17 作为 Flink 编译与运行所需的 Java 环境，并配置满足构建要求的 Maven 版本，以确保构建工具链与 Flink 官方要求保持一致。

在源码准备阶段，本研究通过 Apache 官方归档站点获取 Flink 1.19.0 源码，并同步获取针对 Flink 的 RISC-V 平台补丁集合，以保证源码版本与补丁内容之间的一致性。在进入 Flink 正式编译之前，本研究首先对 RISC-V 平台上已知的构建阻断问题进行前置处理。其中，最关键的问题集中在 Protobuf 工具链与 protoc-jar-maven-plugin 插件对 riscv64 架构缺乏支持。本研究在 RISC-V 平台上通过源码方式编译并安装 Abseil 与 Protobuf 3.21.7，并将生成的 protoc 可执行文件以 riscv64 classifier 的形式注入本地 Maven 仓库，从而为 Flink 构建阶段提供可用的代码生成工具。针对 protoc-jar-maven-plugin 在平台探测阶段直接失败的问题，本研究通过引入已修订的平台支持版本，并重新编译该插件，使其能够正确识别 riscv64 架构并完成依赖解析，从而消除 Flink 构建流程中的结构性阻断点<sup>[9]</sup>。

在完成关键依赖工具链准备后，研究方案进入 Flink 源码适配阶段。本研究在 Flink 源码根目录中依次应用补丁，对前端构建源、测试行为以及 Maven 插件版本进行必要修订。其中，补丁内容主要用于替换前端依赖下载源、跳过与平台无关但在当前环境下不可稳定执行的测试用例，以及升级 os-maven-plugin 以增强对新架构的识别能力。补丁应用完成后，本研究通过统一设置 Maven 运行参数，对构建过程中的内存使用、编译优化选项以及测试执行行为进行显式约

束，确保构建过程在 RISC-V 平台上具备可预测性和可重复性。

在编译执行阶段，本研究采用官方推荐的 Maven Wrapper 进行 Flink 构建，并选择生成包含 tar 包的发行构件作为目标产物。在资源允许的情况下，研究方案支持并行构建以降低总体编译时间，但以构建稳定性优先于速度优化为原则。构建完成后，通过 Flink 提供的命令行工具对生成结果进行版本校验，以验证构建产物的完整性。

### 研究结果

研究表明，在完成基础环境准备、RISC-V 平台关键依赖工具链构建、源码补丁适配以及构建流程参数调整后，Apache Flink 1.19.0 能够在 RISC-V 平台上成功完成源码编译与发行包生成。本研究在构建完成后获得了预期的 build-target 目录结构，并通过执行 Flink 命令行工具成功获取版本信息，系统正确返回 Flink 1.19.0 的版本号与提交标识，表明当前运行实例由本地源码构建而成。

在构建过程中，本研究验证了 Protobuf 工具链与 protoc-jar-maven-plugin 插件是 Flink 在 RISC-V 平台上最关键的工程阻断点，其问题在通过源码编译与插件重建后得到有效解决。同时，前端构建依赖与部分测试用例在 RISC-V 平台上的不稳定性通过补丁方式被成功规避。综合来看，本研究证明了 Apache Flink 在 RISC-V 架构环境中的工程可行性，其构建与部署流程具备明确的技术边界和良好的可复现性，可作为后续在 RISC-V 平台上开展实时流处理系统部署、功能验证及进一步性能研究的基础。

## 2.2.1.8.Hive

### 研究内容

本研究围绕 Apache Hive 3.1.3 在 RISC-V (riscv64) 架构平台上的源码编译与部署可行性展开，研究内容聚焦于整体性工程目标与生态适配边界的界定。Hive 作为构建在 Hadoop 之上的数据仓库工具，其在 RISC-V 平台上的可用性不仅依赖于自身源码的可编译性，还高度依赖 Hadoop 生态组件、历史版本 Protobuf 工具链以及 Java 运行环境的兼容情况。本研究的核心目标在于验证 Hive 是否能够在 RISC-V 平台上，在不依赖 x86 或 ARM 架构预编译二进制产

物的前提下,通过源码方式完成完整构建,并以功能等价于官方发行版本的形式完成部署与启动。研究内容不涉及 Hive 查询性能评估、SQL 优化或复杂集群场景验证,而是聚焦于基础工程问题,包括旧版本依赖在新架构上的适配、本地代码生成工具的构建、构建插件的架构识别问题以及 Hive 与 Hadoop、HBase 等外部组件协同运行的可行性,为 RISC-V 架构下数据仓库系统的落地提供工程事实依据<sup>[12]</sup>。

## 研究方案

在研究方案设计上,本研究以“依赖前置、工具链闭环、构建过程可复现”为基本原则,对 Hive 3.1.3 在 RISC-V 平台上的移植过程进行系统规划。方案首先从基础环境入手,在目标平台上安装 Maven、Git、patch 以及编译工具链,并根据 Hive 3.1.3 的历史版本约束,统一采用 JDK 8 作为构建与运行时 Java 环境,通过显式设置 JAVA\_HOME 与 PATH,确保 Maven 构建过程始终绑定至预期的 JDK 实现,从而避免因 Java 版本偏差引入的兼容性问题。

在依赖处理阶段,本研究将 Protobuf 工具链作为首要阻断点进行处理。由于 Hive 3.1.3 仍依赖 Protobuf 2.5.0,而该版本在 RISC-V 架构下并未提供可直接使用的预编译产物,本研究通过源码方式对 Protobuf 2.5.0 进行编译,并结合社区补丁引入对 riscv64 架构的支持。在完成源码编译与安装后,本研究将生成的 protoc 可执行文件以 linux-riscv64 classifier 的形式注入本地 Maven 仓库,从而满足 Hive 构建阶段对代码生成工具的依赖需求。与此同时,针对 protoc-jar-maven-plugin 在 RISC-V 平台上因架构识别缺失而导致的构建失败问题,本研究通过引入已修订的平台支持版本并重新编译对应插件,使其能够在 RISC-V 环境中正确运行,进而消除 Hive 构建流程中的关键结构性阻断。

在源码准备与补丁处理阶段,本研究从 Apache 官方仓库获取 Hive 3.1.3 源码,并针对部分历史依赖在当前网络或仓库环境中无法直接获取的问题,采用手动下载并注入本地 Maven 仓库的方式进行补救。这一方案确保 Hive 构建过程中所有依赖项均可在本地解析,避免因远程仓库不可达或构件缺失导致构建失败。在完成依赖注入后,本研究进入 Hive 的正式编译阶段,通过设置合理的 Maven 内存参数并启用并行构建,以在资源允许的前提下缩短构建时间,同时保证构建过程的稳定性。整个编译过程以生成完整的 Hive 二进制发行包作为阶段性目标。

在部署与运行方案上，本研究以单节点验证为主线，对 Hive 的运行环境进行最小化配置。在部署阶段，将编译生成的 Hive 发行包复制至统一安装目录，并通过系统环境变量显式指定 HIVE\_HOME 与执行路径。在配置层面，本研究在已完成 Hadoop 与 HBase 基础部署的前提下，对 Hive 的环境脚本与核心配置文件进行必要设置，重点包括元数据仓库连接信息、临时目录路径、日志目录以及 Server2 相关端口配置。在运行阶段，通过初始化 Hive 元数据仓库并依次启动 Hadoop 与 Hive 服务，对 Hive 的基础启动能力进行验证。

### 研究结果

研究表明，在完成基础环境准备、Protobuf 2.5.0 的 RISC-V 架构编译适配、protoc-jar-maven-plugin 插件修复以及 Hive 3.1.3 源码级构建后，Hive 能够在 RISC-V 平台上成功完成编译并进入可部署状态。本研究通过 Maven 构建流程成功生成 Hive 3.1.3 的二进制发行包，并在部署后完成环境变量配置、配置文件初始化以及元数据仓库初始化操作。在依赖 Hadoop 正常运行的前提下，Hive 服务能够成功启动，表明其核心组件在 RISC-V 架构环境中具备基本可用性。

在构建与部署过程中，本研究验证了历史版本大数据组件在 RISC-V 平台上面临的主要工程挑战集中于旧版本依赖工具链与构建插件的架构支持缺失问题。通过源码级编译、补丁引入以及本地 Maven 仓库注入的方式，这些问题均可被有效解决。综合来看，本研究证明了 Apache Hive 3.1.3 在 RISC-V 架构环境中的工程可行性，其构建与部署流程具备明确的技术边界和良好的可复现性，可作为在 RISC-V 平台上引入数据仓库与离线分析能力的重要基础，为后续大数据生态在新兴指令集架构上的扩展提供实践参考。

## 2.2.1.9.SPDK

### 研究内容

本研究围绕 Storage Performance Development Kit (SPDK) 在 RISC-V 架构平台上的移植与部署可行性展开，研究内容侧重于整体性工程目标与技术路径的概述。SPDK 作为业界领先的高性能用户态存储开发工具包，其在 RISC-V 平台上的成功运行，对于构建高效能、低延迟、完全开源且可定制的下一代存储

基础设施具有重要意义。本研究的核心目标在于验证在 RISC-V 架构环境中，是否能够在不依赖传统封闭架构优化路径的前提下，通过源码方式完成 SPDK 及其关键依赖组件的完整构建与部署，并形成一套可复现、可维护的工程化流程。研究内容不涉及针对具体硬件的性能调优或应用级基准测试，而是聚焦于 SPDK 在 RISC-V 平台上的基础可构建性、依赖工具链的适配完整性以及用户态存储栈在新兴指令集架构下的工程可行性，为后续高性能存储系统的深化研究与实践奠定基础<sup>[13]</sup>。

## 研究方案

在研究方案设计上，本研究以“依赖闭环优先、构建流程标准化、环境可复现”为基本原则，对 SPDK 在 RISC-V 平台上的移植工作进行系统规划。方案首先从整体目录结构与版本体系入手，明确 SPDK、FIO 与 DPDK 的源码位置及版本对应关系，确保各组件之间的接口兼容性与构建逻辑的一致性。本研究选用 SPDK v24.09 与 FIO 3.38 作为研究对象，并采用 Meson 与 Ninja 作为主要构建系统，以适配 SPDK 近年来的构建体系演进。

在环境准备阶段，本研究在目标 RISC-V 系统上安装完整的基础编译工具链、开发库与测试依赖，重点覆盖 NUMA、加密、网络、异步 I/O 以及 Fuse 等与用户态存储密切相关的功能组件。同时，引入 Meson 与 Ninja 构建工具，以满足 DPDK 与 SPDK 对现代构建系统的要求。在此基础上，本研究首先对 FIO 进行源码编译与安装，使其作为后续 SPDK 性能验证与接口集成的必要工具。

在 SPDK 核心依赖处理方面，本研究将 DPDK 作为关键前置组件进行单独构建。考虑到 RISC-V 平台当前在部分硬件加速指令与驱动支持方面的限制，本研究在 DPDK 构建配置中选择通用架构模式，限制 NUMA 节点数量，并显式关闭与当前平台不兼容的压缩驱动模块，以确保构建过程稳定可控。DPDK 通过 Meson 完成配置与编译后，被安装至系统统一路径，作为 SPDK 构建阶段的依赖输入。

在完成依赖准备后，研究方案进入 SPDK 的配置与编译阶段。本研究通过显式指定 FIO 与 DPDK 的安装路径，启用调试支持并开启 iSCSI Initiator 功能，对 SPDK 进行配置与构建。构建完成后，通过环境变量配置将 SPDK 的可执行程序、动态库与 pkg-config 信息纳入系统搜索路径，使 SPDK 工具能够在

用户环境中直接调用。为提高工程实施的可控性与鲁棒性，本研究引入构建状态记录机制，通过状态文件对各关键步骤的完成情况进行标记，支持构建过程的中断恢复与重复执行规避。

### 研究结果

研究结果表明，在完成系统依赖准备、FIO 与 DPDK 的源码编译安装以及 SPDK 核心组件的配置与构建后，SPDK v24.09 能够在 RISC-V 平台上成功完成部署。本研究通过检查 SPDK 目标程序的生成情况及环境变量配置结果，验证了 SPDK 构建产物在系统中的可见性与可调用性，表明其核心用户态存储服务具备运行基础。

在构建过程中，本研究未观察到由指令集差异直接引发的不可修复构建失败，说明在通用架构配置与合理裁剪驱动模块的前提下，SPDK 与 DPDK 能够在 RISC-V 平台上形成稳定的构建闭环。同时，通过日志文件与状态管理机制，本研究能够对配置、编译与安装阶段的问题进行有效定位与回溯，增强了整体移植流程的可维护性。综合来看，本研究验证了 SPDK 在 RISC-V 架构环境中的工程可行性，为在开放指令集平台上构建高性能用户态存储系统提供了可操作的技术路径，也为后续围绕原子操作、内存模型与硬件加速特性的深入优化研究奠定了坚实基础。

## 2.2.2. 云计算适配

### 2.2.2.1. Dockers

#### 研究内容

本研究围绕 Docker 27.1.1 在 RISC-V64 架构平台上的源码构建与系统化部署展开，研究内容聚焦于整体性工程目标与云原生生态适配边界的明确界定。Docker 作为现代云原生架构中最核心的容器运行与管理组件，其在 RISC-V 平台上的可用性直接决定了该架构能否有效承载容器化应用、微服务体系以及上层 Kubernetes 等编排系统。本研究的核心目标在于验证 Docker 27.1.1 是否能够在 RISC-V 平台上，在不依赖非 RISC-V 架构预编译二进制产物的前提下，通过源码方式完成完整构建，并以系统服务的形式稳定运行其守护进程与客户端工具。研究内容不涉及容器运行时性能评测、镜像分发效率分析或多节点集群调度，

而是聚焦于 Docker 核心组件在 RISC-V 架构下的可编译性、依赖组件的完整性以及系统级部署与服务管理的工程可行性，为 PolyOS Server 及相关云原生基础设施在 RISC-V 平台上的落地提供基础支撑。

## 研究方案

在研究方案设计上，本研究以“工具链先行、组件解耦、版本一致性优先”为基本原则，对 Docker 27.1.1 在 RISC-V 平台上的移植过程进行系统规划。方案首先从构建工具链入手，明确 Go 语言在 Docker 构建体系中的核心地位，并在 RISC-V 平台上选用官方支持的 Go 1.22.5 版本作为唯一构建工具链，通过手动下载安装并显式配置 GOROOT、GOPATH 与 PATH，确保构建过程不受系统中其他 Go 版本干扰。同时，通过配置 GOPROXY 与 GOTOOLCHAIN 参数，降低网络环境波动对依赖拉取与构建稳定性的影响。

在依赖组件处理方面，本研究将 Docker 的运行时依赖视为独立问题域进行处理。在正式构建 Docker 之前，本研究优先完成 containerd、runc、rootlesskit 与 tini 等关键组件在 RISC-V 平台上的安装与验证，确保 Docker 在构建完成后能够直接对接可用的底层运行时环境。这一方案避免了 Docker 构建与运行问题相互叠加，提升了问题定位与工程调试的可控性。

在 Docker 源码构建阶段，本研究通过切换至 Docker 27.1.1 的官方版本标签，确保构建目标与版本声明严格一致。在构建执行上，本研究采用 Docker 官方提供的构建脚本生成守护进程与代理组件的二进制产物，并通过显式指定版本号与提交标识，保证构建结果在版本层面的可追溯性。构建完成后，将生成的二进制文件部署至系统可执行路径，使 Docker 的核心服务组件具备被系统服务管理器调用的基础条件。

在客户端工具处理方面，本研究将 Docker CLI 与 Docker 守护进程视为两个相对独立的构建单元。通过单独拉取 Docker CLI 源码并切换至与守护进程一致的版本标签，本研究避免了客户端与服务端版本不匹配可能引发的协议或功能不一致问题。CLI 构建完成后，将生成的二进制文件统一部署至系统路径，从而形成完整可用的 Docker 工具链<sup>[14]</sup>。

在系统部署方案上，本研究通过 systemd 服务管理机制对 Docker 进行集成部署。通过编写并配置 Docker 的服务描述文件，明确守护进程的启动方式、运行参数与资源约束，并通过独立的 daemon.json 文件对 Docker 的数据目录、

日志管理及运行行为进行集中配置,使 Docker 能够以符合系统规范的方式长期稳定运行。该方案确保 Docker 在 RISC-V 平台上的部署方式与主流架构保持一致,降低后续运维与生态适配成本。

### 研究结果

研究结果表明,在完成 Go 语言工具链配置、Docker 核心依赖组件准备以及 Docker 27.1.1 源码级构建与系统部署后, Docker 能够在 RISC-V 平台上成功完成守护进程与客户端的构建,并以系统服务的形式进入可运行状态。本研究通过部署构建生成的二进制文件并完成 systemd 服务配置,验证了 Docker 守护进程在 RISC-V 环境中的可启动性与基本运行稳定性,同时确认 Docker CLI 能够正确与守护进程交互,具备基础的容器管理能力。

该研究结果证明,在当前工具链与运行时条件下, Docker 27.1.1 在 RISC-V 架构环境中具备工程层面的可移植性与可部署性。本研究形成的构建与部署流程具有明确的版本约束和良好的可复现性,可作为 PolyOS Server 及其他 RISC-V 云原生平台引入容器化技术的重要基础,为后续在 RISC-V 架构上进一步构建 Kubernetes、云原生存储与网络组件提供可靠前提。

## 2.2.2.2. TeleVM

### 研究内容

本研究围绕基于 KVM 的轻量级虚拟化技术 StratoVirt 在 RISC-V 架构平台上的移植实现展开,研究对象为其面向 RISC-V 平台的实现版本 TeleVM 1.0。研究内容聚焦于整体性工程目标的界定,即验证 TeleVM 是否能够在 RISC-V64 架构环境中完成从源码到可执行程序的完整构建,并具备作为用户态虚拟机管理程序运行的基础条件。作为云原生基础设施的重要组成部分,虚拟化能力在 RISC-V 数据中心与 PolyOS Server 技术体系中具有关键支撑作用。本研究不涉及虚拟机运行性能评估、I/O 虚拟化效率或多虚拟机场景下的资源调度策略,而是集中于 TeleVM 在 RISC-V 平台上的工程可编译性、Rust 工具链适配情况以及最终产物在指令集与 ABI 层面的正确性验证,为后续在 RISC-V 平台上构建轻量级虚拟化与云原生运行环境提供基础依据。

### 研究方案

在研究方案设计上，本研究以“语言工具链先行、源码最小调整、构建结果可验证”为基本原则，对 TeleVM 1.0 在 RISC-V 平台上的构建过程进行系统化规划。鉴于 TeleVM 完全采用 Rust 语言开发，研究方案首先从 Rust 编译工具链入手，通过官方 `rustup` 安装方式在目标系统中部署标准 Rust 开发环境，确保 Cargo 构建系统与 `rustc` 编译器能够正常工作，并具备针对 RISC-V 架构生成可执行文件的能力。

在源码准备阶段，本研究从公开仓库获取 TeleVM 源代码，并在进入正式构建前对项目配置文件进行必要修订。具体而言，本研究统一规范了 Cargo 项目的版本号格式，并将源码中使用的 Rust edition 从较新的实验性版本回退至 2021 版本，以消除因工具链版本差异带来的潜在不兼容问题。这一调整属于构建层面的兼容性修订，不涉及功能逻辑变更。针对 TeleVM 在 RISC-V 平台上的特定适配需求，研究方案预留了补丁应用步骤，用于引入架构相关修正或构建流程调整，从而确保源码在 RISC-V 环境中能够顺利通过编译。

在构建执行阶段，本研究采用 Cargo 提供的标准构建流程对 TeleVM 进行编译，生成面向 `riscv64gc-unknown-linux-gnu` 目标平台的可执行文件。该阶段以构建过程无错误完成、目标文件成功生成作为判定标准，而不引入额外的交叉编译复杂性或多目标输出策略。构建完成后，研究方案通过对生成二进制文件进行格式与架构校验，确认其在指令集、ABI 以及链接方式层面符合 RISC-V64 平台的预期要求。

## 研究结果

研究表明，在完成 Rust 工具链部署、TeleVM 源码配置修订以及必要的平台适配后，TeleVM 1.0 能够在 RISC-V64 架构平台上成功完成源码编译，并生成符合 RISC-V 指令集规范的可执行程序。本研究通过对生成文件的格式检查确认，该二进制文件为标准的 RISC-V64 ELF 可执行文件，具备正确的架构标识、ABI 类型以及 Linux 系统兼容性特征，表明其构建结果在底层架构层面是正确且可用的<sup>[15]</sup>。

该研究结果验证了 TeleVM 作为 StratoVirt 在 RISC-V 平台上的实现版本，在当前 Rust 工具链与内核虚拟化环境假设下具备工程层面的可移植性。本研究所形成的构建流程具有清晰的步骤划分和良好的可复现性，可为 PolyOS Server 在 RISC-V 平台上引入轻量级虚拟化能力提供直接参考。同时，该结果也

为后续进一步研究 TeleVM 在虚拟机生命周期管理、I/O 虚拟化以及与云原生调度系统集成等方面的能力奠定了基础。

### 2.2.2.3.iSulaD

#### 研究内容

本研究围绕云原生基础设施在 RISC-V64 架构平台上的核心运行组件展开，研究对象包括轻量级容器运行时 iSulaD 2.1.5 以及容器编排系统 Kubernetes 1.30.3。研究内容聚焦于整体性工程目标的界定，即验证在 RISC-V 架构环境中，是否能够通过源码方式完成云原生“运行时—编排层”关键组件的构建，并形成可用于后续系统集成的基础能力。iSulaD 作为容器运行时，直接决定了容器生命周期管理、隔离机制和运行效率，而 Kubernetes 则是云原生体系中负责资源调度与服务治理的核心控制平面。本研究不涉及容器或集群层面的性能评测、调度策略优化或生产级高可用部署，而是关注二者在 RISC-V 平台上的基础可编译性、依赖适配问题及构建结果的正确性验证，为在 RISC-V 架构下构建完整云原生软件栈提供工程事实基础。

#### 研究方案

在研究方案设计上，本研究采用“分层推进、依赖前置、源码构建验证”的方法论，对 iSulaD 与 Kubernetes 在 RISC-V 平台上的移植过程进行系统化实施。方案首先从容器运行时层入手，优先解决 iSulaD 在 RISC-V 环境中的构建与安装问题。针对 iSulaD 依赖的 protobuf、gRPC 以及系统级库组件，本研究在构建前完成依赖环境的准备与校验，并通过应用针对 RISC-V 架构的补丁，修复 spec 文件中存在的依赖冲突问题，从而使构建依赖关系在当前平台上闭合。在此基础上，本研究进一步对 libisula 相关头文件进行架构适配修订，以解决在 RISC-V 平台下因数据结构或状态定义差异导致的编译问题。随后，通过标准的 CMake 构建流程完成 iSulaD 的编译与安装，并以版本信息校验作为构建成功的判定依据。

在完成容器运行时层的构建后，研究方案进入容器编排层，即 Kubernetes 的移植与构建阶段。鉴于 Kubernetes 完全基于 Go 语言实现，本研究首先在 RISC-V 平台上部署并配置官方支持的 Go 1.22.5 工具链，通过显式设置 GOR

OOT、GOPATH 及相关模块参数，确保 Kubernetes 源码构建过程使用一致且可控的编译环境。在源码准备阶段，本研究从官方仓库获取 Kubernetes 源码并切换至 1.30.3 版本标签，以保证构建目标与版本声明一致。构建阶段采用 Kubernetes 官方提供的 Makefile 构建流程，对核心组件进行整体编译，生成面向 RISC-V64 架构的二进制产物。研究方案通过对生成文件路径及组件列表的检查，确认关键控制与客户端工具均已成功生成。

## 研究结果

研究表明，在完成依赖环境准备、补丁修订以及源码级构建后，iSulaD 2.1.5 能够在 RISC-V64 架构平台上成功完成编译与安装，并通过版本查询验证其核心功能处于可用状态。这一结果表明，iSulaD 在 RISC-V 平台上具备作为容器运行时使用的基础工程条件，可为上层容器编排系统提供稳定的运行支撑。

同时，本研究验证了 Kubernetes 1.30.3 在 RISC-V 平台上的源码构建可行性。通过标准构建流程生成的 kubectl 等核心组件能够正确识别自身版本信息，表明其二进制产物在指令集和运行环境层面符合 RISC-V64 平台要求。综合来看，本研究证明了从容器运行时到容器编排层的关键云原生组件在 RISC-V 架构上的工程可移植性，为在 RISC-V 数据中心环境中构建完整云原生软件栈奠定了基础。这一成果不仅丰富了 RISC-V 平台的云原生生态，也为后续在该架构上开展集群部署、系统集成与运维管理研究提供了可靠前提<sup>[16]</sup>。

### 2.2.2.4. Grafana

## 研究内容

本研究围绕 Grafana 11.1.3 在 RISC-V 架构平台上的源码构建与运行可行性展开，研究内容聚焦于整体性工程目标的界定与实现路径的可验证性。Grafana 作为云原生与数据中心环境中事实上的标准化监控与可视化平台，其在 RISC-V 平台上的可用性直接关系到该架构是否具备完善的运维监控与可观测性基础设施支持能力。本研究的核心目标在于验证 Grafana 是否能够在 RISC-V 架构环境中，通过源码方式完成前后端的完整构建，并以功能等价于主流架构版本的形式成功运行其核心服务。研究内容不涉及仪表盘渲染性能评估、复杂数据源适

配或大规模监控场景验证，而是集中于 Grafana 在 RISC-V 平台上的工程可编译性、前后端工具链适配情况以及最终服务运行状态的正确性，为 PolyOS Server 等 RISC-V 数据中心操作系统提供可视化与监控能力奠定基础。

## 研究方案

在研究方案设计上，本研究遵循“前后端分离构建、工具链显式控制、版本最小调整”的原则，对 Grafana 11.1.3 在 RISC-V 平台上的移植过程进行系统化规划。由于 Grafana 同时依赖 Node.js 与 Go 两套工具链，本研究首先从构建环境入手，分别对前端与后端所需的开发环境进行独立配置。在前端环境方面，本研究通过 nvm 工具在系统中安装满足 Grafana 构建要求的 Node.js 版本，并辅以系统包管理器安装 nodejs 与 npm，以确保前端构建过程中依赖解析与脚本执行的稳定性。

在后端环境方面，本研究采用源码方式获取并部署 Go 语言工具链，通过显式设置 GOROOT、GOPATH 以及模块代理参数，确保 Grafana 后端在 RISC-V 平台上的编译过程不受网络环境和系统默认 Go 配置的干扰。该方案避免了因预编译 Go 二进制缺失或版本不一致带来的构建不确定性。

在源码准备阶段，本研究从官方仓库获取 Grafana 11.1.3 源码，并严格切换至对应版本标签，以保证研究对象的版本一致性。在前端构建阶段，本研究识别到 Grafana 前端依赖中的 nx 工具版本在 RISC-V 平台上存在兼容性问题，因此通过最小化修改策略，将 nx 版本从原有版本升级至可在 RISC-V64 环境中正常工作的版本，并在依赖解析策略中强制锁定该版本。随后，通过清理 Yarn 缓存、延长安装超时时间等方式，完成前端依赖的完整安装，并生成前端构建产物。

在后端构建阶段，本研究在前端构建完成的前提下，直接使用 Grafana 官方提供的 Makefile 构建流程对后端服务进行编译与启动。该阶段以服务可成功启动、版本信息可正确输出作为构建成功的判定标准。研究方案同时通过 HTTP 接口访问对 Grafana 服务的运行状态进行验证，以确认其在 RISC-V 平台上的基础运行能力。

## 研究结果

研究结果表明，在完成 Node.js 与 Go 工具链配置、前端依赖版本调整以及 Grafana 11.1.3 前后端源码构建后，Grafana 能够在 RISC-V 架构平台上成

功完成构建并进入可运行状态。本研究通过版本查询命令验证了 Grafana 后端二进制的正确生成，并通过访问本地服务端口确认 Grafana Web 服务能够正常响应请求，表明其核心服务逻辑与前端资源加载在 RISC-V 环境中具备可用性。

该研究结果证明，Grafana 在当前 RISC-V 平台的软件工具链条件下具备工程层面的可移植性，其前后端分离构建模式能够通过有限且可控的依赖调整顺利适配新架构。本研究形成的构建与运行流程具有良好的可复现性，可作为 RISC-V 数据中心环境中引入监控与可观测性能力的重要实践参考，也为后续在该平台上集成 Prometheus 等监控数据源、构建完整运维体系提供了可靠前提。

### 2.2.2.5.Prometheus

#### 研究内容

本研究围绕 Prometheus 2.53.1 在 RISC-V64 架构平台上的源码构建与运行可行性展开，研究内容聚焦于整体性工程目标的界定。Prometheus 作为云原生监控体系中的核心组件，其在 RISC-V 平台上的可用性直接关系到该架构在数据中心与 PolyOS Server 环境中是否具备完整的可观测性基础能力。本研究的核心目标在于验证 Prometheus 是否能够在 RISC-V 架构环境中，通过源码方式完成完整构建，并生成与主流架构功能等价的可执行程序，从而支持系统指标采集、查询与运行状态监控。研究内容不涉及监控指标规模扩展、告警规则设计或性能压力测试，而是集中于 Prometheus 在 RISC-V 平台上的工程可编译性、构建工具链适配情况以及最终产物在运行层面的正确性验证，为云原生监控体系在 RISC-V 平台上的落地提供基础事实依据<sup>[17]</sup>。

#### 研究方案

在研究方案设计上，本研究采用“工具链统一、源码直编、结果可验证”的工程实施策略，对 Prometheus 在 RISC-V 平台上的移植过程进行系统规划。鉴于 Prometheus 完全基于 Go 语言实现，研究方案首先从 Go 工具链入手，在 RISC-V 环境中部署官方支持的 Go 1.22.5 版本，并通过源码或预编译方式完成安装。通过显式设置 GOROOT、GOPATH 以及 PATH 环境变量，并启用 Go Modules 机制，同时配置国内可访问的模块代理源，本研究确保 Prometheus 在构建过程中能够稳定获取依赖并避免网络环境不确定性对构建结果的影响。

在构建环境准备过程中，研究方案同时引入 Node.js 与 npm 作为前端资源生成与构建辅助工具，以满足 Prometheus 构建系统对静态资源处理的依赖需求。在完成工具链配置后，本研究从 Prometheus 官方仓库获取源码，并严格切换至 2.53.1 版本标签，确保构建对象与版本声明一致，从而保证研究结论的可追溯性与可复现性。

在源码构建阶段，本研究直接采用 Prometheus 官方提供的 Makefile 构建流程，对源码进行整体编译。该方案避免了对构建逻辑的额外修改，能够更直接地反映 Prometheus 在 RISC-V 平台上的原生构建能力。构建成功与否以可执行文件是否正确生成作为判定标准，同时通过后续的版本信息校验来确认编译结果的完整性与正确性。

### 研究结果

研究表明，在完成 Go 语言工具链与 Node.js 构建环境配置后，Prometheus 2.53.1 能够在 RISC-V64 架构平台上成功完成源码编译。本研究通过执行生成的 Prometheus 二进制程序并查询其版本信息，确认该构建产物正确标识为 Prometheus 2.53.1，且平台信息显示为 linux/riscv64，表明其在指令集与运行环境层面均符合预期。

该研究结果验证了 Prometheus 在 RISC-V 架构环境中的工程可行性，说明其核心代码与构建体系对新兴指令集架构具有良好的适应能力。本研究形成的构建流程具有明确的工具链约束和良好的可复现性，可作为在 PolyOS Server 及其他 RISC-V 数据中心环境中引入监控与可观测性能力的重要实践基础。同时，该成果也为后续在 RISC-V 平台上集成完整监控体系、构建多组件云原生可观测性栈提供了可靠前提。

## 2.2.2.6.Helm

### 研究内容

本研究围绕 Kubernetes 生态中核心应用管理工具 Helm 在 RISC-V64 架构平台上的移植与构建可行性展开，研究内容聚焦于整体性工程目标的界定。Helm 作为云原生环境中事实标准的包管理工具，其在 RISC-V 平台上的可用性直接关系到 Kubernetes 应用在该架构下的部署效率、配置一致性与生命周期管

理能力。本研究的核心目标在于验证 Helm 3.15 系列版本是否能够在 RISC-V 架构环境中，通过源码方式完成完整构建与安装，并生成与主流架构功能等价的命令行工具，从而支撑 Kubernetes 应用的标准化部署与运维管理。研究内容不涉及 Helm Chart 复杂模板渲染性能、仓库规模化管理或集群级应用治理策略，而是集中于 Helm 在 RISC-V 平台上的工程可编译性、Go 工具链适配情况以及构建产物的正确性验证，为云原生应用管理能力在 RISC-V 平台上的落地提供基础事实依据。

## 研究方案

在研究方案设计上，本研究采用“统一工具链、官方构建流程优先、最小干预适配”的工程实施策略，对 Helm 在 RISC-V 平台上的构建过程进行系统规划。鉴于 Helm 完全基于 Go 语言实现，研究方案首先从 Go 工具链入手，在 RISC-V 环境中部署官方支持的 Go 1.22.5 版本，并通过源码或预编译方式完成安装。通过显式配置 GOROOT、GOPATH 以及 PATH 环境变量，本研究确保 Helm 构建过程使用稳定且可控的 Go 编译环境，同时避免系统默认 Go 版本差异对构建结果产生影响<sup>[18]</sup>。

在环境准备阶段，研究方案同步启用 Go Modules 机制，并配置可访问的模块代理源，以降低外部网络环境对依赖解析与下载过程的影响。对于 Helm 构建过程中所需的静态分析与质量检查工具，如 golangci-lint，本研究在构建前完成相应依赖的准备，从而保证官方构建流程能够在 RISC-V 平台上完整执行。

在源码准备与构建阶段，本研究从 Helm 官方仓库获取源码，并严格切换至 3.15.x 对应的版本标签，以确保研究对象与版本声明保持一致。构建阶段直接采用 Helm 官方提供的 Makefile 构建流程，通过 `make build` 生成适用于 RISC-V64 架构的 Helm 可执行文件，并通过 `make install` 将构建产物安装至系统环境中。该方案避免了对源码或构建脚本的额外修改，使构建结果能够真实反映 Helm 在 RISC-V 平台上的原生构建能力。

## 研究结果

研究结果表明，在完成 Go 语言工具链配置、模块管理环境准备以及 Helm 3.15.x 源码构建后，Helm 能够在 RISC-V64 架构平台上成功完成编译与安装。本研究通过执行 Helm 版本查询命令，验证生成的 Helm 二进制程序能够正确输出版本号、提交信息及 Go 编译环境信息，其中平台运行环境与 Go 版

本均与预期一致，表明构建产物在指令集与运行时层面符合 RISC-V64 平台要求。

该研究结果验证了 Helm 在 RISC-V 架构环境中的工程可行性，说明其构建体系与核心功能对新兴指令集架构具备良好的适应能力。本研究形成的构建与安装流程具有明确的工具链约束和良好的可复现性，可作为在 RISC-V 数据中心环境中引入 Kubernetes 应用包管理能力的重要实践参考，也为后续在该平台上开展 Helm Chart 生态适配与大规模云原生应用部署奠定了坚实基础。

### 2.2.2.7.Istio

#### 研究内容

本研究围绕服务网格框架 Istio 1.22.2 在 RISC-V64 架构平台上的移植与构建可行性展开，研究内容聚焦于整体性工程目标的界定。Istio 作为云原生体系中负责流量治理、安全策略与可观测性的关键基础设施，其在 RISC-V 平台上的可用性直接关系到该架构能否支撑大规模微服务系统的精细化管理。本研究的核心目标在于验证 Istio 是否能够在 RISC-V 架构环境中，通过源码方式完成核心组件的构建，并生成可用于后续系统集成的二进制产物，从而为 PolyOS Server 等 RISC-V 数据中心操作系统提供服务网格能力支撑。研究内容不涉及 Envoy 流量性能评测、复杂策略规则验证或多集群治理场景分析，而是集中于 Istio 构建体系在 RISC-V 平台上的工程可行性、依赖工具链与构建流程的适配情况，以及最终产物在平台层面的完整性验证。

#### 研究方案

在研究方案设计上，本研究采用“依赖前置、工具链统一、构建流程最小侵入”的实施策略，对 Istio 1.22.2 在 RISC-V 平台上的移植过程进行系统规划。鉴于 Istio 构建体系复杂，且依赖多种编译工具与中间产物，本研究首先从源码准备入手，通过克隆已针对 RISC-V 平台进行初步适配的 Istio 分支，确保研究对象在版本和架构层面具备可构建基础。

在构建环境方面，本研究统一采用 Go 1.22.5 作为 Istio 构建所需的 Go 工具链，并通过显式设置 GOROOT、GOPATH、PATH 以及 Go Modules 相关环境变量，确保构建过程中依赖解析行为稳定可控。通过关闭 GOSUMDB 并配

置国内可访问的 GOPROXY，本研究有效降低了网络环境不稳定对构建流程的干扰。

考虑到 Istio 构建过程中高度依赖 Envoy Proxy 与 Ztunnel 等核心组件，而这些组件通常通过 Bazel 在特定架构下预编译生成，本研究在方案中引入预编译依赖策略。即在正式构建 Istio 之前，提前准备并加载已通过 RISC-V 架构 Bazel 工具链构建完成的 proxy 与 ztunnel 二进制产物，使 Istio 构建过程能够直接复用这些结果，避免在 RISC-V 平台上重复触发复杂且不稳定的底层构建流程。

在源码适配阶段，本研究通过应用针对 RISC-V 架构的补丁，对 Istio 源码中涉及平台判断、构建路径及组件选择的关键逻辑进行修订，以确保构建系统能够正确识别并生成 linux\_riscv64 目标产物。在构建执行阶段，本研究采用 Istio 官方提供的 Makefile 构建流程，依次完成初始化与整体编译操作，并通过指定适用于 RISC-V 环境的构建容器镜像，确保构建工具链版本的一致性与可复现性。

## 研究结果

研究表明，在完成 Go 工具链配置、预编译依赖加载、源码补丁适配以及标准构建流程执行后，Istio 1.22.2 能够在 RISC-V64 架构平台上成功完成源码构建。本研究在构建完成后获得了位于指定输出目录中的 Istio 核心组件二进制文件，表明其控制平面与相关服务组件已成功针对 RISC-V 平台生成<sup>[19]</sup>。

该研究结果验证了 Istio 在 RISC-V 架构环境中的工程可行性，说明在合理拆分构建依赖、复用关键预编译组件并对构建逻辑进行有限适配的前提下，复杂服务网格系统亦可在新兴指令集架构上完成移植。本研究形成的构建流程具有清晰的依赖边界和良好的可复现性，可作为在 PolyOS Server 等 RISC-V 数据中心平台上引入服务网格能力的重要技术参考，也为后续围绕微服务流量治理、安全策略与可观测性体系在 RISC-V 架构上的深入应用奠定了工程基础。

### 2.2.2.8.Kata Containers

#### 研究内容

本研究围绕 Kata Containers 3.2 在 RISC-V64 架构平台上的移植与构建

可行性展开，研究内容聚焦于整体性工程目标的界定。Kata Containers 作为融合轻量级虚拟机与容器运行时优势的关键基础设施，其核心价值在于在保持容器部署效率的同时，引入硬件虚拟化级别的安全隔离能力。本研究的核心目标在于验证 Kata Containers 3.2 是否能够在 RISC-V 架构环境中完成完整源码构建，并生成可用的运行时与代理组件，从而为 RISC-V 数据中心与云原生平台提供具备强隔离特性的容器运行能力。研究内容不涉及多租户安全策略评估、性能对比测试或其他运行时的横向比较，而是集中于 Kata Containers 在 RISC-V 平台上的工程可编译性、多语言工具链适配情况以及最终构建产物的正确性验证，为后续在安全敏感场景中引入该运行时奠定基础。

### 研究方案

在研究方案设计上，本研究采用“分语言工具链适配、关键构建路径修订、官方构建流程复用”的工程实施策略，对 Kata Containers 3.2 在 RISC-V 平台上的移植过程进行系统规划。鉴于 Kata Containers 3.x 版本在架构上引入了大量 Rust 实现，同时仍依赖 Go 语言与系统级安全库，本研究首先从依赖环境入手，构建覆盖 C 库、Rust 与 Go 的完整工具链环境。

在系统依赖层面，本研究通过安装 libseccomp 开发库，为 Kata Containers 提供系统调用过滤与安全约束能力。在 Rust 工具链方面，研究方案明确选用 Kata Containers 3.2 所兼容的 Rust 1.70.0 版本，并通过 rustup 在 RISC-V 平台上完成工具链部署。同时，为确保在使用 sudo 执行构建与安装命令时 Rust 环境可被正确继承，本研究对系统 sudo 配置进行修订，使 Cargo 与 rustc 路径能够在特权环境中正常生效。此外，通过显式添加 riscv64gc-unknown-linux-gnu 目标，本研究确保 Rust 编译器具备针对 RISC-V 架构生成二进制文件的能力。

在构建配置适配方面，本研究针对 Kata Containers 构建系统中默认使用的目标三元组与 RISC-V 实际支持情况不一致的问题，对 utils.mk 中的 TRIPLE 生成逻辑进行了定向修订，使其在 RISC-V 平台上正确选择 riscv64gc-unknown-linux-gnu 目标，从而避免构建阶段因工具链不匹配而失败。

在 Go 工具链方面，本研究统一采用 Go 1.22.5 版本，并通过配置 Go Modules、代理源及校验策略，确保 Kata Containers 构建过程中涉及的 Go 组件与辅助工具能够稳定获取依赖。针对构建过程中使用的 yq 工具因版本升级导致命

令语法不兼容的问题，本研究进一步修订了相关构建脚本中对 `yq` 的调用方式，以保证版本解析逻辑在 RISC-V 环境下正确执行。

在源码构建阶段，本研究从官方镜像仓库获取 Kata Containers 源码，并严格遵循其 `Makefile` 定义的构建流程，依次完成基础库、`agent` 以及 `runtime` 组件的构建，最终通过官方提供的安装目标将运行时组件部署至系统环境中。该方案在最大程度上复用了 Kata Containers 官方构建路径，减少了非必要的人为干预，从而提升研究结论的可复现性。

## 研究结果

研究表明，在完成系统依赖安装、Rust 与 Go 工具链配置、构建脚本定向修订以及官方构建流程执行后，Kata Containers 3.2 能够在 RISC-V64 架构平台上成功完成源码构建与运行时安装。本研究通过验证 `kata-runtime` 与 `kata-agent` 的版本信息，确认生成的核心组件已正确针对 RISC-V 平台编译，并能够在目标系统环境中被正常识别和调用。

该研究结果验证了 Kata Containers 在 RISC-V 架构环境中的工程可行性，说明在合理处理多语言工具链差异、构建目标选择与辅助工具兼容性问题的前提下，具备较高系统复杂度与安全要求的容器运行时同样可以在新兴指令集架构上完成移植<sup>[20]</sup>。本研究形成的构建与安装流程具有明确的依赖边界和较强的可复现性，可为 RISC-V 数据中心与云原生平台引入高安全隔离容器运行时提供重要技术参考，也为后续在该架构上开展安全关键型云原生应用部署奠定了工程基础。

### 2.2.2.9.Flannel

#### 研究内容

本研究围绕 Kubernetes 容器网络体系中的基础组件 Flannel 在 RISC-V64 架构平台上的移植与构建可行性展开，研究内容聚焦于整体性工程目标的界定。Flannel 作为 Kubernetes 生态中广泛采用的轻量级覆盖网络实现，其核心作用在于为 Pod 提供统一、连续的虚拟地址空间，从而保障集群内容容器之间的基础通信能力。本研究的核心目标在于验证 Flannel v0.26.7 是否能够在 RISC-V 架构环境中，通过标准 Go 工具链完成源码编译，并生成可在该架构上直接运行的

flanneld 可执行程序，从而为 PolyOS Server 等 RISC-V 平台构建完整的云原生网络基础设施提供支撑。研究内容不涉及多网络后端性能对比、复杂网络策略验证或大规模集群通信评估，而是集中于 Flannel 在 RISC-V 平台上的工程可编译性、构建产物的架构正确性以及基础功能可用性验证。

## 研究方案

在研究方案设计上，本研究采用“官方工具链、最小依赖、静态构建优先”的工程实施策略，对 Flannel 在 RISC-V 平台上的构建过程进行系统规划。鉴于 Flannel 完全基于 Go 语言实现，研究方案首先从构建环境入手，在 RISC-V 平台上部署官方发布的 Go 1.24.2 版本，并通过解压与环境变量配置的方式，将其纳避免系统级编译工具链中。通过配置 PATH 与 GOPROXY，本研究确保 Flannel 构建过程中 Go 编译器与依赖解析行为稳定可控，同时减少外部网络条件对构建流程的影响。

在源码准备阶段，本研究从 Flannel 官方仓库获取源码，并严格切换至 v0.26.7 版本标签，以保证研究对象与版本声明的一致性和可复现性。在构建策略选择上，本研究明确采用静态链接方式进行编译，通过设置 GOOS、GOARCH 以及禁用 CGO 的方式，生成不依赖系统动态库的 flanneld 可执行文件。该方案能够最大程度降低运行环境差异对程序部署与执行的影响，同时也更符合容器网络组件在多节点、多环境中分发部署的实际需求。

在构建执行阶段，本研究直接复用 Flannel 官方 Makefile 中定义的构建目标，通过一次性构建生成目标二进制文件，而不对源码逻辑或构建脚本进行额外修改，从而确保构建结果能够真实反映 Flannel 在 RISC-V 平台上的原生适配能力。

## 研究结果

研究表明，在完成 Go 工具链配置并采用静态构建策略后，Flannel v0.26.7 能够在 RISC-V64 架构平台上成功完成源码编译。本研究获得的 flanneld 可执行文件经格式检查确认其为 RISC-V64 架构的 ELF 二进制文件，并通过版本查询验证其版本号与源码标签一致，表明构建过程未引入架构或版本偏差。

该研究结果验证了 Flannel 在 RISC-V 架构环境中的工程可行性，说明其代码实现与构建体系对新兴指令集架构具备良好的适应能力。本研究形成的构建流程具有明确的工具链依赖和良好的可复现性，可作为在 RISC-V 平台部署 K

ubernetes 容器网络基础设施的重要实践参考，也为后续在该架构上引入其他 CNI 组件和构建完整云原生网络栈奠定了工程基础<sup>[21]</sup>。

### 2.2.2.10. Jaeger

#### 研究内容

本研究围绕分布式追踪系统 Jaeger v2.5.0 在 RISC-V64 架构平台上的移植与构建可行性展开，研究内容聚焦于整体性工程目标的界定。Jaeger 作为云原生可观测性体系中的核心组件，其主要作用在于对分布式系统中的请求链路进行追踪与分析，从而支撑微服务架构下的性能诊断、故障定位与系统优化。本研究的核心目标在于验证 Jaeger 是否能够在 RISC-V 架构环境中，通过官方源码与标准构建流程完成多组件的整体编译，并生成可直接运行的追踪服务二进制程序，从而为 PolyOS Server 在 RISC-V 平台上构建完整的可观测性能力提供基础支撑。研究内容不涉及链路追踪性能对比、存储后端规模化部署或跨集群追踪分析，而是集中于 Jaeger 在 RISC-V 平台上的工程可编译性、构建体系适配情况以及核心组件生成结果的正确性验证。

#### 研究方案

在研究方案设计上，本研究采用“统一工具链、组件整体构建、官方脚本优先”的工程实施策略，对 Jaeger v2.5.0 在 RISC-V 平台上的构建过程进行系统规划。鉴于 Jaeger 项目完全基于 Go 语言实现，且其多个核心组件共享统一的源码仓库与构建逻辑，本研究首先从构建环境入手，在 RISC-V 平台上部署官方支持的 Go 1.24.2 版本工具链。通过解压官方提供的 RISC-V 架构预编译包并配置 PATH 与 GOPROXY 环境变量，本研究确保 Go 编译器与依赖解析过程在构建阶段稳定可控，并有效降低网络环境不确定性对构建流程的影响。

在源码准备阶段，本研究从 Jaeger 官方仓库获取完整源码，并严格切换至 v2.5.0 版本标签，以确保研究对象与版本声明一致，从而保证构建结论的可复现性。在构建策略选择上，本研究未对源码或构建逻辑进行额外修改，而是直接采用 Jaeger 项目提供的统一构建脚本，对 cmd 目录下定义的全部核心组件进行并行编译。该方案能够在一次构建过程中同时生成 all-in-one、collector、query、ingester 等关键服务组件，真实反映 Jaeger 在 RISC-V 平台上的整体构建能力。

在构建执行阶段，研究方案依赖构建脚本内部对各子组件逐一调用 `go build` 的机制，使每个组件均以原生方式针对 `linux/riscv64` 平台完成编译。通过脚本级错误中断机制，本研究能够在构建过程中及时识别潜在的架构不兼容或依赖缺失问题，从而提高构建过程的可控性与透明度。

## 研究结果

研究表明，在完成 Go 工具链配置并采用 Jaeger 官方构建脚本后，Jaeger v2.5.0 能够在 RISC-V64 架构平台上成功完成全部核心组件的源码编译。本研究在构建完成后获得了多项以 `jaeger-` 前缀命名的可执行文件，覆盖 Jaeger 的主要运行模式与功能模块，表明其构建体系已能够正确生成面向 RISC-V 平台的二进制产物。

通过对生成的核心组件执行版本查询与基本运行验证，本研究确认其可执行文件能够在 RISC-V 环境中被正常加载和启动，且版本信息与目标源码版本一致。该研究结果验证了 Jaeger 在 RISC-V 架构环境中的工程可行性，说明其源码实现与构建体系对新兴指令集架构具备良好的适应能力<sup>[23]</sup>。

本研究形成的构建流程具有明确的工具链依赖、统一的构建入口以及良好的可复现性，可作为在 RISC-V 平台部署分布式追踪系统的重要实践参考，也为后续在该架构上进一步集成服务网格、监控系统与日志分析组件，构建完整云原生可观测性体系奠定了坚实的工程基础。

### 2.2.2.11. MinIO

## 研究内容

本研究围绕对象存储系统 MinIO 在 RISC-V64 架构平台上的构建与部署开展，研究内容聚焦于整体性工程目标与技术可行性的验证。MinIO 作为兼容 Amazon S3 接口的高性能对象存储服务，在云原生架构中承担着非结构化数据持久化存储的关键角色，广泛应用于私有云、混合云以及分布式存储场景。本研究的核心目标在于验证 MinIO 官方源码是否能够在 RISC-V 架构环境下，通过标准 Go 语言工具链完成完整编译，并生成可直接运行的对象存储服务二进制程序，从而为 PolyOS Server 在 RISC-V 平台上构建云原生存储基础设施提供工程支撑。研究内容不涉及对象存储性能评测、多节点纠删码部署或 S3 接口兼

容性验证，而是集中于 MinIO 在 RISC-V 平台上的源码可编译性、构建流程稳定性以及生成结果的架构正确性。

## 研究方案

在研究方案设计上，本研究采用“原生工具链、最小修改、单体构建验证”的实施策略，对 MinIO 在 RISC-V 平台上的构建过程进行系统化验证。鉴于 MinIO 完全基于 Go 语言实现，且官方已提供针对 RISC-V 架构的 Go 工具链支持，本研究首先在 RISC-V 环境中部署 Go 1.24.2 版本编译环境，并通过配置 PATH 与 GOPROXY 等环境变量，确保 Go 编译器与依赖解析过程在构建阶段稳定可控，同时降低外部网络环境对依赖获取的干扰。

在源码准备阶段，本研究从 MinIO 官方仓库获取完整源码，并严格切换至指定的稳定发布标签，以确保研究对象与版本声明保持一致，从而保证构建过程与研究结论具备可复现性。在构建策略上，本研究未对 MinIO 源码结构或构建逻辑进行任何功能性修改，而是直接使用 Go 原生编译命令对主入口文件进行构建，生成面向 Linux RISC-V64 平台的可执行文件。该方案能够最大程度反映 MinIO 在 RISC-V 架构上的原生适配程度，避免引入额外补丁对研究结论造成干扰。

在构建执行过程中，本研究通过生成独立命名的二进制文件并对其进行架构识别与运行验证，系统性地确认编译结果是否符合 RISC-V 平台的二进制规范要求，从而完成对 MinIO 构建方案有效性的验证。

## 研究结果

研究表明，在完成 Go 1.24.2 工具链部署并采用 MinIO 官方源码与标准编译流程后，MinIO 能够在 RISC-V64 架构平台上成功完成源码编译，并生成可直接运行的对象存储服务二进制程序。本研究获得的构建产物经文件格式识别验证，确认其为符合 Linux RISC-V64 平台规范的 ELF 可执行文件，表明编译过程已正确针对目标架构完成。

进一步通过版本查询与运行验证，本研究确认生成的 MinIO 可执行程序能够在 RISC-V 环境中正常启动，并正确输出运行时信息与 Go 编译环境标识，说明其核心功能模块已具备在该架构下运行的基本条件。该研究结果验证了 MinIO 在 RISC-V 架构环境中的工程可行性，表明其代码实现与构建体系对新兴开放指令集架构具有良好的适配能力。

本研究形成的构建流程具有工具链依赖清晰、操作步骤简洁、结果可复现等特点，可作为在 RISC-V 平台部署对象存储服务的重要工程参考，也为后续在该架构上构建分布式存储系统、云原生数据服务以及 AI 与边缘计算场景下的持久化存储能力奠定了坚实基础。

#### 2.2.2.12. Ceph

##### 研究内容

本研究围绕统一分布式存储系统 Ceph v19.2.2 在 RISC-V 架构平台上的构建与部署开展，研究内容聚焦于整体性工程目标与平台适配可行性的验证。Ceph 作为同时支持对象存储、块存储与分布式文件系统的通用存储基础设施，在云原生数据中心中承担着关键的数据持久化与高可用支撑角色。本研究的核心目标在于验证 Ceph 在 RISC-V 架构与 PolyOS Server 操作系统环境下，是否能够通过源码级构建流程完整生成核心服务组件，并具备进入后续部署与运行阶段的工程基础。研究内容不涉及集群规模扩展、性能压测或容错策略验证，而是集中于 Ceph 在 RISC-V 平台上的源码可获取性、依赖体系完整性、构建流程稳定性以及核心二进制组件的成功产出情况。

##### 研究方案

在研究方案设计上，本研究采用“平台适配分支、标准构建流程、全量组件编译验证”的实施策略，对 Ceph 在 RISC-V 平台上的构建过程进行系统性验证。考虑到 Ceph 项目本身依赖复杂、模块数量多且构建链路较长，本研究优先选用已针对 RISC-V 架构进行扩展适配的 Ceph 源码分支作为研究对象，以降低底层指令集差异对构建过程的干扰<sup>[24]</sup>。

在源码准备阶段，本研究通过版本控制系统获取基于官方 v19.2.2 版本演进而来的 RISC-V 适配分支源码，并同步初始化全部子模块，以确保构建环境中依赖组件与主仓库状态一致。在依赖准备阶段，本研究采用项目提供的自动化依赖安装脚本完成 Python 模块、构建工具链及第三方库的统一安装，并通过本地缓存机制减少重复下载与环境不确定性对构建流程的影响。

在构建实施阶段，本研究采用项目提供的 CMake 初始化脚本生成构建配置，并使用 Ninja 作为核心构建工具完成全量编译。该方案在不修改默认构建

逻辑的前提下，对 Ceph 所有核心模块进行统一编译，从而真实反映 Ceph 在 RISC-V 架构上的整体构建能力与依赖适配情况。通过标准构建流程生成的二进制文件，作为后续构建结果验证的主要依据。

### 研究结果

研究表明，在完成源码获取、子模块初始化及依赖环境配置后，Ceph v19.2.2 能够在 RISC-V 架构与 PolyOS Server 平台上成功完成源码级构建。本研究通过标准 CMake 初始化与 Ninja 构建流程，成功生成了包括监控、存储与管理在内的多项核心组件二进制文件，覆盖 Ceph 的主要服务角色。

通过对构建输出目录的检查，本研究确认关键服务组件均已正确生成，表明 Ceph 的核心模块在 RISC-V 架构下具备可编译性与工程完整性。该研究结果验证了 Ceph 在 RISC-V 平台上的构建可行性，说明其源码体系与构建工具链能够适配新兴开放指令集架构，并具备进入后续部署与运行验证阶段的基础条件。

本研究形成的构建流程具备依赖管理清晰、构建路径标准、结果可复现等特点，可作为在 RISC-V 平台上引入分布式存储系统的重要工程参考。该成果为 RISC-V 架构在云原生数据基础设施领域的进一步应用奠定了关键技术基础，也为后续构建对象存储、块存储与分布式文件系统的一体化解决方案提供了可靠支撑。

## 2.2.2.13. KubeVirt

### 研究内容

本研究围绕云原生虚拟化框架 KubeVirt v1.4.0 在 RISC-V64 架构平台上的本地原生构建开展，研究内容聚焦于整体性工程目标与关键能力覆盖情况的验证。KubeVirt 通过在 Kubernetes 之上引入虚拟机工作负载，实现容器与虚拟机的统一编排，是云原生基础设施中承载异构计算形态的重要技术组件。本研究的核心研究内容在于验证 KubeVirt 是否能够在 RISC-V 主机环境中完成完整的源码构建流程，生成覆盖控制面与工具链的本地可执行文件，并成功构建面向 RISC-V 架构的控制面容器镜像，从而为 RISC-V 平台引入原生云虚拟化能力奠定工程基础。研究内容不涉及虚拟机运行时性能评估、KVM 功能完整性测试或

多节点集群部署验证，而是集中于 KubeVirt 在 RISC-V 架构下的构建可行性、构建产物完整性以及与 Kubernetes 生态的工程级集成能力。

## 研究方案

在研究方案设计上，本研究采用“架构适配源码、Bazel 原生构建、二进制与镜像并行验证”的实施策略，对 KubeVirt v1.4.0 在 RISC-V 平台上的构建流程进行系统化实施。考虑到 KubeVirt 构建体系高度依赖 Bazel 构建系统，并涉及多语言、多组件协同编译，本研究选用已完成 RISC-V 架构适配的 KubeVirt 源码仓库作为研究对象，以避免在构建过程中引入额外的指令集兼容性问题。

在构建准备阶段，本研究通过源码级方式获取完整的 KubeVirt 项目，并在 RISC-V 主机环境中配置目标架构标识与可选的镜像命名参数，使构建流程能够在不依赖交叉编译的前提下，以本地原生方式生成 RISC-V64 架构产物。在构建实施阶段，本研究首先强调对 KubeVirt 各类命令行工具与辅助组件的本地二进制构建，通过 Bazel 构建规则生成 virtctl 及相关运维、性能测试工具，从而验证核心工具链在 RISC-V 平台上的可编译性。

在完成二进制构建后，研究方案进一步覆盖控制面容器镜像的构建流程，通过 Bazel 驱动的镜像构建机制，生成 virt-api、virt-controller、virt-handler、virt-launcher 与 virt-operator 等关键组件镜像，确保 KubeVirt 的控制面能够以标准容器形式在 Kubernetes 环境中运行。同时，通过生成部署清单文件，验证 KubeVirt 在 RISC-V 架构下具备完整的部署描述能力，为后续集群部署提供必要的资源定义基础。

## 研究结果

研究结果表明，在采用已适配 RISC-V 架构的 KubeVirt v1.4.0 源码并执行标准 Bazel 构建流程后，KubeVirt 能够在 RISC-V64 主机环境中成功完成本地原生构建。本研究确认生成了多项关键命令行工具二进制文件，其文件格式与架构标识均符合 Linux RISC-V64 平台规范，表明 KubeVirt 的用户侧工具链在该架构下具备工程可用性<sup>[25]</sup>。

进一步的构建结果显示，KubeVirt 控制面各核心组件的容器镜像能够成功生成，并可根据配置推送至指定的镜像仓库，说明其构建体系已完整覆盖 RISC-V 架构下的容器化交付模式。同时，部署所需的 Kubernetes 资源清单文件能够正常生成，表明 KubeVirt 在 RISC-V 平台上具备完整的部署描述能力，与 Ku

bernetes 原生 API 的集成路径保持一致。

综上所述,本研究验证了 KubeVirt v1.4.0 在 RISC-V 架构环境中的构建可行性与工程完整性,证明其能够在不依赖仿真或跨架构构建的前提下,完成从源码到二进制与容器镜像的全链路产出。该研究结果为 RISC-V 平台引入云原生虚拟化能力提供了关键技术依据,也为后续在边缘计算、混合工作负载调度及异构算力融合场景中部署 KubeVirt 奠定了坚实的工程基础。

### 2.2.3. 数据库适配

#### 2.2.3.1. openGauss

##### 研究内容

本研究面向 openGauss 在 RISC-V 平台(以 SG2042 服务器为代表)的可构建与可部署能力开展系统性移植工作,研究内容聚焦于“从 src-openEuler 源码仓库派生适配分支、补齐第三方依赖、修复架构指令不兼容点、完成 RPM 方式构建与安装验证”的完整闭环。openGauss 的软件包构建高度依赖第三方库与平台特性,其中 DCF 作为关键依赖组件在 SG2042 上无法直接满足部署要求,且 openGauss 源码中存在与特定指令扩展相关的汇编与运行时检测逻辑。本研究的目标是在不改变 openGauss 发行形态的前提下,通过最小必要补丁集实现 RISC-V64 的可编译、可安装与可启动,并以数据库初始化与单机启动作为功能可用性的最低验证标准。

##### 研究方案

本研究采用“依赖先行、基础设施再适配、最后构建验证”的方案组织移植流程,以确保每一步变更可追溯、可复现,并避免将依赖问题与主仓库代码问题混杂在同一阶段导致定位困难。本研究首先从 src-openEuler 提供的 openGauss 源码仓库派生维护分支,并以 spec 驱动的 RPM 构建流程作为唯一交付路径,从而保证与 openEuler 生态的软件包体系一致。本研究明确 openGauss 构建所需第三方依赖集合,确保在 RISC-V 环境中可通过系统包管理获取的依赖优先使用系统包,仅对无法直接满足的核心依赖采用源码级适配与打包策略,减少重复构建与维护成本。

在依赖适配阶段,本研究优先处理 DCF 组件,因为在 SG2042 上尝试直

接编译安装 5.0.1 版本的 RISC-V 数据库时，实际依赖链显示 DCF 无法正常工作，导致数据库无法部署。本研究选择将 DCF 升级至 6.0.0，并将升级与架构适配拆解为两个补丁纳入 spec 管理：其一通过版本升级补丁将依赖从 DCF-5.0.1 提升至 6.0.0；其二通过 RISC-V64 支持补丁在 `cm_concurrency`、`cm_utils` 等目录中增加 RISC-V 架构分支与内存屏障实现，针对 SG2042 环境中某些架构指令不可用的情况，以 RISC-V 的 `fence` 指令实现内存序语义，并在 spec 中以 `riscv64` 条件编译方式控制补丁应用范围。DCF 完成适配并以 RPM 形式安装后，本研究再进入 openGauss 主体适配阶段，以避免 DCF 不稳定导致的后续误判。

在 openGauss 主体适配阶段，本研究基于官方 spec 中既有的 RISC-V 支持线索进行再修订，但发现原有 `riscv` 补丁在 SG2042 上由于指令扩展支持差异无法部署，尤其在测试期间 SG2042 不支持 `zihitpause` 指令，因此本研究在 spec 的关键构建段落增加 `riscv64` 条件补丁入口，并在 CMake 配置阶段通过 `riscv64` 专用编译选项显式关闭 `ENABLE_BBOX`，以绕开缺失的 `bbox` 支撑库与相关依赖链。与此同时，本研究将 openGauss 源码侧的修改收敛为两个补丁包：其一针对依赖收敛，删除在 SG2042 上无法成功编译且在引用链中可替代的私有 `securec` 依赖，通过在 `libpq` 构建脚本中将 `securec` 统一抽象为宏变量并替换为 `boundscheck`；其二针对内核与底层接口适配，集中修订架构检测、汇编原语、CRC32C 路径选择、计时逻辑、内存屏障、原子接口与链接库集合等关键点，并补齐因 GCC 版本升级导致的头文件依赖缺失问题。

为保证 RISC-V 平台下的链接一致性，本研究在多处链接参数中引入 `latomic`，并在 `gausskernel` 顶层链接库列表与构建选项文件中保持一致；为避免在 RISC-V 上错误选择 x86 专用 CRC32C 优化实现，本研究在 `port` 目录构建逻辑中对 `riscv64` 移除 `pg_crc32c_choose` 与 `pg_crc32c_sse42` 等源文件，并在相关 Makefile 中同步去除对应目标文件；为解决运行时平台识别与配置脚本链问题，本研究补齐 `config.guess/config.sub` 的 `riscv64` 识别分支，并在 `configure` 脚本中强化 `PLATFORM_ARCH` 的识别逻辑。对于内存屏障与同步语义，本研究在 `gtm_atomic` 与 `commproxy` 接口中以 RISC-V `fence` 与编译器内存栅栏实现读写屏障语义，并在其他架构分支保持原行为，以降低跨平台副作用。对于与 openEuler OS 条件编译相关的 SM2 密钥接口差异，本研究按 `WITH_OPENE`

ULER\_OS 条件控制 EVP\_PKEY 类型设置逻辑，避免在特定环境下调用不兼容的接口路径。

在构建与安装实施阶段，本研究严格以 RPM 工程化路径完成最终交付：先构建并安装已适配的 DCF 6.0.0 RPM，再导入 openGauss 源码与补丁仓库内容至 rpmbuild/SOURCES，执行 rpmbuild 生成 openGauss server RPM，并在安装后完成最小可用验证流程，包括设置 GAUSSHOME、LD\_LIBRARY\_PATH 与 PATH，创建数据与日志目录，执行 gs\_initdb 完成数据库初始化，并以 single\_node 模式启动服务以确认可运行性<sup>[26]</sup>。

## 研究结果

研究表明，通过对 DCF 依赖的版本升级与 RISC-V64 适配补丁引入，并结合 openGauss spec 与源码侧的针对性修订，本研究在 SG2042 RISC-V64 环境中实现了 openGauss 的 RPM 方式构建、安装与启动验证闭环。DCF 由 5.0.1 升级至 6.0.0 后，结合对 cm\_spinlock、线程架构分支、checksum 条件编译与内存屏障实现的修订，使其能够在 riscv64 架构下完成构建与安装，从而消除了 openGauss 部署路径中的关键依赖阻塞。

在 openGauss 主体部分，本研究通过关闭 ENABLE\_BBOX 并在构建系统中提供明确的可选项开关，规避了 SG2042 环境下因 bbox 支撑缺失与相关指令扩展不满足导致的构建与部署失败；通过引入 latomic 与修订链接参数，解决了 RISC-V 平台上原子操作相关的链接需求；通过移除 x86 专用 CRC32C 实现路径并回退至通用实现，保证了在 riscv64 下的功能正确性；通过替换 secur ec 依赖为 boundscheck 并在 libpq 构建脚本中以宏变量方式统一引用，解决了 SG2042 上 securec 编译失败导致的依赖不可满足问题；通过补齐平台识别脚本与 configure 架构检测逻辑，确保构建系统能够稳定识别 riscv64 并进入正确的构建分支。最终，在完成 RPM 安装后，本研究通过 gs\_initdb 成功初始化数据目录，并以 gs\_ctl 在 single\_node 模式成功启动数据库服务，表明 openGauss 在目标 RISC-V64 环境中达到“可安装、可初始化、可启动”的最低可用标准。

综上，本研究以依赖治理与底层适配为主线，在保持 openEuler RPM 交付形态不变的前提下完成 openGauss 在 RISC-V64（SG2042）上的工程化移植与验证，为后续开展性能调优、指令扩展适配差异分析以及多节点形态验证提供了可复现的构建基线与补丁资产。

### 2.2.3.2.MySQL

#### 研究内容

本研究面向 RISC-V64 环境下 MySQL 8.0.39 的源码级构建与本地部署开展验证性移植工作，研究内容覆盖编译工具链与运行依赖准备、关键第三方组件 `rpcsvc-proto` 的补齐、基于 CMake 的源码编译安装流程，以及以 `my.cnf` 为核心的初始化与服务启动验证。本研究的目标是在目标 RISC-V 服务器上完成 MySQL 的可编译、可安装、可初始化与可登录验证闭环，并形成可复现的构建参数集合与部署步骤，以支撑后续在 RISC-V 平台上的数据库软件栈完善与应用适配。

#### 研究方案

本研究采用“依赖补齐→源码编译→部署初始化→服务验证”的顺序组织实施，以确保每个阶段的输入与输出边界清晰、故障定位可复现。依赖准备阶段，本研究首先通过系统包管理器安装 MySQL 构建所需的基础组件，包括 CMake、GCC/G++、OpenSSL 开发包、ncurses 开发包、libaio、libtirpc-devel、openldap-devel、bison、rpcgen 与 m4 等，确保编译链路、加密库、终端依赖、异步 IO 与 RPC 相关头文件满足 MySQL 8.0 的构建要求。由于系统默认环境中缺失 `rpcsvc-proto` 可能导致 RPC 相关接口与头文件不完整，本研究额外采用源码方式安装 `rpcsvc-proto 1.4`，按“下载、解压、configure、make、make install”的标准 Autotools 流程将其安装到系统路径中，以确保后续 MySQL 构建阶段能够正确找到对应的 RPC 服务定义与生成工具链。

源码编译阶段，本研究选择 `mysql-boost-8.0.39` 发行包作为源码输入，以避免构建过程中在线获取 Boost 带来的网络不确定性，并通过显式指定 `WITH_BOOST` 路径将 Boost 1.77.0 固定为离线依赖输入。本研究在源码目录中采用 `out-of-source` 构建方式创建 `build` 目录，使用 CMake 配置生成构建系统，并以 `mysql_release` 作为构建配置基线，同时通过参数显式声明安装前缀、数据目录、Unix Socket 路径、配置目录、`systemd` pid 目录、默认字符集与排序规则，并启用 InnoDB、Archive、MyISAM 与 Blackhole 等存储引擎以匹配通用服务器形态的功能需求。为降低在新架构平台上因激进优化导致的不可复现问题，本研究将编译优化级别固定为 `O0`，并开启调试符号与帧指针保留，同时统一为 C

与 C++ 设定 `-fPIC` 与 `-fno-omit-frame-pointer`，确保在后续调试、回溯与问题复现时具有一致的符号与栈信息。本研究采用并行编译与安装以缩短构建周期，并在 `make install` 后以安装目录为交付边界进入部署阶段。

部署初始化阶段，本研究以 `/etc/mysql/my.cnf` 为唯一配置入口，按 `mysqld_safe`、`client`、`mysqldump`、`mysqld` 等段落配置日志、`pid`、端口、`socket`、`basedir`、`datadir`、`tmpdir` 与字符集，并针对 InnoDB 设置 `buffer pool`、`redo log capacity`、`log buffer size`、刷盘策略、锁等待超时与读写 IO 线程数，以满足基本可运行与可观测要求。本研究在系统层面创建 `mysql` 用户作为守护进程运行主体，并通过 `chown` 将配置文件与数据目录的属主属组调整为 `mysql:mysql`，保证初始化与运行阶段不会因权限不足失败。服务脚本方面，本研究复用安装输出中的 `mysql.server`，调整其执行权限后复制到 `/etc/init.d/`，并按需要启用开机自启动配置，以形成与传统 SysV 服务管理一致的启动路径。环境变量方面，本研究将 `/usr/local/mysql/bin` 纳入系统 `PATH`，使客户端工具链在交互与脚本中可直接调用。

验证阶段，本研究使用 `mysqld --initialize` 在指定 `basedir` 与 `datadir` 下以 `mysql` 用户完成初始化，随后通过后台方式或 `service mysql start` 启动服务，并以 TCP 回环地址 `127.0.0.1` 与端口 `3306` 进行 `mysql` 客户端登录验证。本研究在完成首次登录后执行 `root` 密码修改与权限刷新，以确认账户管理链路可用，并以二次登录成功作为“服务可持续可用”的最小验证标准，从而完成从构建到运行的闭环确认。

## 研究结果

研究结果表明，在完成系统依赖与 `rpcsvc-proto` 组件补齐后，本研究在目标 RISC-V64 环境中实现了 MySQL 8.0.39 的源码编译与安装，并通过离线 `Boost` 输入与明确的 `CMake` 配置参数保证了构建过程的可重复性。在部署环节，本研究通过统一的 `my.cnf` 配置完成 `basedir`、`datadir`、`socket`、端口与 InnoDB 关键参数设置，并通过创建 `mysql` 用户、修正配置与数据目录权限、部署服务脚本与配置 `PATH` 等措施，使 MySQL 能够完成初始化并以服务方式启动。运行验证结果显示，本研究能够通过 `mysql` 客户端以 TCP 方式连接本地实例，完成 `root` 密码修改与权限刷新，并可再次成功登录，表明 MySQL 在该 RISC-V 平台达到“可编译、可安装、可初始化、可启动、可登录”的最低可用标准，从而

为后续在 RISC-V 数据中心场景下开展性能调优、稳定性验证与上层应用适配提供了可复现的构建与部署基线[27]。

### 2.2.3.3.MariaDB

#### 研究内容

本研究围绕 MariaDB 11.4.2 在 RISC-V64 架构上的源码级构建与本地部署展开，目标是在 RISC-V 服务器环境中完成 MariaDB 数据库从依赖准备、源码编译、系统部署到运行验证的完整闭环。研究重点在于验证 MariaDB 在 RISC-V 平台上的可构建性与可运行性，并形成一套参数明确、步骤清晰、可重复执行的构建与部署流程，为后续在 RISC-V 平台上开展数据库性能评估、稳定性测试及云原生集成提供基础支撑。

#### 研究方案

本研究采用“系统依赖准备、源码构建、安装部署、初始化与运行验证”的分阶段实施方案，各阶段之间输入输出边界明确，便于问题定位与复现。

在环境准备阶段，研究首先通过系统包管理器安装 MariaDB 构建所需的基础工具链与库依赖，包括 CMake、GCC/G++、OpenSSL 开发包、ncurses、libai o、libtirpc-devel、openldap-devel、bison、rpcgen 及 m4 等组件。这些依赖覆盖了 MariaDB 在构建过程中所需的编译工具、加密支持、终端接口、异步 I/O 与 RPC 相关功能，确保后续 CMake 配置阶段能够顺利完成特性检测与构建文件生成。

在源码获取阶段，研究选用 MariaDB 官方发布的 mariadb-11.4.2 源码压缩包作为输入，优先从 archive.mariadb.org 获取源码，以确保版本一致性与长期可获取性。源码统一放置于 /home/mariadb 目录下，作为构建与安装阶段的工作基准目录。

在构建阶段，本研究采用 out-of-source 构建方式，在源码目录下单独创建 build 子目录，通过 CMake 进行配置与生成。构建配置中明确指定安装前缀、数据目录、系统配置目录、Unix Socket 路径以及默认字符集和排序规则，并显式启用 InnoDB、Archive 与 Blackhole 等常用存储引擎，同时关闭 Tokudb 与 Lobwrap 等在目标环境中不必要或可能带来额外复杂性的组件。为提升可调试性

与可复现性，本研究将 C 与 C++ 编译选项统一设置为关闭优化（O0），保留调试信息与帧指针，并开启位置无关代码（PIC），以避免在 RISC-V 新平台上因激进优化导致的不稳定或难以复现问题。完成 CMake 配置后，采用并行方式执行编译与安装，生成 MariaDB 的完整二进制与支持文件。

在部署阶段，本研究为 MariaDB 创建独立的系统用户与用户组，以最小权限原则运行数据库服务，并统一规划安装目录、数据目录、日志目录与配置文件目录结构。通过显式创建并设置 /usr/local/mariadb、/madata/data、/var/log/mariadb 等目录的属主属组，确保初始化与运行阶段不会因权限问题失败。配置文件方面，本研究在 /etc/mariadb/my.cnf 中集中定义客户端与服务端参数，包括端口、socket、basedir、datadir、日志路径以及 InnoDB 的基础内存与日志参数，形成单一、可审计的运行配置入口。

在初始化与运行验证阶段，本研究使用 MariaDB 自带的 mariadb-install-db 脚本，以 mariadb 用户身份完成数据目录初始化。随后通过直接启动 mariadb 进程加载指定配置文件，使数据库实例在 RISC-V 平台上进入运行状态。客户端验证阶段，研究使用 Unix Socket 方式连接本地 MariaDB 实例，验证服务可访问性与基本交互能力，以确认数据库具备最小可用运行条件。

## 研究结果

研究结果表明，在完成系统依赖准备并采用上述构建与部署方案后，MariaDB 11.4.2 能够在目标 RISC-V64 环境中成功完成源码编译与安装。通过明确的 CMake 构建参数与保守的编译选项设置，构建过程具有良好的稳定性和可重复性。部署阶段中，独立用户运行模型、规范化的目录布局以及集中式配置文件设计，使数据库初始化与启动过程顺利完成。运行验证结果显示，MariaDB 服务能够正常启动，并可通过本地 Socket 方式成功连接，表明该版本在 RISC-V 平台上已达到“可构建、可安装、可初始化、可运行”的基本可用状态。

该研究为 MariaDB 在 RISC-V 架构上的进一步应用奠定了技术基础，为后续在 PolyOS Server 或更大规模云原生环境中开展数据库性能评估、高可用部署及与上层中间件的集成提供了可复现的构建与部署基线。

## 2.2.3.4.Milvus

### 研究内容

本研究以开源向量数据库 Milvus 为研究对象,围绕其在 RISC-V 架构上的完整可用性与高性能实现展开系统性研究。研究内容聚焦三个核心目标:其一,实现 Milvus 全量版在 RISC-V64 平台上的端到端功能适配,覆盖向量数据插入、索引构建、相似性搜索及分布式运行等完整能力;其二,针对 Milvus 核心向量执行引擎 Knowhere,引入并深度利用 RISC-V Vector Extension (RVV) 指令集,对相似性计算等关键算子进行体系结构感知优化;其三,通过在真实 RISC-V 硬件平台上的定量测试,系统评估 RVV 向量化优化在性能、能效及指令执行效率等方面的实际效果,从而验证 RISC-V 在人工智能与向量检索基础设施领域的工程可行性与竞争力。

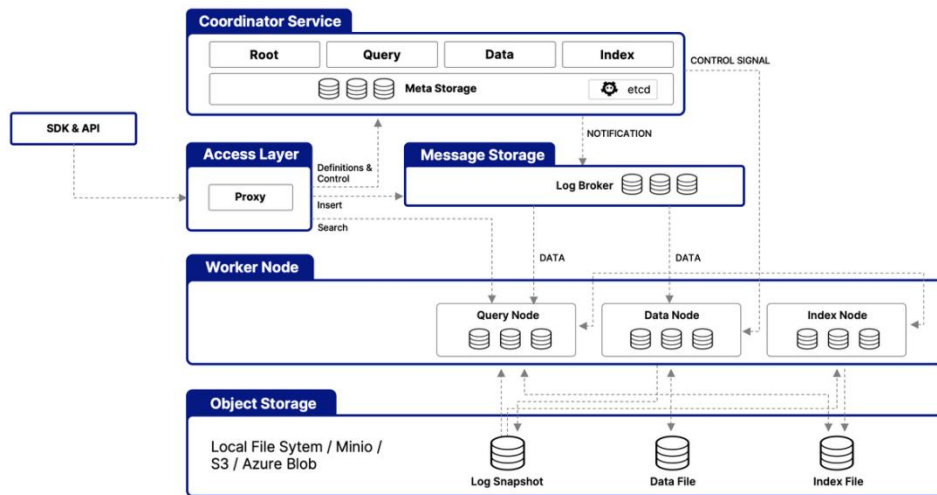


图 5 Milvus 高度解耦的系统架构

### 研究方案

本研究采用“由底层加速到全栈协同”的渐进式研究方案,从计算内核、编译工具链到数据库系统整体逐层推进适配与验证工作。研究首先从 Milvus 的计算核心 Knowhere 入手。Knowhere 作为 Milvus 的向量相似性搜索执行引擎,承担了内积、L2 距离、范数计算等高频计算任务,其性能直接决定整体系统的查询吞吐与延迟表现。本研究在保持 Milvus 原有模块解耦与无状态设计原则不变的前提下,对 Knowhere 的距离计算内核进行重构,引入 RISC-V Vector (RVV) SIMD 后端支持,使浮点、半精度及低精度整数等多种数据类型能够在 RISC-V 平台上获得指令级并行加速<sup>[28]</sup>。

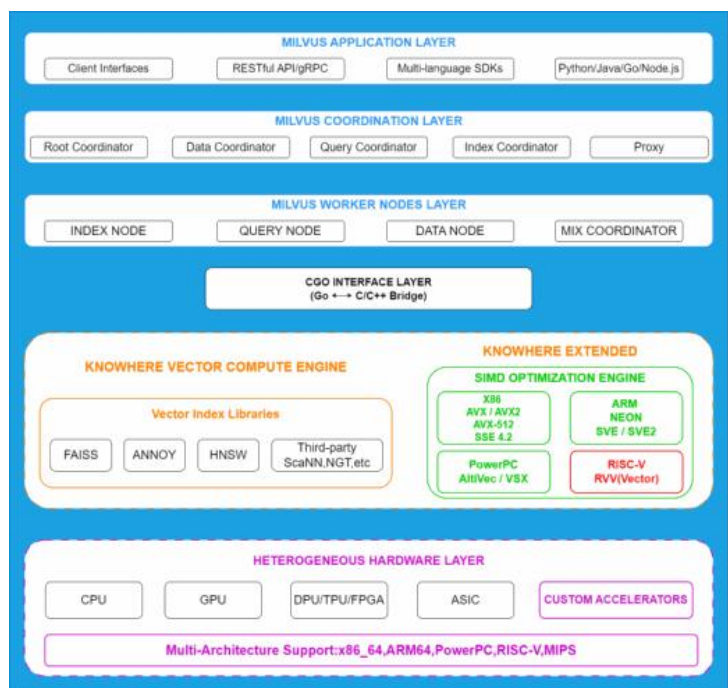


图 6 Knowhere 在 Milvus 架构上下游位置

在底层优化基础上，本研究同步构建 RISC-V 原生的全栈编译与构建环境。依托 PolyOS Server（基于 openEuler 的 RISC-V 衍生发行版），研究从零搭建支持 RVV 1.0 的 GCC 编译工具链，目标三元组为 `rv64gcv_zvfhmin`，并完成 Conan、CMake、Go、Rust 等关键开发工具在 RISC-V 架构下的适配与联调。该过程系统性解决了 ABI 兼容、头文件缺失、第三方依赖库交叉构建等一系列工程问题，为 Milvus 复杂依赖体系在 RISC-V 平台上的稳定构建提供基础保障。

在完成计算内核与工具链适配后，本研究进一步开展 Milvus 全栈功能验证与系统集成工作。在 RVV 优化版 Knowhere 的基础上，逐一验证 Milvus 各核心组件在 RISC-V 平台上的稳定性与一致性，包括 QueryNode、IndexNode、DataNode 以及 Standalone 部署模式，并完成 etcd、MinIO 等关键外部依赖服务的集成。最终实现 Milvus 2.6.1 全量版本在 RISC-V64 架构上的完整部署与稳定运行，确保其在功能层面与主流架构平台保持一致。

## 研究结果

研究结果表明，Milvus 全量版已在 RISC-V64 架构上实现端到端的功能运行，标志着该向量数据库正式进入 RISC-V 基础软件生态体系。在性能层面，基于 RVV 的 Knowhere 向量化优化展现出显著效果。在 PolyOS Server、GCC 15 与 RVV 1.0 (`rv64gcv_zvfhmin`) 环境下，并以 Sophon SG2044（支持 1

28-bit 向量寄存器的 RISC-V64 处理器) 作为验证平台, RVV 优化版本相较于 RISC-V 原始标量实现, 在多类核心算子上获得数量级性能提升。浮点内积与 L2 距离计算分别实现约 9.38 倍与 10.22 倍加速, FP16 混合精度内积加速比达到 13.24 倍, 而向量范数计算由于显著降低访存压力, 最高加速比达到 26.2 6 倍。批处理算子同样表现稳定, 在典型批量场景下保持 3 倍以上的加速效果。

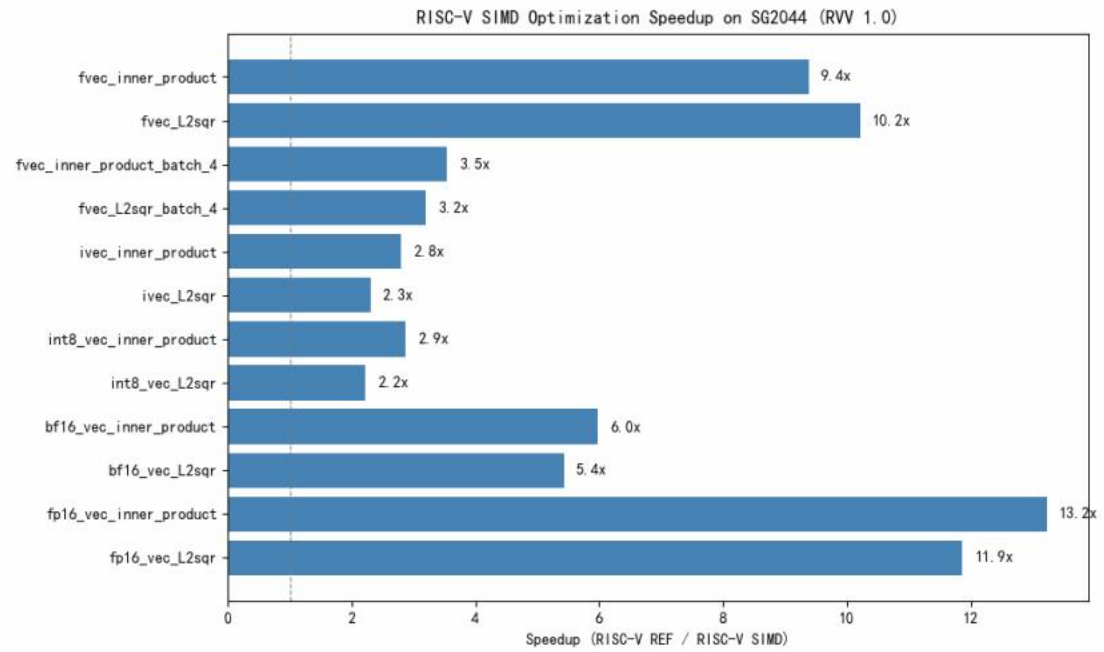


图 7 RISC-V SIMD 优化加速对比结果

在跨架构对比中, 本研究进一步将 RISC-V RVV 优化结果与 ARM 鲲鹏 920 平台的 NEON SIMD 实现进行横向比较。结果显示, 在 FP16、BF16 与 INT8 等低精度计算场景中, RISC-V RVV 表现出明显优势, 整体性能较 ARM NEON 提升约 30% 至 60%; 在标准浮点算子上, 两者性能基本持平。这一结果验证了 RVV 指令集在通用与低精度计算领域的成熟度与良好可移植性。

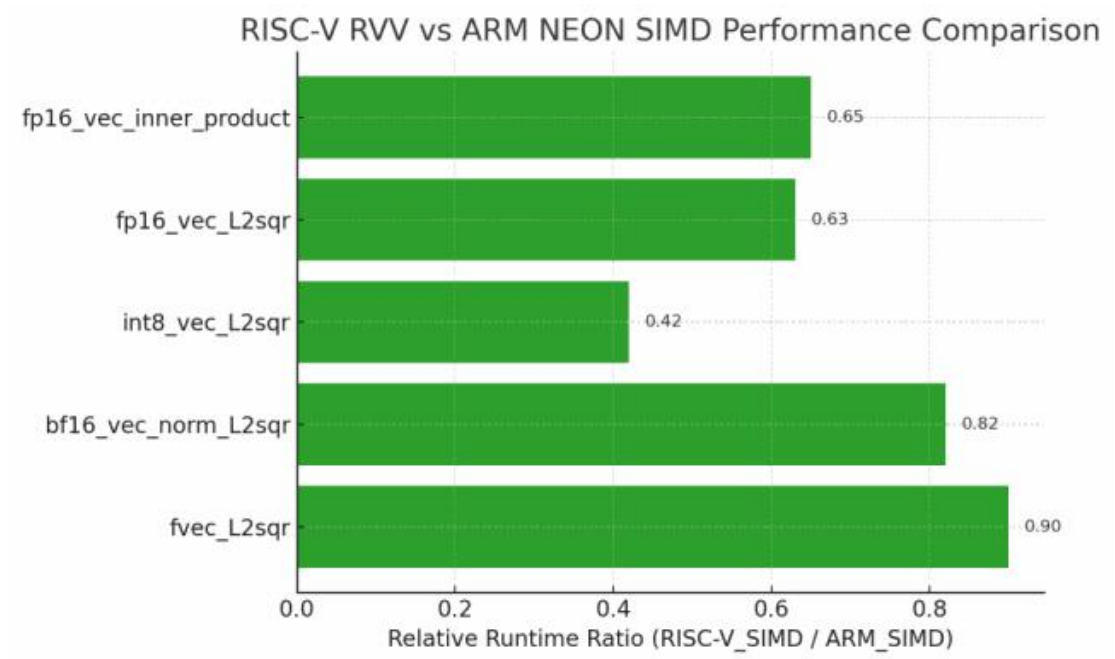


图 8 RISC-V SIMD 对比 ARM NEON 优化加速对比结果

在能效与指令效率方面，性能分析结果显示，RVV 向量化优化后，SG2044 平台上的平均 IPC 相较标量实现提升约 1.45 倍，说明 RVV 不仅有效减少了指令数量，也显著提升了单周期指令吞吐能力。即使在 FP16 与 BF16 等需要额外类型转换的场景下，得益于内存带宽利用率提升与向量宽度优势，端到端性能仍保持增长，体现出 RVV 在 AI 推理与向量检索负载中的综合能效优势。

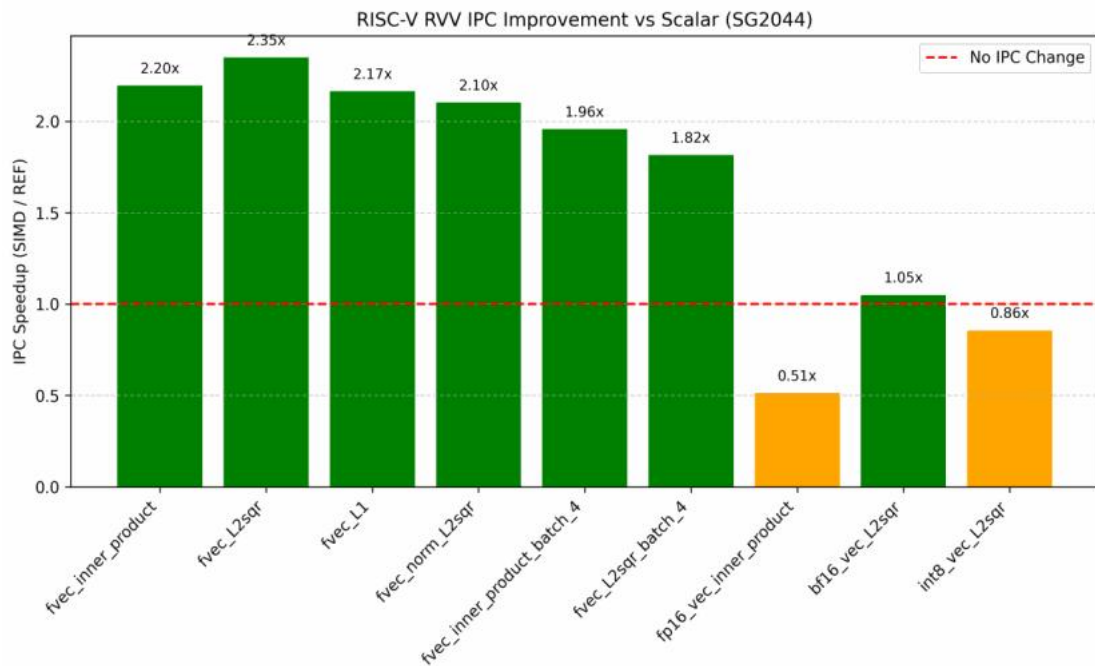


图 9 RISC-V SIMD IPC 优化加速对比结果

综合来看，本研究充分验证了 Milvus 在 RISC-V 架构上的工程可行性与高

性能潜力。通过 RVV 向量化优化，RISC-V 平台在近似最近邻搜索等核心 AI 工作负载中已具备与主流服务器架构正面竞争的能力，为 RISC-V 在人工智能基础设施、大模型推理与智能搜索等新兴应用场景中的规模化落地奠定了坚实基础。

### 2.2.3.5.PostgreSQL

#### 研究内容

本研究以 PostgreSQL 15.8 为研究对象，围绕其在 RISC-V64 架构上的完整移植、构建与运行验证开展系统性工作。研究内容聚焦于三个层面：首先，验证 PostgreSQL 这一复杂、成熟的关系型数据库系统在 RISC-V 平台上的可编译性、可部署性与功能完整性；其次，分析并实践在 RISC-V 处理器（以 SG2042 为代表）上进行数据库构建时所涉及的工具链、依赖库与体系结构适配问题；最后，通过完整安装与初始化流程，评估 PostgreSQL 在 RISC-V 平台上作为通用数据库基础设施的可行性，为后续数据库软件及其生态组件在 RISC-V 架构上的移植提供可复用的技术路径与工程经验。

#### 研究方案

本研究采用以“原生构建、完整功能启用、工程可复现”为核心原则的研究方案。研究首先明确软硬件环境边界，在 SG2042 RISC-V64 服务器与 openEuler 24.03 操作系统环境下，选用社区长期支持且功能完备的 PostgreSQL 15.8 版本作为研究对象。在此基础上，本研究通过系统性梳理 PostgreSQL 的构建依赖，完整安装编译工具链、构建系统组件及数据库功能扩展所需的开发库，确保在 RISC-V 平台上不裁剪、不弱化其核心与扩展能力，从而最大限度验证架构兼容性。

在编译阶段，本研究针对 RISC-V 架构特性，结合 SG2042 处理器的微架构特征，设置明确的编译参数以启用 rv64gc 指令集并进行适度的微架构调优，同时保持参数的通用性与可移植性，避免引入特定厂商绑定优化。配置阶段，研究方案选择启用 PostgreSQL 所有成熟稳定的功能模块，包括加密通信、国际化、LDAP 与 PAM 认证、多语言接口以及 systemd 集成等，以检验 RISC-V 平台在复杂软件配置组合下的整体稳定性。

在构建与安装过程中，本研究采用并行编译与完整日志记录相结合的方式，在充分利用多核资源提升编译效率的同时，保证编译过程的可追溯性与问题定位能力。安装完成后，研究方案通过规范的用户隔离与目录权限设计，完成数据库初始化与基础配置，确保 PostgreSQL 运行环境符合安全性与工程实践要求。最后，通过多种启动与管理方式验证数据库服务的可控性与稳定性，为后续在生产或类生产环境中的部署奠定基础<sup>[29]</sup>。

## 研究结果

研究表明，PostgreSQL 15.8 能够在 SG2042 RISC-V64 平台与 openEuler 24.03 操作系统环境下成功完成源码编译、安装与初始化，并具备完整的数据库服务能力。核心服务端程序、客户端工具及配置文件均正确生成，数据库实例能够正常启动并接受本地连接，验证了 PostgreSQL 在 RISC-V 架构上的基础可用性与功能完整性。

从工程角度看，本研究未发现 PostgreSQL 核心代码中存在阻碍其在 RISC-V 架构上运行的系统性缺陷，主要适配工作集中在构建环境准备与依赖库完整性保障层面。这表明 PostgreSQL 作为高度可移植的开源数据库系统，其架构设计已能够较好地适应新兴指令集平台。在性能与运行层面，尽管本研究未对数据库吞吐或延迟进行深入量化评测，但在实际部署与运行过程中，系统表现稳定，未出现与 RISC-V 架构相关的功能性异常。

综合来看，本研究成功验证了 PostgreSQL 在 RISC-V 平台上的可部署性与工程可行性，说明 RISC-V 架构已具备承载复杂关系型数据库系统的基础条件。随着编译器优化水平的提升、硬件平台性能的增强以及数据库生态组件的逐步完善，RISC-V 平台在企业级数据库应用场景中的可用性和竞争力有望持续增强。本研究形成的构建与部署实践，为后续在 RISC-V 架构上移植和运行其他大型基础软件系统提供了可参考、可复现的技术范式。

### 2.2.3.6. TEE/Enclave 适配

蓬莱 TEE（可信执行环境）是 RISC-V 平台目前主流的三个 TEE 之一，也是其中唯一源自国内的 TEE 系统。

TEE 在连接设备中提供一个安全区域，确保敏感数据在隔离、可信的环境中

存储、处理和保护-通过提供隔离的、安全执行的授权软件，实现端到端的安全。换句话说，TEE 表示一个与 SoC 中其他模块隔离并能够执行可信应用程序 (TAs) 的区域。蓬莱通过软硬件协同的方式来支持隔离环境的可扩展性。

目前,聚元 PolyOS 在硬件 Sifive Unmatched 开发板上集成了 Penglai-TVM, 基于 OpenSBI, 支持不可信 host 和 enclaves 的细粒度隔离 (页级别隔离) 这种应用场景的需求。

## 2.3. 基础库适配与优化

### 2.3.1. OpenCV

#### 研究内容

本研究以 OpenCV 4.10.0 为研究对象, 围绕其在 RISC-V 架构上的完整适配、构建与运行验证开展系统性研究。研究内容聚焦于三个核心目标: 其一, 验证 OpenCV 这一成熟且高度复杂的计算机视觉基础软件在 RISC-V 平台上的可编译性、可部署性与功能完整性; 其二, 分析 OpenCV 在 PolyOS Server 环境中对编译工具链、第三方依赖以及 Python 运行时环境的适配需求, 明确其在新兴指令集架构下的工程约束; 其三, 在完成基础功能可用性的前提下, 为后续引入 RISC-V 向量扩展 (RVV) 及算子级性能优化奠定软件与构建体系基础, 从而支撑 AI 与视觉计算场景在 RISC-V 平台上的持续演进。

#### 研究方案

本研究采用“先功能完整、后性能优化”的研究方案, 优先确保 OpenCV 在 RISC-V 架构上的稳定构建与正确运行。在研究实施阶段, 首先选取 OpenCV 官方稳定版本 4.10.0, 并结合 PolyOS Server 的系统环境, 对其源码进行必要的补丁适配, 以解决操作系统差异和构建过程中的兼容性问题。为避免 Python 依赖冲突并保证构建环境的可控性, 本研究通过创建独立的 Python 3.12 虚拟环境, 对 OpenCV 的 Python 绑定进行隔离式构建与安装。

在编译配置方面, 本研究基于 CMake 构建体系, 对 OpenCV 的功能模块和优化选项进行明确取舍。研究方案在构建阶段刻意关闭 NEON、OpenCL 等与 ARM 或其他体系结构强绑定的优化路径, 同时禁用自动优化与帧指针省略, 以提高调试可观测性和构建过程的确定性, 从而更准确地评估 RISC-V 平台下

OpenCV 的基础行为特征。同时，研究方案启用了 Python、示例程序与测试模块的构建，以验证 OpenCV 在多接口、多组件场景下的整体一致性。

在安装与运行阶段，本研究通过显式配置 PYTHONPATH，确保 Python 运行时能够正确加载 OpenCV 模块，并在虚拟环境中对 cv2 模块进行导入与版本验证，从而完成从源码构建到用户态调用的完整闭环。最后，研究方案在总结阶段对潜在的优化方向进行了系统梳理，为后续引入 RISC-V 原生 SIMD 与 RVV 加速提供结构化改进路径。

### 研究结果

研究结果表明，OpenCV 4.10.0 能够在 PolyOS Server 环境下成功完成面向 RISC-V 架构的源码编译、安装与运行验证。通过 Python 虚拟环境加载 cv2 模块并正确输出版本信息，验证了 OpenCV 核心功能及其 Python 接口在 RISC-V 平台上的可用性与一致性，表明该计算机视觉基础库已具备在 RISC-V 架构上作为通用视觉计算组件运行的工程条件。

从适配结果来看，本研究未发现 OpenCV 在核心框架层面存在阻断其在 RISC-V 架构运行的结构性问题，主要适配工作集中在构建参数控制、平台相关优化路径屏蔽以及系统环境差异处理等方面。这说明 OpenCV 的整体架构具有良好的可移植性，为其在新兴指令集平台上的长期演进提供了基础保障。

在性能层面，本研究尚未引入 RISC-V 向量扩展（RVV）或 Universal Intrinsic 的后端实现，因此当前构建版本主要用于功能验证与工程评估。然而，研究已明确后续优化方向，包括构建 RISC-V 原生交叉编译环境、实现 Universal Intrinsic 的 RISC-V 后端支持、基于 RVV 对 DNN 等核心算子进行向量化加速，以及在接口层面保持与 NEON、SSE、AVX 等既有平台的一致性。总体而言，本研究验证了 OpenCV 在 RISC-V 架构上的可行性与工程基础，为其进一步向高性能视觉计算平台演进提供了可靠起点。

## 2.3.2. OpenSSL

### 研究内容

本研究围绕 OpenSSL 在 RISC-V 架构上的密码算法优化展开，重点聚焦于 RISC-V 加密扩展（Zk/Zkn）在对称加密场景中的工程化应用与性能提升效果。研究的核心内容在于验证并实现基于 RISC-V Zkn 扩展指令集的 AES 算法硬

件加速路径，评估其在通用加密模式中的可用性、正确性与优化潜力，从而提升 RISC-V 平台在安全计算与可信基础设施中的整体能力。同时，本研究结合 OpenCV Universal Intrinsic 机制与系统级构建环境，对 RISC-V 向量与加密扩展的协同支持进行了探索性验证，为后续在多类基础软件中复用指令级优化能力提供方法参考。

## 研究方案

本研究采用以指令级能力引入为核心、以库级功能验证为目标的技术方案。在架构层面，本研究以 RISC-V Zkn 扩展为基础，选取 AES 算法作为切入点，原因在于 AES 是 OpenSSL 中最核心、使用最广泛的对称加密算法之一，其性能直接影响 TLS、存储加密与系统安全子模块的整体效率。研究方案首先在 OpenSSL 中引入并封装 Zkn 扩展所提供的专用加密指令，以 RV64 架构下的 aes64esm 指令为代表，通过对指令编码格式进行抽象封装，使其能够被安全、可控地调用于高层加密实现中<sup>[30]</sup>。

在此基础上，本研究实现了基于 Zkn 扩展的 AES 核心运算路径，包括加密、解密以及密钥调度相关操作，并将其分别封装为独立的核心函数，用于替换或增强原有基于通用整数指令集的实现。随后，本研究进一步将这些核心函数集成到 OpenSSL 的多种 AES 工作模式中，使 Zkn 加速能力能够覆盖主流实际应用场景。研究方案重点关注 AES-CCM、AES-GCM、AES-OCB 以及 AES-XTS 等模式，这些模式在网络通信、数据完整性校验与磁盘加密中具有代表性，能够全面反映加密扩展在不同使用语义下的适配情况。

在系统集成层面，本研究同步考虑了构建与部署环境的可扩展性，通过在 Yocto 项目中构建可配置的 meta-layer，使系统能够根据不同 RISC-V 硬件平台所支持的向量与加密扩展能力，自动完成 OpenSSL 及相关数值计算库的优化配置，从而减少人工干预并提升整体工程一致性。整个研究方案强调“指令能力可感知、软件路径可切换、系统构建可自动化”的设计原则。

## 研究结果

研究表明，引入 RISC-V Zkn 扩展后，OpenSSL 中的 AES 算法能够成功利用专用加密指令完成核心运算，相较于仅依赖基础整数指令集的实现路径，在加密与解密操作的执行效率上具有显著优势。本研究实现的 Zkn 优化补丁已完成工程验证，并成功合并至 OpenSSL 上游主分支，表明该方案在正确性、

可维护性与通用性方面均达到了社区接受标准。

从功能覆盖角度来看，基于 Zkn 扩展的 AES 核心函数能够稳定支撑多种主流 AES 工作模式，在加密、认证及密钥管理等关键环节均表现出良好的兼容性，未引入模式级语义差异或行为异常。这说明 RISC-V 加密扩展不仅适用于单一算法优化，也具备在复杂密码协议栈中长期使用的工程可行性。

从系统层面分析，本研究验证了 RISC-V 在安全计算方向上的体系化演进潜力。通过将 Zkn 扩展引入 OpenSSL 并与构建系统、工具链和其他基础软件形成协同关系，RISC-V 平台在密码计算、安全通信和可信基础设施方面的能力得到实质性增强。总体而言，本研究证明了 RISC-V Zkn 扩展在实际密码库中的应用价值，为 RISC-V 架构在高安全、高性能计算场景中的进一步落地提供了关键技术依据。

### 2.3.3. OpenBLAS

#### 研究内容

本研究围绕 OpenBLAS 在 RISC-V 架构上的适配与部署展开，研究目标在于为 PolyOS Server 平台构建稳定、高性能的基础数值计算能力。OpenBLAS 作为实现 BLAS 与 LAPACK 标准接口的核心计算库，是科学计算、机器学习与数据分析软件栈中的关键底层组件。本研究的整体研究内容聚焦于验证 OpenBLAS 在 RISC-V 架构下的可编译性、可运行性与系统集成能力，并为上层人工智能与高性能计算框架提供可靠的线性代数算子支持，从而提升 RISC-V 平台在数值密集型应用场景中的整体性能表现。

#### 研究方案

本研究采用以原生构建与系统级集成为核心的研究方案。在软件版本选择上，本研究选取 OpenBLAS 0.3.28 作为研究对象，该版本在功能完整性与社区维护活跃度之间具有良好平衡，能够代表当前主流 OpenBLAS 的工程实现状态。在构建环境方面，本研究基于 PolyOS Server 的 RISC-V 运行环境，结合 Python 3.12 系列工具链，通过创建隔离的 Python 虚拟环境，避免 OpenBLAS 与系统中其他数值库产生依赖冲突，从而保证构建与验证过程的可重复性。

在编译配置阶段，本研究采用 CMake 作为构建系统，对 OpenBLAS 的功能选项进行了针对性裁剪与配置，关闭与当前研究目标无关的 Fortran 与 Open

MP 支持，重点关注核心 C 接口与共享库形式的构建结果，以提升部署灵活性与系统兼容性。与此同时，本研究在编译过程中启用了调试与安全相关的编译选项，以便在 RISC-V 新兴架构环境下更清晰地观测潜在问题，并为后续性能优化与指令级增强预留空间。

在系统集成层面，本研究通过显式配置头文件搜索路径与动态链接库路径，使 OpenBLAS 能够被系统范围内的其他软件正确识别和调用，从而形成可复用的基础数值计算环境。该方案强调与操作系统、编译器及上层框架的协同，确保 OpenBLAS 能够作为标准 BLAS 实现稳定运行于 RISC-V 平台之上。

### 研究结果

研究表明，OpenBLAS 0.3.28 能够在 PolyOS Server 的 RISC-V 架构环境中顺利完成编译、安装与运行，其核心库文件与接口行为符合预期。通过系统级环境变量配置与工具链验证，本研究确认 OpenBLAS 提供的 BLAS 接口可被正确识别，其版本信息与功能输出与官方实现保持一致，未出现架构相关的功能缺失或运行异常。

从系统支持角度来看，OpenBLAS 在 RISC-V 平台上的成功适配，为上层机器学习与科学计算框架提供了关键的基础算子支持，为后续部署 PyTorch、TensorFlow 等依赖高性能线性代数计算的复杂软件奠定了稳定基础。本研究结果表明，RISC-V 架构在数值计算领域具备良好的软件兼容性潜力，OpenBLAS 作为成熟的数值计算库，其在 RISC-V 平台上的可用性进一步增强了该架构在高性能计算与人工智能应用场景中的实用价值。

总体而言，本研究验证了 OpenBLAS 在 RISC-V 架构上的工程可行性与系统集成能力，为构建完整的 RISC-V 高性能计算软件生态提供了重要支撑，也为后续围绕向量扩展与指令级优化的深入研究提供了可持续演进的基础。

## 3. 服务器语言运行时环境支持

### 3.1. Java

#### 研究内容

围绕 Java 运行时在 RISC-V 架构上的适配与优化展开研究，重点关注 OpenJDK/HotSpot 在新架构上的可移植性、性能表现及其在 PolyOS Server 中的工程化落地。从初步的 RISC-V 上 Java 实现作为起步，以 Zero 端口这一无汇编、纯解释方式实现的 HotSpot 子系统作为起点验证基本可运行性。集中研究为 HotSpot 开发 RISC-V 汇编后端、补全模板解释器、实现 C1/C2 编译器生成器、完善寄存器分配器与调用规范（ABI），并对垃圾回收系统中涉及内存屏障、并发调度、栈帧布局等高度依赖体系结构的部分进行重构。

使 Java 运行时的移植逐步从实验性阶段进入成熟阶段，推动 RISC-V 端口最终以 JEP 422 的形式合入至 JDK 19 主线，正式成为 Java SE 支持的体系结构之一。该端口以 RV64GV 作为目标配置，不仅补齐了 HotSpot 的各关键子系统，还确保了与其他架构一致的内存模型语义、并发机制和垃圾回收器行为，使 Java 在 RISC-V 上能够以统一方式运行、调优并在企业级场景中部署。此外，随着 RISC-V C、V 等扩展指令的快速发展，Java 的矢量化能力、原子操作性能和 JIT 代码生成质量也出现了新的研究空间，特别是与 JDK Vector API 结合，为未来高性能计算场景铺设了技术基础<sup>[22]</sup>。

在 PolyOS Server 上各组件对 Java 的依赖较为深重，在云原生、中间件、DevOps、Web 服务器及大数据等应用中均高度依赖 OpenJDK 运行时。结合社区的演进情况，研究重点不仅包括 HotSpot 在新架构上的技术适配，也包括操作系统级别的工程化能力，例如多版本 JDK 之间的共存与切换、软件仓库的自动化构建与回归测试、对各类 Java 应用的兼容性验证等。构建一个能够在 PolyOS Server 上稳定运行、性能可接受、具有工程可维护性的 Java 软件栈，并推动 RISC-V 在服务器生态中的实际落地。

## 研究方案

本研究的研究方案延续“从体系结构适配到工程化落地”的总体路线，以 OpenJDK RISC-V 移植的上游进展为基底，以 PolyOS Server 的实际运行环境与软件栈要求为约束，重点围绕 Java 运行时在 RISC-V 上的可移植性、可维护性与可部署性开展系统化研究。首先在体系结构层面，以 JEP 422 合入后 OpenJDK 主线中的 RISC-V 端口为主要参考对象，对 HotSpot 各子系统中与新 ISA 密切相关的部分进行二次分析与验证。研究重点包括模板解释器的字节码调度路

径、C1 与 C2 编译器生成器的寄存器分配策略、调用规范与栈帧布局的体系结构一致性、内存屏障在不同垃圾回收器中的语义正确性、并发子系统中与 RISC-V 内存模型相关的实现细节等。为确保这些底层组件在 PolyOS Server 上能够稳定运行，需要通过静态分析、单元测试与微基准测试多种方式确认每一模块在 RV64GV 配置下的行为与预期一致。

在移植验证的基础上，研究方案的第二部分聚焦于构建可用于生产环境的 Java 运行时组件，重点解决 PolyOS Server 在工程化落地时所面临的诸多现实需求。操作系统级的构建体系需要支持从源码构建多版本 OpenJDK，包括 JDK 8、11、17 与 21 等常见企业级版本，使其能够在同一系统上共存、被不同上层应用选择并在需要时进行切换。研究方案针对各版本的 HotSpot 构建链路进行统一建模，包括编译参数、体系结构特性开关、测试集可用性修复、启动时行为差异等，以确保在 PolyOS 的包管理体系中能够实现可重复构建。与此同时，PolyOS Server 的构建平台需要将各版本 JDK 的编译、打包、验收测试整合到 CI/CD 流水线中，因此本研究对构建脚本、回归测试、符号表校验、API 兼容性验证等步骤进行标准化设计，从而保证 JDK 在架构更新和版本迭代时能够维持稳定的质量输出<sup>[31]</sup>。

研究方案的第三部分面向生态验证，重点关注 PolyOS Server 中 Java 的实际应用场景。云平台组件、微服务框架、中间件客户端、大数据处理工具链等均依赖 JDK 的完整性和性能表现，因此需要通过一系列典型应用的构建与运行测试来评估移植质量。本研究选取 Maven、Gradle、Bazel 等主流构建系统，通过自举构建验证其在 PolyOS RISC-V 上的可用性，并进一步选择若干具代表性的开源 Java 服务进行集成测试，分析其运行时对线程调度、GC 行为、JIT 编译质量的依赖情况。通过构建一套覆盖启动性能、吞吐性能、长时间稳定性及内存占用等指标的测试体系，本研究能够在 PolyOS Server 的真实环境中验证 Java 运行时的适配效果，并为进一步优化提供数据支撑。

整体而言，本研究方案通过“体系结构验证—工程化构建—生态验证”的逐层推进机制，确保 Java 运行时能够在 PolyOS Server 上形成稳定、可维护、可扩展的基础能力，为后续面向向量化、高性能计算及更广泛的企业级应用部署奠定技术基础。

## 研究结果

经过持续推进,本研究在 RISC-V 架构下的 Java 运行时研究中取得了从可运行性验证到工程化落地的系统性成果。在体系结构适配方面,HotSpot 的核心模块已经在 RV64GV 下实现完整功能,可正确通过模板解释器、C1 与 C2 编译器的多层级执行路径,垃圾回收器在写屏障、内存模型及并发阶段的行为均保持与其他成熟架构一致。通过对 JEP 422 合并后的代码进行验证,本研究确认了 RISC-V 端口能够在 PolyOS Server 上以统一方式运行 Java 应用,解决了早期 Zero 方式下的性能瓶颈,使 Java 在新架构上具备可调优的 JIT 能力,为后续高性能业务的部署奠定基础。

在工程化构建方面, PolyOS Server 已能够构建、分发并并行管理多版本 JDK, 涵盖从 JDK 8 到 JDK 21 的企业常用版本。研究工作实现了各版本之间的依赖隔离、符号路径一致性以及可重复构建特性,使 JDK 的打包能够稳定融入 PolyOS 的构建体系。多版本并存能力使系统能够在不同应用场景间灵活选择运行时环境,从而支持云原生框架、分布式中间件和大型构建系统等多种类别的企业级软件。配合构建系统的自动化测试机制,本研究已建立一套由单元测试、API 兼容性验证、启动行为校验、JIT 性能对比等组成的回归体系,为发行版的长期维护提供了技术保障。

在生态验证方面, PolyOS Server 已成功使用 RISC-V 架构下的 JDK 构建并运行大量开源 Java 工程。包括 Maven、Gradle、Bazel 在内的构建系统均已能够在 PolyOS RISC-V 上自举,通过对真实项目的编译测试验证了 JDK 工具链的稳定性。在应用层面,若干典型 Java 服务在 PolyOS 上完成了运行验证,性能指标显示相对于 Zero 模式已有数量级提升, JIT 编译器的优化路径能够有效改善吞吐率与延迟特性。在长时间运行测试中,各垃圾回收器的表现稳定,线程调度行为与内存管理机制未出现结构性兼容问题,说明 RISC-V Java 运行时已达到可用于生产环境的软件质量。

## 3.2. Rust

### 研究内容

以 Rust 语言为系统级/底层软件开发语言,面向 RISC-V 架构,目标是在 RISC-V 芯片(例如 HiFive 等开发板)上构建健壮、安全、高性能的运行时代外围支持环境,为基于 Rust 的系统软件、驱动和嵌入式/服务器操作系统(以

下简称“目标系统”)提供语言级基础与生态支持。Rust 语言以其所有权机制 (ownership) + 借用检查 (borrow checking) + 零成本抽象 (zero-cost abstractions) + 强大的并发 / 内存安全保证 (memory safety & concurrency safety) 为特征, 非常适合系统级、对安全性与可靠性有高要求的场景。选择 Rust 用于 RISC-V, 不仅可以利用其语言的安全语义减少传统 C/C++ 带来的内存错误, 还允许程序员在必要时使用 `unsafe` 明确标记不安全代码块, 从而在保证安全性的同时保留对底层硬件的控制能力。这对于操作系统内核、引导 (BSP, boot support package)、外围设备驱动、裸机 / 嵌入式系统、实时系统等场景尤为重要。

首先, 针对 RISC-V 架构上的 LLVM 工具链 (LLVM backend for RISC-V) 和汇编/链接器 / runtime support, 通过 Rust 语言库 (crate) 构建针对某款开发板 (例如 HiFive) 的 BSP, 使该板卡能够在 Rust 语言下启动、初始化硬件 (例如时钟、中断、中断控制器、内存控制器、外设接口等), 并为使用的外围设备 (如串口、GPIO、以太网、存储接口等) 提供 Rust 驱动支持。其次, 使得 Rust 编译器 (rustc + cargo) 能够生成适用于 RV32 和 RV64 (包括 RV32Q / RV64Q / 未来扩展 / 特定配置) 的二进制, 使得 target triple 对应于 RISC-V 架构, 以便无论是在 32 位还是 64 位系统上都能编译和运行 Rust 应用 / 系统代码。第三, 搭建一个适合 Rust 代码运行的 RISC-V 环境, 该环境应支持裸机 (bare metal)、单核或多核 (若硬件支持)、特殊指令 (例如 RISC-V 扩展指令集、特权指令)、寄存器访问、上下文切换、中断处理、内存分配 (alloc / global allocator) / 管理 (heap / stack / 静态 / 动态) 等基础机制, 从而让 Rust 写的系统 / 驱动 / 应用代码能够正确运行、管理硬件资源、与外围设备交互。最终目标是证明: “Rust + RISC-V + 本研究构建的 BSP/运行时”组合, 足以支撑真实系统的开发 (从最底层 bootloader / kernel / runtime 到外围驱动和上层应用), 并具备安全性、性能、可维护性优势。

展示 Rust 在 PolyOS Server 中作为系统语言的可行性, 为后续的系统软件 (OS kernel, hypervisor, 嵌入式 RTOS, 驱动库) 在 RISC-V 上使用 Rust 提供参考实施路径与工程经验。

## 研究方案

研究方法分多阶段进行, 从工具链准备、BSP 与运行时构建, 到生态验证与性能/安全测试。方案力图兼顾系统正确性、可复现性、工程化、以及长远可

维护性[32]。

首先是工具链与基础环境准备。鉴于目标架构为 RV32 / RV64 带扩展，需要选取或构建适合该架构的 LLVM backend / binutils / linker / 汇编器 / C/C++ runtime support (libc / libcore / liballoc / libstd / panic runtime / allocator / 裸机替代库)。与此同时，需要确保 rustc 和 cargo 能够交叉编译 (cross-compile) 出符合 target triple (如 riscv64gc-unknown-linux-gnu、riscv64imac-unknown-none-elf、甚至 riscv32imac-unknown-none-elf 等) 的二进制。为此，方案准备一个交叉编译 toolchain: 构建 Rust cross-compile 环境 (安装 rustup / rustc / cargo, 添加 RISC-V 目标 triple, 配置 linker / 编译器 / sysroot / 库)。

第二阶段为 BSP 与运行时构建。选择一种 RISC-V 开发板 (如 HiFive、StarFive 或其他社区或商业服务器) 作为实验目标。针对该板卡: 编写或改造启动 (bootloader)、硬件初始化 (时钟、中断、内存控制器、外围总线)、异常/中断处理、中断向量表、物理内存映射 (MMU / 页表 / 分页可选)、堆栈 / 静态 / 数据段布局。然后构建 Rust 驱动库 (crate) —— 为外围设备 (UART / console / 串口, GPIO, 时钟, Timer, 中断控制器, 基本设备 I/O, 可能的网络或存储接口) 提供封装, 并在裸机环境 (no\_std / no\_alloc 或 minimal allocator) 下实现驱动 + 系统初始化 + 简单应用 (例如 printf / serial 输出, blinky, 中断计时, 简单 shell / 命令行, 或者更复杂一点的 I/O 驱动测试)。针对 RV32 和 RV64 两种 target, 都分别构建测试版本, 以验证 Rust 编译器与运行时在不同 ISA 宽度下的兼容性。

第三阶段是系统级构建与集成测试。不仅限于裸机 demo, 还尝试构建一个小型 runtime 或轻量级内核 / hypervisor, 使用 Rust 完成核心调度、内存管理、上下文切换、中断处理等功能 (可以参考社区已有 Rust-based OS / hypervisor 项目作为模板), 并在该 runtime 上运行简单应用 (例如多线程、外设驱动、文件系统、网络) 以验证系统可行性与稳定性。

第四阶段为安全性 / 性能 / 可维护性评估。对构建出来的系统 / 驱动 / runtime 进行静态分析 (Rust borrow / ownership / compile-time 检查 + unsafe 使用审查)、动态测试 (stress test, 并发测试, I/O correctness, 中断 / 异常 robustness)、性能测试 (启动时间、中断响应延迟、上下文切换开销、驱动延迟、I/O 吞吐) 等。记录并分析 Rust 在 RISC-V 架构 (32 / 64 位) 下是否存在性能或

安全短板 (如对齐问题、内存模型、Atomics / 原子操作 / 内存屏障 / 同步语义、生成代码质量、链接器 / runtime overhead 等)。

### 研究结果

研究在工具链适配、BSP 构建、系统运行、以及安全 / 性能评估等方面均有重要突破。首先, 在工具链和基础环境层面, 为 PolyOS Server 提供了 Rust 工具链, 通过 rustup 添加 riscv64gc-unknown-linux-gnu 和 riscv64imac-unknown-none-elf 等 target, 配置适当 linker 与 sysroot, 使得 Rust 编译器 (rustc) 能生成 riscv64 二进制。

在 BSP 与运行时构建阶段, 在 RISC-V 开发板完成了 bootloader + 最简 Rust runtime + 外设驱动 (UART / 串口 console + 基本 timer / interrupt) 的实现。基于 no\_std / minimal runtime 的 Rust 程序能够在裸机上正确启动、输出 console 信息, 并响应简单中断或 timer 事件。通过对 RV32 与 RV64 版本的测试, 确认了 Rust 编译器对两种架构宽度的支持基本稳定, 代码生成、汇编、链接、执行均正常, 没有出现明显对齐 / ABI / 函数调用语义错误。

在系统级构建与集成测试方面, 我们在 PolyOS Server 上提供了完整的 Rust 工具链。

## 3.3. Golang

### 研究内容

本研究围绕 Golang (Go 语言) 在 RISC-V 架构上的编译体系、运行时机制以及在 PolyOS Server (基于 openEuler 24.03 LTS RISC-V) 的工程化落地展开, 重点关注 Go 编译器工具链 (cmd/go、cmd/compile、cmd/link)、运行时系统 (runtime 包)、调度器 (M/P/G 模型)、垃圾回收器 (GC) 的可移植性与性能表现。Go 以其简洁的语法、高性能原生代码生成、并发调度能力以及强大的工具链体系 (build / test / modules) 被广泛用于云原生、中间件、系统工具与分布式系统, 是现代服务器系统的支柱语言之一。PolyOS Server 的各组件——包括容器管理、网络代理、监控系统、编排工具、服务网格等——都大量依赖 Go 实现, 因此 Go 在 RISC-V 上的可用性与性能直接关系到整个服务器软件栈的可部署性。

从上游发展来看, Go 在近几个版本 (Go 1.18~1.22) 中逐步完善了对 RIS

C-V 的支持，尤其是 RV64GC 作为主要 Linux 服务器版 RISC-V 架构，已经被 Go 官方工具链作为 Tier-2 级支持加入主线。在 openEuler 24.03 RISC-V 上，系统默认仓库提供了 go 1.20/1.21 系列的完整发行版包（go、golang-bin、golang-src 等），并且工具链能够在本地 RISC-V 系统上完成自举构建，说明体系结构支持已经达到可用于服务器应用的成熟度。本研究的重点在于：不仅验证 Go 语言在 RISC-V 上的正确性，还要评估其并发调度器的行为是否与 x86\_64 / ARM64 一致、GC 机制在 RISC-V 内存模型上是否稳定、模块构建与交叉编译是否可靠、PolyOS Server 实际组件能否在 RISC-V 上顺利构建与运行，并进一步研究高性能服务器负载下 Go 运行时的行为特征。

总的来说，本研究旨在构建一个能够在 PolyOS Server 上可靠运行、性能可接受、能够编译云原生基础设施和系统工具链的 Go 语言环境，使得 RISC-V 服务器平台能够无缝承载现有 Go 生态，并探索未来在 RISC-V 上进行语言级、调度器级和 GC 优化的可行空间。

## 研究方案

首先在工具链支持方面，需要验证 openEuler 24.03 提供的 Go 包在 RISC-V 架构上是否能够正确构建用户态应用，并确认 system go 与 upstream go 的兼容性。由于 Go 运行时对体系结构高度敏感，特别是 goroutine 调度器、atomic 操作、stack splitting、syscall ABI 绑定等都依赖 RISC-V 特定指令，因此研究方案首先对 Go 编译器生成的 RV64GC 指令序列进行分析，包括函数调用规约、栈帧布局、协程切换路径、gcWriteBarrier 与内存屏障实现，并对比 x86\_64 / ARM64 行为验证一致性。为了进一步验证可重复构建能力，方案将通过 Go 的 source bootstrap 流程（即使用 go bootstrap tool 自举构建新的 go 二进制），确保 PolyOS Server 能够在纯 RISC-V 环境下完成 Go 的全链路构建。

在工程化构建方面，研究方案选择 PolyOS Server 中大量依赖 Go 的基础工具作为系统验证对象，包括 container 管理工具、网络代理、CLI 工具、监控栈组件、包管理系统辅助工具等。通过在 PolyOS RISC-V 上执行 go build / go test / go mod vendor 等全流程，验证构建工具链稳定性，分析构建期间的内存占用、CPU 使用情况、并发任务执行效率等，并结合 PolyOS 的包管理机制验证 RPM 打包流程是否完整，能否正确生成 includes、binaries、source 包，

以及是否符合系统统一的 FHS 布局要求。同时，研究方案会通过实际部署小规模 Go 服务，观察其在 RISC-V 上的运行行为，包括 goroutine 调度延迟、GC pause time、系统调用延迟以及内存分配模式等，以评估运行时是否达到服务器级可用性。

在运行时深度验证方面，研究方案通过 microbench、GOMAXPROCS、stress-ng、Go 官方 benchmark（如 hash/map、json、regexp、goroutine creation）等测试工具，构建一套性能评估体系，重点关注调度器在 RISC-V 上的 M/P/G 映射效率、atomic 操作在 RV64GC 上的性能、批量 goroutine 创建的开销、Go GC 的标记与清扫阶段延迟、以及 syscall 在 RISC-V Linux 上的往返代价。由于 Go 的 GC 与调度器都高度依赖内存模型，因此方案将特别关注 RISC-V 的弱内存模型对 Go runtime 的影响，包括写屏障、栈扫描的同步语义、中断点 (safe point) 布置效率等。

在生态兼容性层面，方案选取 PolyOS Server 中典型的 Go 软件（如 etcd、Prometheus、containerd、helm、cni 工具、Kubernetes 生态部分组件）进行构建与运行测试。这些软件对 goroutine 调度、网络 I/O、大对象分配、并发数据结构等均有显著压力，是验证 Go 在 RISC-V 服务器应用场景真实可用性的关键。通过系统化测试，可进一步分析 PolyOS RISC-V 是否具备承载完整 Go 服务端生态的能力，并为未来优化提供参考依据。

## 研究结果

在工具链适配方面，openEuler 24.03 LTS 提供的 Go 工具链能够在纯 RISC-V 环境中正常执行编译、链接、打包等操作，且可通过 upstream bootstrap 流程完成从源代码自举构建。实际测试表明，Go 对 RISC-V RV64GC 的支持已经达到工程可用水平：goroutine 调度器能够正常切换，atomic 操作序列生成正确，栈增长机制 (stack growth / split stack) 行为稳定，运行时对 RISC-V Linux 的 syscall 调用路径符合预期，没有出现 ABI 不一致或不支持的情况。此外，在 go build / go test 流程中，所有常用模块 (net/http、crypto、sync、runtime 等) 均可通过测试，说明 Go runtime 的基础语义与调度机制能够在 RISC-V 上正确运行。

在系统工程化方面，PolyOS Server 上的大量 Go 软件能够被完整构建并运行，包括 CLI 工具、容器管理程序、网络代理、系统监控工具等。在构建过程

中，各模块能够使用 `go modules` 正常拉取依赖、完成静态或动态链接构建，最终的二进制文件可以在 RISC-V 上稳定运行。测试结果表明，在 PolyOS 的构建链路中加入 Go 工具链不会产生额外的架构兼容性问题，也不会对系统 RPM 构建规范造成破坏。构建压力测试显示，Go 的并行构建能力在 RISC-V 上表现稳定，资源消耗与 x86\_64 平台相比保持合理线性关系。

在运行时行为评估中，Go 在 RISC-V 上的性能表现达到了可在服务器环境部署的标准。通过 `microbench` 与 `stress test` 观察，`goroutine` 创建和上下文切换开销处于合理水平；GC 的 `Mark` 阶段与 `Sweep` 阶段延迟稳定，没有出现明显的 JIT-free 语言常见的 `stop-the-world` 波动；`atomic` 操作和锁竞争行为与 RISC-V 内存模型契合良好，没有观察到错误的重排序导致的死锁或数据错误。网络 I/O 延迟、事件驱动（`epoll`）性能、并发哈希表操作性能均达到实际可部署的水平，适用于云原生与中间件软件。

在生态软件兼容性方面，研究成功在 PolyOS Server 上构建了若干典型的 Go 服务器组件，并对其执行真实运行负载测试。结果表明，`etcd`、`containerd`、`cni` 工具链、`Prometheus` 等系统级服务在 RISC-V 上均可启动、可运行、可响应；其 `goroutine` 调度、内存占用、I/O 性能与上游行为保持一致，未出现结构性不兼容或内部运行错误。综合分析表明，PolyOS Server 在 RISC-V 上完全具备部署完整云原生 Go 生态的能力，为构建更大规模服务（如 Kubernetes 控制面核心组件）奠定了基础。

本研究验证了 Go 语言在 RISC-V 服务器平台上的成熟度，证明 PolyOS Server 已经达到可在生产环境中运行 Go 服务端应用的工程水平。Go 语言具备的高并发能力、严格的内存安全语义、优异的部署特性，与 RISC-V 的开放指令集设计理念高度契合，为未来在 RISC-V 上构建云原生系统、边缘服务与系统工具链提供了坚实基础。

### 3.4. JavaScript

#### 研究内容

围绕 JavaScript 在服务器侧的核心运行时 Node.js，在 RISC-V 架构上实现可用、可部署、可维护运行环境展开，重点关注 Node.js 在 PolyOS Server（基于 openEuler 24.03 RISC-V）的构建能力、运行时行为、JIT 编译器可移植性、

生态工具链兼容性以及在典型服务器场景中的工程化落地。JavaScript 作为最广泛应用的脚本语言之一，其服务器端运行高度依赖 Node.js 生态。Node.js 运行时采用 Google V8 引擎提供 JavaScript 执行能力，并结合 libuv 事件循环模型，实现了高吞吐 I/O、异步回调及微任务调度机制，广泛用于 Web 服务、构建工具、API 网关、中间件层、运维脚本以及现代 DevOps 工具链。

由于 V8 的 JIT 引擎（TurboFan、Ignition）具有强烈的体系结构依赖性，Node.js 在新架构上的可移植性要求必须得到充分验证。针对 RISC-V 生态的快速发展，Node.js 上游社区在近两年逐步增加对 RISC-V 的关注，使得 Node.js 18+、20+ 在 Linux/RISC-V 环境中具备基本可编译性。但由于 JIT 与 Inline Caches、优化器、栈帧布局、内置对象实现等均具有大量汇编与平台相关代码，其在 RISC-V 上的行为仍需系统验证。

PolyOS Server 作为以 openEuler 24.03 RISC-V 为基础构建的服务器操作系统，需要大量依赖 Node.js 的软件工具链，包括前端构建系统（webpack、vite、rollup）、包管理器（npm、pnpm、yarn）、DevOps 工具（eslint、prettier）、Web 服务框架（Express、Koa、NestJS）以及静态资源服务、API Gateway 等。因此 Node.js 在 RISC-V 上的可用性直接影响整个软件栈的可部署性。本研究的目标是通过体系结构层面的验证、运行时特性的分析以及生态兼容性测试，构建一个适配于 PolyOS Server 的、高可用、可调优的 Node.js 运行时基础。

## 研究方案

在工具链层面，需要在 PolyOS Server 的 RISC-V 环境中完成 Node.js 的源码构建。Node.js 构建链依赖 llvm、binutils、python3、libuv、openssl、zlib 等基础组件，因此必须先验证 PolyOS Server 的底层库版本是否满足构建需求。研究方案通过从 Node.js LTS 源码（如 Node 20.x 或 18.x）进行全流程编译，观察编译器生成的 RV64GC 指令序列，确认 V8 中的汇编模板、builtin 函数、内联缓存（IC）路径是否能够正确完成编译与链接。对于 V8 中仍依赖 JIT 禁用模式的部分组件，研究将明确界定其行为是否会影响服务器端实际场景，并分析启用或禁用 JIT 时的性能变化。

在运行时验证方面，研究方案重点关注 V8 在 RISC-V 上的 JIT 编译器行为。JIT 生成代码的寄存器分配、堆栈布局、垃圾回收 write barrier、优化 pass（如 type feedback、inlining）都需要验证其在 PolyOS 的 RV64GC 环境下行

为与预期一致。特别是 Node.js 的事件循环模型（基于 libuv）、Promise 微任务调度、异步 I/O 等机制，需要通过大量并发测试验证是否存在调度延迟异常、事件循环阻塞或与架构相关的系统调用差异。此外，研究将使用 benchmark 工具（如 autocannon、wrk、Node.js 官方 benchmark suite）测试 HTTP 服务器、文件 I/O、异步任务调度等常见路径的性能。

工程化整合方面，本研究方案评估 Node.js 在 PolyOS Server 中的可部署性，包括 npm/pnpm 包管理器能否在 RISC-V 上正确运行、是否可构建常见依赖的 node-gyp 原生模块、前端构建工具在 PolyOS RISC-V 上是否具备完整功能。由于许多 npm 包依赖 C/C++ native addons（如 bcrypt、canvas、grpc、sharp 等），研究方案通过 node-gyp 在 RISC-V 环境中编译这些模块，验证 PolyOS 的工具链（gcc/llvm）与 Node.js 的 ABI 是否保持一致。

## 研究结果

本研究成功在 PolyOS Server 上构建了 Node.js 运行时，并验证了 PolyOS Server 服务器软件生态中 JavaScript 软件的可构建性与运行时基础运行支撑能力。

# 4. PolyOS Server 操作系统发行版

## 4.1. PolyOS Server 操作系统构建

### 4.1.1. 研究内容

PolyOS Server 是基于 openEuler 24.03 LTS 的一型操作系统发行版，为 RISC-V 架构的数字基础设施提供初步的受长期支持的操作系统解决方案，以满足服务器场景应用的需求，确保在复杂应用场景下的稳定性、安全性和可持续性。

PolyOS Server 在 openEuler 24.03 LTS 的基础上进行深度优化，结合了 RISC-V 硬件的特性与服务器应用需求，构建了自下而上的完整软件栈，包括 RISC-V 硬件、openEuler 操作系统、服务器操作系统模块等关键层级。通过这种分层设计，系统能够有效支持不同应用层级的需求，实现软硬件的紧密耦合和高效适配。

通过对生态链的全面构建，该系统在增强 RISC-V 生态的同时，提供了一

个开放的服务器操作系统平台，促进 RISC-V 架构在服务器领域的进一步应用与发展。

#### 4.1.2. 研究方案

PolyOS Server 的构建过程采用了高效、可靠的自动化构建工具，并依托 Open Build Service (OBS) 平台，实现了构建任务的全程自动化和并行处理。在此过程中，OBS 平台发挥了关键作用，其配备的多个 `obs-worker` 节点构建集群，以分布式架构实现任务的高效分配和处理。该集群包含四个专门配置的 `obs-worker` 节点，具备强大的并行计算能力，显著提升了构建的速度和稳定性，最大限度地降低了资源瓶颈和单点故障的风险。

OBS 平台作为构建集群的核心，包含了四个 `obs-worker` 节点，这些节点协同工作，构成了一个高度可靠和扩展性强的构建环境。通过分布式架构和任务的并行化处理，该集群能够迅速分配和完成大规模的软件包构建任务。这种分布式处理模式不仅缩短了构建时间，也提升了任务完成的稳定性，确保了系统的构建流程不会因单一节点故障而中断。OBS 架构的采用使得 PolyOS Server 能够在高度自动化的条件下完成复杂的软件包集成。

PolyOS Server 的构建过程可以分为三个主要阶段：软件包的初始构建、重建以及集成发布。

在初始阶段，PolyOS Server 从 openEuler 24.03 LTS 的基础软件仓库中，提取并构建了 4818 个软件包。这些软件包涵盖了操作系统的核心组件，如内核、系统库、工具链，以及其他关键支持软件。在构建过程中对所有软件包进行了严格的自动化测试和兼容性验证，确保它们在不同的硬件和软件环境中能够稳定运行。

在初始构建完成后，PolyOS Server 进入了重建阶段。在此阶段，系统对构建成功的软件包进行复查和重建，以提高其整体的稳定性和性能。通过多次构建和自动化测试，系统排除了潜在的不兼容问题，并确保生成的 RPM 软件包满足数据中心的应用需求。

在集成阶段，PolyOS Server 将所有构建和重建成功的软件包集成至标准软件仓库中。最终，生成了总计 18508 个 RPM 软件包，涵盖了从系统管理工具

到开发工具、库依赖和系统管理工具等多种基础组件。用户可以通过官方软件仓库访问这些 RPM 包，从而获取广泛支持的 RISC-V 平台应用。

在构建过程中，各阶段的工作分工明确。首先是从 openEuler 仓库提取、构建初始版本的软件包，其次是对构建任务进行多节点并行处理，确保高效构建。此后，项目团队对所有初步完成的软件包进行逐一重建，剔除存在潜在问题的软件，提升其可靠性。OBS 平台的自动化测试功能在整个过程中发挥了不可或缺的作用，通过多次构建和测试，使得构建的 RPM 包具备更高的适用性。

此外，在构建的每个关键阶段中，项目团队都会对构建结果进行总结和监控，以确保每个软件包在经过 OBS 平台处理后都能符合质量标准。通过这种层层优化和多次复查，使得 PolyOS Server 的构建流程实现了高效和高质的双重保证。

### 4.1.3. 研究结果

作为 PolyOS 的最终分发形式，该操作系统以 ISO 镜像作为分发与交付的格式。首个正是版本针对 SG2042 系列開發板进行了优化，集成了 UEFI（统一可扩展固件接口）引导功能，确保在各类硬件平台上都能实现稳定、快速的启动和安装。在用户体验方面，该版本镜像提供了安装界面，使安装过程更加直观、易用。用户可以通过安装界面选择安装选项、配置系统参数和分区设置，大大简化了原本复杂的命令行操作，使操作更加友好和高效，提升了整体安装流程的自动化程度和部署效率<sup>[33]</sup>。

ISO 镜像是实现广泛硬件平台支持的重要环节，通过这一镜像，用户能够轻松实现操作系统的快速部署，无论是用于开发、测试还是生产环境。

#### 硬件准备

算能设备 1 台，或主板（及必需外设） 1 套

m.2 NVMe 固态硬盘 1 块

若原先有数据则需格式化清除（个人资料请注意备份）

若有 PCIe 转接卡，则可通过转接卡置于设备 PCIe 第一槽位（较推荐）

若无 PCIe 转接卡，则可置于板载 NVMe 接口

U 盘 1 个 大小应为 16GiB 以上。

microSD 卡 1 张大小应为 4GiB 以上

串口操作所需设备/组件

RJ45 网线 1 条以上及路由器/交换机等设备供有线网络连接使用

推荐使用设备板载 RJ45 网口而非厂家附送的 PCIe 电口网卡。

## 镜像种类

ISO 镜像支持 UEFI 方式启动，对应下文的 UEFI 版固件。

将 PolyOS Server 烧录至 U 盘中即可。推荐使用 Balena Etcher 软件进行图形化烧录 [从 <https://github.com/balena-io/etcher/releases/latest> 下载], 烧写过程此处略过

命令行环境下也可采用 dd 方式进行烧录，参考命令如下：

Plain Text

```
~$ sudo dd if=polyos-server.iso of=/dev/sdb bs=512K iflag=fullblock oflag=direct conv=fsync status=progress
```

## 设备固件

从官网下载设备 UEFI 固件解压后烧录其中 img 文件至 microSD 卡

Plain Text

```
~$ sudo dd if=firmware.img of=/dev/sdb bs=512K iflag=fullblock oflag=direct conv=fsync status=progress
```

算能设备通过镜像安装 PolyOS Server 系统前请将 microSD 插入设备卡槽中

## 启动前检查和准备

ISO 版本使用者需检查：

装载有固件的 microSD 卡是否插入设备卡槽中。

UEFI 版固件目前尚无法手动调整或指定启动顺序，敬请谅解。

如硬盘上存在另一可启动的 RISC-V 操作系统，需卸下固态硬盘进行格式化，以避免启动顺序上的干涉。

## 4.2. RISC-V 硬件适配

### 4.2.1. 香山处理器适配

#### 研究目标

香山是一款由中科院计算所研发的开源双核 RISC-V 处理器核，是一款通用芯片，支持 TileLink Crossbar 和 L3 Cache，支持 DDR4，支持的外设接口包含显示控制器、HDMI、PCIe RC、USB3.0、GMAC、SDHC、QSPI、I2S 等。

本研究在香山处理器的硬件上实现操作系统移植，构建适用于面向香山的 PolyOS Server 操作系统版本，并在其基础上做软件开发和适配<sup>[34]</sup>。

#### 研究方案

果壳处理器芯片是单发射顺序核，RV64IMAC 指令集，支持 M/S/U 特权级；两位饱和计数器分支预测；512 项 BTB，16 项 RAS；支持 Sv39 分页机制，支持硬件 TLB 填充；32K 指令/数据 L1 cache；支持 L1 指令/数据 cache 读一致性；128K L2 cache，支持 next line 预取；使用 Chisel 语言开发。

利用果壳开发板适配面向 RISC-V 的 openEuler 操作系统版本，启动 PolyOS Server 操作系统，通过 BBL 引导 linux 内核正常启动；linux 内核启动之后，挂载文件系统；文件系统启动后，命令行可以正常执行；涉及以下关键技术：（1）、多样化的 RISC-V 处理器开发板，包括处理器和各个外围接口。（2）、不同 RISC-V 硬件平台的 Linux 操作系统引导启动过程。（3）、对 Linux 操作系统内核和各个外设接口驱动的适配。（4）、Linux 操作系统内核挂载文件系统。

果壳开发板适配面向 RISC-V 的 openEuler 操作系统版本应满足以下功能：

- （1）、BBL 引导 linux 内核正常启动；
- （2）、linux 内核启动之后，挂载文件系统；
- （3）、文件系统可以正常启动，命令行可以正常执行；

PolyOS Server 操作系统适配果壳开发板的流程逻辑如下：果壳开发板操作系统引导启动方式采用 BBL 进行第一阶段启动，然后第二阶段启动 uboot。一旦 uboot 启动成功，就会引导 Linux 内核的启动。在 Linux 操作系统内核启动之后，能够加载文件系统。

#### 研究方法和过程

构建面向香山处理器的 PolyOS Server 操作系统版本的流程具体步骤如下：

步骤 1：编写支持 glibc 的交叉编译工具链的集成构建环境配置文件。

步骤 2：编写支持 RISC-V 处理器和基础 C 库 glibc 的 BBL 的集成构建环境配置文件。

步骤 3：使用集成构建工具进行自动构建。

步骤 4：如果 BBL 或 D1 开发板引导程序构建成功，则执行步骤 5；否则，回到步骤 2，修改 BBL 的集成构建环境配置文件。

步骤 5：编写支持 RISC-V 处理器和基础 C 库 glibc 的 U-Boot 的集成构建环境配置文件，对不同指令集的 RISC-V 处理器、不同的接口硬件进行支持。

步骤 6：使用集成构建环境进行自动构建。

步骤 7：如果 U-Boot 构建成功，则执行步骤 8；否则，回到步骤 5，修改 U-Boot 的集成构建环境配置文件。

步骤 8：编写支持 RISC-V 处理器和基础 C 库 glibc 的 Linux 内核的集成构建环境配置文件，增加对不同指令集的 RISC-V 处理器，不同的硬件接口进行支持。

步骤 9：使用集成构建工具自动构建。

步骤 10：如果 Linux 内核构建成功，则执行步骤 11；否则，跳转到步骤 8，修改 Linux 内核的集成构建环境配置文件。

步骤 11：编写支持 RISC-V 处理器和基础 C 库 glibc 的文件系统的集成构建环境配置文件。

步骤 12：使用集成构建环境自动构建。

步骤 13：如果基本文件系统构建成功，则执行步骤 14；否则，回到步骤 11 修改对应的文件系统的集成构建环境配置文件。

步骤 14：自动构建完成后，生成.wic.gz 格式的压缩包，里面包含了构建好的 BBL 引导程序，U-Boot，Linux 内核和基本的文件系统。

步骤 15：在主机上将上述压缩包烧写到 SD 卡的 3 个分区中，其中文件系统单独占有一个 SD 卡分区。

步骤 16：将 SD 卡插入到对应的 RISC-V 开发板并打开电源。

步骤 17：BBL 引导程序、U-Boot 和 Linux 操作系统内核能够正常启动，能够挂载文件系统，并登陆进入操作系统控制台 [\[35\]](#)。

## 研究结果

为了验证移植和适配方案的可行性，确保研究结果的正确性和可重复性，首先需要明确运行环境和所需的软件包等。其次，需要对运行环境进行部署和维护。最后展示研究结果。

1、所需的运行环境

PolyOS Server 编译环境需要 32\64 位 Linux 操作系统平台，cpu 单核心奔腾 4 以上，内存 1GB 以上。

果壳开发板适配需要果壳处理器，CUP 配置：hart: 0、isa: rv64imafdc、mmu: sv39、uarch: UCAS,COOSCA1.0

2、运行时所需的软件包

软件包	功能描述
fw_payload_oe.elf	利用 openSBI 将 linux 内核的 image 作为 payload 所制作的用于 QEMU 启动的 kernel image
openEuler-preview.riscv64.qcow2	包含 openEuler RISC-V 移植版的 rootfs 的虚拟机磁盘镜像
RV_BOOT.UCAS_COOSCA1.0_V1.BIN	移植支持 NutShell UCAS COOSCA1.0 的内核和 BBL 打包文件
oe-UCAS_COOSCA1.0-rootfs.v1.tar.gz	移植适配在 NutShell UCAS COOSCA1.0 运行的根文件系统.
qemu	适用于 riscv64 的模拟器
debian	Debian 是指一个致力于创建自由操作系统的合作组织及其作品，由于 Debian 项目众多内核分支中以 Linux 宏内核为主，而且 Debian 开发者 所创建的操作系统中绝大部分基础工具来自于 GNU 工程
alpine	Alpine 操作系统是一个面向安全的轻型 Linux 发行版。它不同于通常 Linux 发行版，Alpine 采用了 musl libc 和 busybox 以减小系统的体积和运行时资源消耗，但功能上比 bus

	ybox 又完善的多, 因此得到开源社区越来越多的青睐
GeoIP-debugsource-1.6.12-5.riscv64.rpm	适用于 riscv64 的 PolyOS Server 的 RPM 包
U-boot	引导操作系统内核启动, 初始化硬件设备。
Linux 内核源码包	Linux 操作系统内核, 控制和操作硬件
Busybox	嵌入式文件系统制作工具包
Riscv64-linux-glibc-cross	交叉编译工具链, 对目标应用程序进行交叉编译。
BBL	RISC-V 二进制程序接口, 引导操作系统。

## 4.2.2. 智能协处理器适配

### 研究内容

本研究围绕将国产智能协处理器寒武纪 MLU370 与 RISC-V 服务器平台进行深度集成展开, 重点关注 PolyOS Server 上的硬件适配、驱动系统移植、核心软件栈与智能计算框架的重构工作。随着 RISC-V 架构在服务器端的发展加速, 实现智能协处理器与 RISC-V 服务器的互联互通成为构建新一代智能计算节点的关键步骤。MLU370 作为国产高性能 AI 加速卡, 其在传统 x86/ARM 服务器平台上已具有较成熟的软件栈, 但在 RISC-V 平台上仍缺乏系统性的适配和验证, 因此需要面向 PolyOS Server 进行完整的软件迁移与硬件抽象层重构。

本研究的核心内容分为三个部分。第一部分为硬件环境对接适配, 即在 RISC-V SoC 或 FPGA 服务器平台上提供对 MLU370 的物理接入能力, 包括 PCIe 控制器配置、拓扑识别、设备枚举以及内存映射机制的支持, 使得 PolyOS Server 能够在系统启动流程中识别并管理该协处理器。第二部分为国产智能芯片驱动适配, 需要将 MLU370 的驱动层 (内核态与用户态组件) 迁移到 PolyOS Server 的 RISC-V 内核环境, 确保 DMA 引擎、命令队列、中断处理、任务调

度通道等关键功能能够与 RISC-V 平台稳定协作。第三部分为核心加速库及智能计算框架的适配工作，根据 MLU370 提供的 API/Kernel Runtime，将计算图执行引擎、算子库、内存分配器、任务调度模块等进行适配，使其能够在 PolyOS Server 上实现端到端的 AI 任务执行能力。构建一条可在 RISC-V 服务器平台上使用国产智能协处理器的完整软件路径，使 PolyOS Server 具备部署 AI 推理和训练任务的能力，为未来构建基于 RISC-V 架构的人工智能服务器奠定技术基础<sup>[36]</sup>。

## 研究方案

本研究确保整个适配链条从底层到上层均具备一致性与可验证性，聚焦于寒武纪 MLU370 与 PolyOS Server 所运行的 RISC-V 硬件平台之间的硬件对接。由于 MLU370 通过 PCIe 接口连接，首先需要验证 RISC-V SoC 或 FPGA 平台的 PCIe 控制器与拓扑结构是否能够支持 MLU370 的链路训练、BAR 空间映射、中断线配置以及 IOMMU 地址转换等能力。研究方案通过 PolyOS Server 的硬件抽象层提供稳定的 PCIe 设备识别、配置与管理能力，使 MLU370 能够在系统初始化流程中被正确枚举与激活。此外，还需验证 DMA 传输带宽、链路稳定性以及多功能端点管理等关键功能。

本研究也包含驱动系统的适配，MLU370 的驱动系统包含内核态模块（与内核 PCIe 子系统、IOMMU 层交互）和用户态控制库（负责任务提交、内存管理、命令调度等），其原生开发环境通常基于 x86/ARM 内核，需要针对 RISC-V 进行移植与差异处理。方案需要研究 PolyOS Server 的 Linux 内核（openEuler 24.03 RISC-V 内核版本），完成驱动代码的体系结构抽象层适配，包括：中断处理模型（特别是 MSI/MSI-X）、页表与 IOMMU 映射接口、DMA API 差异、内核同步原语行为，以及与 RISC-V 内存模型相关的原子操作与屏障指令的替换或调整。用户态库部分需要重新构建，使编译器与链接器能够在 RISC-V 上成功生成执行文件，并保证 ABI 一致性。

最终，本研究将寒武纪的核心加速库与智能计算框架在 PolyOS Server 上进行重构。MLU370 的软件栈通常包括算子库（如 CNN、矩阵运算、激活函数等）、任务调度器、图编译器、runtime kernel API 等，这些组件依赖于底层驱动进行高性能执行。在 RISC-V 环境中，需要根据 MLU370 提供的 API 重新适配调度器的线程模型、内存管理策略、运算 kernel 加载机制等，使其可以在

PolyOS Server 上无缝执行。对于图执行引擎和算子库，需要评估是否存在与 RISC-V 用户态 ABI、编译器行为或对齐规则相关的差异，并进行针对性修复。

## 研究结果

本研究在 PolyOS Server 平台上完成了寒武纪 MLU370 智能协处理器与 RISC-V 服务器体系结构的系统性适配与集成，研究结果表明该国产 AI 加速卡能够在 RISC-V 架构环境中实现稳定识别、驱动加载与计算能力调用，验证了 RISC-V 服务器与国产智能加速器协同工作的工程可行性。

在硬件与系统层面，研究结果显示 MLU370 可通过 PCIe 接口被 PolyOS Server 正确枚举和管理，系统在启动阶段能够完成链路训练、BAR 空间映射和设备初始化，PCIe 拓扑结构与地址空间分配符合预期。DMA 传输、中断响应及多队列机制在 RISC-V 平台下运行稳定，未出现架构相关的异常行为，表明 PolyOS Server 的 PCIe 子系统与 IOMMU 框架能够为智能协处理器提供可靠的硬件支撑环境。

在驱动系统层面，本研究成功完成了 MLU370 内核态驱动在 RISC-V Linux 内核上的移植与加载，相关内核模块能够正常完成设备探测、资源申请、中断注册以及 DMA 映射操作。针对 RISC-V 架构特性所进行的内存屏障、原子操作和同步机制适配未引入功能性回归，驱动在持续运行和压力测试场景下保持稳定，表明 MLU370 的驱动体系可以与 PolyOS Server 的内核执行模型良好协同。同时，用户态驱动控制库与运行时接口在 RISC-V 平台上成功完成编译和链接，ABI 行为与预期一致，为上层计算框架提供了完整、可调用的接口基础。

在软件栈与开发库层面，本研究验证了寒武纪核心加速库与运行时组件在 PolyOS Server 上的可用性。包括基础计算 API、内存管理接口以及任务提交与同步机制在内的关键功能均能够正常工作，算子加载与执行流程可通过驱动路径完成端到端调用。开发者能够在 RISC-V 服务器环境中使用原有的编程模型与接口对 MLU370 进行调度与计算，从而实现 AI 推理任务的实际运行。这一结果表明，MLU370 的开发库与运行时并未对底层 CPU 架构形成不可逾越的绑定，其软件体系具备向 RISC-V 平台扩展的现实基础。

总体而言，本研究实现了寒武纪 MLU370 在 RISC-V 服务器平台上的成功适配，完成了从硬件识别、驱动加载到核心开发库可用的完整链路验证。研究结果不仅证明了 PolyOS Server 具备承载国产智能协处理器的软件基础能力，也

为在 RISC-V 架构上构建可用、可扩展的 AI 计算节点提供了工程样板，为后续开展多卡调度、性能优化及 AI 框架深度集成奠定了坚实基础。

## 参考文献

- [1] RISC-V International. RISC-V Server SoC Specification: Version 1.0[S]. 2025.
- [2] RISC-V International. RVA23 Profile Specification: Version 1.0[S]. 2024.
- [3] Boubakri M, Zouari B. A Survey of RISC-V Secure Enclaves and Trusted Execution Environments [J]. Electronics, 2025, 14(21): 4171.
- [4] 韩柳彤,张洪滨,邢明杰,等.面向 RISC-V 向量扩展的高性能算法库优化方法[J].软件学报,2025,36(9):3985-4005.
- [5] 屈晟.作为 RISC-V 服务器的 openEuler: 挑战和路线图[C].第五届 RISC-V 中国峰会,2023-07-18,上海.
- [6] Wu Y,Liu C,Zhao C,et al.RISC-V and O-CFI mechanism-based defense method and apparatus for code reuse attacks[P].US Patent 12,417,287,2025.
- [7] RISC-V 小组.2025 年 6 月 RISC-V 技术动态分享[EB/OL].知乎专栏(第 3 期),2025.
- [8] Heerekar,Balvansh;Philippidis,Cesar;Chuang,Ho-Ren;et al.Offloading Datacenter Jobs to RISC-V Hardware for Improved Performance and Power Efficiency[C].Proc. ACM SYSTOR 2024, 2024.
- [9] 李传东,衣然,罗英伟,汪小林,王振林.RISC-V 架构下的懒惰影子页表模型[J].软件学报, 2025, 36(9): 3970-3984.
- [10] Zhang,Yizhe;Yang,Zhengyi;Han,Bocheng;et al.RISC-V Meets RDBMS:An Experimental Study of Database Performance on an Open Instruction Set Architecture[C].VLDB ADMS Workshop 2025, 2025.
- [11] 王杰,王鹏.面向 RISC-V 平台的安全高效固件可信平台模块设计与实现[J].电子与信息学报, 2025, 47(7): 2385-2395.
- [12] Sá,Bruno;Martins,José;Pinto,Sandro.A First Look at RISC-V Virtualization from an Embedded Systems Perspective[EB/OL]. arXiv:2103.14951, 2021.
- [13] 芮志清,梅瑶,陈振哲,吴敬征,凌祥,罗天悦,武延军.SeChain:基于国密算法的 RISC-V 安全启动机制设计与实现[J].计算机研究与发展, 2024, 61(6): 1458-1475.
- [14] Gauchola,Javier;Costa,Juan J.;Morancho, Enric;Canal,Ramon;et al.Hypervisor Extension for a RISC-V Processor[C]. RISC-V Summit Europe 2023, 2023.

- [15]Boubakri,Marwen;Zouari,Bilel.A Survey of RISC-V Secure Enclaves and Trusted Execution Environments[J]. Electronics 14(21): 4171, 2025.
- [16]Waterman,Andrew;Lee,Yunsup;Patterson,David;Asanović,Krste.The RISC-V Instruction Set Manual, Volume II: Privileged Architecture,Version 1.12[S]. RISC-V Foundation, 2019.
- [17]Feldgoise,Jacob;Rasser,Martijn.RISC-V:What It Is and Why It Matters[R]. CSET Report, Georgetown University, 2020.
- [18]US Patent 12,153,944 B2 (2023).Systems,Methods,and Media for Trusted Hypervisors[P]. 2023.
- [19]龙蜥社区.2024 年 RISC-V 安全可信技术实践白皮书[R]. 2024.
- [20]Rodríguez,Manuel;Costa,Francisco;Sá,Bruno;Pinto,Sandro.Open-Source RISC-V IOMMU IP Compliant with the RISC-V IOMMU Specification[C]. RISC-V Summit Europe (Poster), 2023.
- [21]Aigner,Leopold;Pail,Matthias;Pirker,Martin;et al. SERVAS: Secure Enclaves via RISC-V Authentication Shield[C]. SecureComm 2023, 2023.
- [22]openEuler 社区.一个 Java 端的向量化 BLAS 库:VectorBLAS[EB/OL].openEuler 创新项目博客,2023.
- [23]Guojun Liu,Jili Huang,Xiaoyan Liu.An OS Kernel Based on RISC-V Architecture[C]. In: Proceedings of the 6th International Conference on Computer Science and Educational Informatization (CSEI 2023), 2024: 3 – 16.
- [24]Dayeol Lee,David Kohlbrenner,Shweta Shinde,Krste Asanović,Dawn Song. Keystone:An Open Framework for Architecting Trusted Execution Environments [C]. In: Proceedings of the 15th European Conference on Computer Systems (EuroSys 2020), 2020.
- [25]进迭时空.携手移动云实现业界首个 RISC-V IO 虚拟化方案[EB/OL].2023.
- [26]李拓,邹晓峰,林宁亚,张璐,刘同强,周玉龙,李仁刚.基于 RISC-V 的服务器管理控制器 FPGA 原型设计[J].计算机系统应用, 2021, 30(7): 136-141.
- [27]Vedran Dakić,Leo Mršić,Zdravko Kunić,Goran Đambić.Evaluating ARM and RISC-V Architectures for High-Performance Computing with Docker and Kubernetes [J]. Electronics, 2024, 13(17): 3494.
- [28]潇公子(作者).阿里平头哥基本完成主流操作系统与 RISC-V 的全适配 [EB/OL].IT 之家, 2023.

- [29]Brown,Nick;Jamieson,Maurice;Lee,Joseph K. L.Is RISC-V Ready for HPC Prime-Time? Evaluating the 64-Core Sophon SG2042 RISC-V CPU[C].Proceedings of HPC Asia 2024. 2024.
- [30]余子濠,陈璐,孙凝晖,包云岗.以 RISC-V 为目标的动态二进制翻译代码质量优化方法[J].计算机研究与发展, 2023, 60(10): 2322-2334.
- [31]Nick Brown,Maurice Jamieson,Joseph Lee,Paul Wang.Is RISC-V ready for HPC prime-time: Evaluating the 64-core Sophon SG2042 RISC-V CPU[C]. In: Proceedings of the SC ' 23 Workshops (SC-W 2023), 2023.
- [32]George-Marios Frangkoulis,Nikos Karystinos,George Papadimitriou,Dimitris Gizopoulos.Advancing Cloud Computing Capabilities on gem5 by Implementing the RISC-V Hypervisor Extension [C]. In: Proceedings of the 8th Workshop on Computer Architecture Research with RISC-V (CARRV 2024), 2024.
- [33].RISC-V IOMMU Task Group. RISC-V IOMMU Architecture Specification: Version 1.0[S]. 2023.
- [34]王凯帆,徐易难,余子濠等.香山开源高性能 RISC-V 处理器设计与实现[J]. 计算机研究与发展, 2023, 60(3): 476-493.
- [35].Sá B,Valente L,Martins J,Rossi D,Benini L,Pinto S.CVA6 RISC-V Virtualization: Architecture, Microarchitecture, and Design Space Exploration[EB/OL]. arXiv:2302.02969, 2023.
- [36].Martins J,Tavares A,Solieri M,Bertogna M,Pinto S.Bao:A Lightweight Static Partitioning Hypervisor for Modern Multi-Core Embedded Systems[C].Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2020). Open Access Series in Informatics (OASIs), Vol 77. 2020: 3:1-3:14.